

Gordion: Design and Implementation Notes

Version 2

Laurence G. Yaffe

Department of Physics, University of Washington, Seattle WA 98195, USA

July 24, 2026

Contents

1	Introduction	1
2	Overall structure	1
3	Program scope	2
4	Implementation considerations	4
4.1	Symbol encoding	4
4.2	Observable storage	5
4.3	Observable canonicalization	6
4.4	Commutator evaluation	7
4.5	Observable generation & classification	8
4.6	Observable approximation	11
4.7	Generator selection & normalization	12
4.8	Polynomial representation	14
4.9	Numerics	15
4.10	Multi-threading	17
4.11	Input and output files	19
4.12	Disk swapping	20
4.13	Resource estimates	21
4.14	Data structures	22
5	Programming assumptions & conventions	23
6	Compilation & linking	24
7	Source files	24
7.1	Assess.{h,cpp}	24
7.2	Blab.{h,cpp}	29
7.3	Build.{h,cpp}	29
7.4	Canon.{h,cpp}	33

7.5	Commute.{h,cpp}	36
7.6	Counter.h	39
7.7	Coupling.{h,cpp}	39
7.8	Data.h	40
7.9	Gen.{h,cpp}	42
7.10	Global.{h,cpp}	44
7.11	Gordion.{h,cpp}	49
7.12	Gripe.h	50
7.13	Index.{h,cpp}	51
7.14	Init.{h,cpp}	51
7.15	Linalg.h	52
7.16	Numerics.{h,cpp}	53
7.17	Obs.{h,cpp}	58
7.18	Ode.{h,cpp}	63
7.19	Op.{h,cpp}	64
7.20	Parse.{h,cpp}	65
7.21	Poly.{h,cpp}	68
7.22	Print.{h,cpp}	71
7.23	Rep.{h,cpp}	74
7.24	RK.{h,cpp}	76
7.25	Save.{h,cpp}	77
7.26	Splice.h	81
7.27	Symb.{h,cpp}	82
7.28	Symm.{h,cpp}	84
7.29	Term.h	86
7.30	Test.{h,cpp}	86
7.31	Theory.{h,cpp}	87
7.32	Version.h	90
A	Command summary	90
B	Observable counts	101
C	Symmetry transformations	104
C.1	Cubic lattice symmetries	104
C.2	Charge conjugation	106
C.3	Time reversal	108
C.4	Hermitian conjugation	109
C.5	Implications	111
C.6	Euclidean theory symmetries	112
D	Prior work errata	116
D.1	CSVA II [3]	116
D.2	CSVA I [2]	116
	References	118

1 Introduction

Gordian is a program for solving, numerically, for low energy properties of a variety gauge theories in the large N limit using the coherent state variational method.¹ The name **Gordian** is an allusion to the Greek legend associated with Alexander the Great, who is reputed to have sliced through a horrendously complicated knot, instead of carefully untying it, in Gordion (Latin: Gordium, Phrygian: Gordum), the capital city of ancient Phrygia. Dealing with arbitrarily complicated loops is at the core of the coherent state variational method as implemented in **Gordian**. It is hoped that this program helps cut through the complexity of large N gauge theories ☺.

2 Overall structure

The key steps comprising the coherent state variation algorithm [3] are:

1. Construction of a list of all observables of the chosen theory to be retained in the finite truncation set, and whose expectation values will be computed. Here “observables” mean Wilson loops, possibly containing one or two electric field insertion (in Hamiltonian theories), or fermion bilinears, possibly including one electric field insertion.
2. Selection of the finite set of coherence algebra generators, for a given theory, which will be used to generate deformations of a coherent state, and whose coefficients will serve as Riemann normal coordinates in the neighborhood of any given point in the large N phase space. These are anti-Hermitian combinations of Wilson loops, possibly with one electric field insertion, or fermion bilinears.
3. Evaluation, symbolically, of the commutators of all selected observables and generators. These define the geodesic equations on the large N phase space and encode how observable expectation values vary as one deforms the state.
4. Evaluation, symbolically, of the commutators defining the first and second variations of the Hamiltonian (or Euclidean free energy) with respect to Riemann normal coordinates. Also, for Hamiltonian theories, evaluation of commutators of generators whose expectation values define the Lagrange bracket (the inverse Poisson bracket) of the large N phase space.
5. Numerical minimization of the Hamiltonian (or Euclidean free energy) using Newton iteration, with each iterative step involving numerical evaluation of the symbolic expressions for the gradient and curvature of the Hamiltonian (or free energy), solution of the linear equations predicting the location of the minimum, and numerical integration of the geodesic equations describing the change in expectation values of all observables as one moves to the predicted minimum.

¹For background on this approach for solving large N limits, see Refs. [1–4]. Appendix D lists corrections for a number of misprints in these references. Prior work on this approach is also presented in the Princeton University Ph.D. theses of F. R. Brown [5], U. Lindqwister [6], T. Dickens [7], and W. Somsy [8].

6. In Hamiltonian theories, numerical evaluation of the symbolic expressions for the curvature of the Hamiltonian and the Lagrange bracket, in any chosen symmetry channel, and computation of the resulting small oscillation frequencies around the minimum, yielding a determination of the low-lying glueball or meson spectrum.

Each of these steps is carried out first for the $O(N^2)$ pure gauge dynamics, for some sequence of gauge coupling values, and then again for the sub-leading $O(N)$ fundamental representation fermion dynamics, for some sequence of fermion mass values.

3 Program scope

In early work on the coherent state variation method in the 1980's [3], separate C programs were written to handle different aspects of the overall procedure: symbol definition (the encoding used to represent Wilson loops, loops decorated with electric field insertions, and matter field bilinears), observable generation (the recursive construction of the truncated set of loops and bilinears used to encode properties of a coherent state), polynomial script generation (the construction of the geodesic equations for retained observables, as well the gradient and curvature of the Hamiltonian or Euclidean free energy), the actual minimization numerics (iteratively performing Newton minimization, repeatedly evaluating the gradient and curvature to predict the minimum location and integrating the geodesic equations to move to the predicted minimum), and computing small oscillations around the minimum to extract glueball or meson spectra. Given the constraints on available memory at the time, this partitioned approach with explicit passing of information between stages via large disk files was essential.

For the current implementation, the design goals were to:

- Create a unified program that can handle all needed steps from beginning to end.
- Design a program capable of handling $U(N)$ gauge theories² in multiple dimensions, in either Hamiltonian or Euclidean lattice formulations, with or without fundamental representation fermions. If fermions are present, it is assumed that the number of fermion flavors remains fixed as $N \rightarrow \infty$.
- Test the efficacy of possible approximation schemes for non-retained observables.
- Write highly time- and space-efficient code.

Regarding the last point, even though compute capabilities have vastly increased since the 1980's, the computational complexity of numerically minimizing the large N Hamiltonian (or Euclidean free energy) to a given accuracy inevitably grows exponentially with the correlation length of the theory, since the number of Wilson loops grows exponentially with any reasonable measure of their size. Because the continuum limit entails diverging correlation length, for any given level of computational resources a more efficient implementation will allow one to reach larger correlation lengths and hence better control the continuum limit.

²Note that the difference between $U(N)$ and $SU(N)$ gauge theories first appears at relative order $1/N^2$ in a large- N expansion; there is no difference in their large N limits as discussed above.

Of most physical interest are the cases of Hamiltonian lattice Yang-Mills or QCD [9] on lattices of two or three space dimensions, or Euclidean lattice Yang-Mills or QCD [10] on three or four dimensional space-time lattices. Including an ability to handle theories on lower-dimensional lattices, possibly compactified, allows one to test the program (and the method) on various exactly soluble models including the Euclidean one-plaquette model [11], the Hamiltonian one-plaquette model [12–14], 1+1 dimensional QCD in a Hamiltonian lattice formulation [15–18] and two dimensional Euclidean Yang-Mills on an infinite lattice [11]. Permitting one or more dimensions to be periodically compactified also enables one to study finite temperature equilibrium states in either a Euclidean formulation or in a Hamiltonian formulation with a compact spatial dimension, and to study the spectrum of strings wrapping a compactified dimension. Central to the approach is the fact that unbroken translation symmetry permits one to restrict the coherent state minimization to translationally invariant states, thereby allowing one to work directly in infinite volume.

These considerations motivated the following specific program design decisions and assumptions:

- The lattice dimension may be 1, 2, 3 or 4, with each dimension either infinite, or periodically compactified.
- Zero, one or two fermion flavors are allowed,³ with each fermion flavor defined as having one conjugate pair of fermion operators per site per flavor. For Hamiltonian theories, this is a staggered fermion discretization.
- All theories under consideration are invariant under cubic lattice symmetries (translations, permutations and reflections) together with charge conjugation (C) and time reversal.⁴
- None of these symmetries are spontaneously broken, and observables of interest are invariant under all symmetries.⁵
- Use modern C++, instead of some higher level programming language, thereby enabling more efficient memory use and explicit control over performance critical aspects such as copy versus move semantics when passing and returning data structures, while also having strong type checking, template meta-programming, highly useful standard container classes, and reliable highly optimizing compilers.

³When fermions are present, any compactified dimension length must be an even number of lattice spacings. Supporting multiple fermion flavors potentially allows one to study possible spontaneous flavor symmetry breaking, perhaps induced by the inclusion of four-fermi operators.

⁴The Yang-Mills θ angle is zero. For Hamiltonian theories, only translations by an even number of lattice sites are required to be symmetries of the theory.

⁵Performing spectrum calculations at non-zero momentum is a possible future addition but would require extending the code to evaluate commutators of observables carrying non-zero momentum.

4 Implementation considerations

4.1 Symbol encoding

Observables are represented by strings of symbols in which every symbol either denotes a possible combination of gauge fields residing on a single lattice link ℓ , or a fermion field residing on a lattice site. In Hamiltonian theories, for each lattice dimension, there are ten possible combinations of gauge fields which may be combined together on any given lattice link:

- A forward directed link variable, $U[\ell]$;
- A backward directed link variable, $U[\bar{\ell}] \equiv U[\ell]^\dagger$;
- An electric field operator, $E[\ell]$ (transforming at the starting site of the link);
- A reflected electric field, $U[\bar{\ell}]E[\ell]U[\ell]$ (transforming at the link end site);
- A squared electric field, $E[\ell]^2$;
- A reflected electric field square, $U[\bar{\ell}]E[\ell]^2U[\ell]$;
- An electric field inserted forward link, $E[\ell]U[\ell]$;
- An electric field inserted backward link, $U[\bar{\ell}]E[\ell]$;
- A squared electric field inserted forward link, $E[\ell]^2U[\ell]$;
- A squared electric field inserted backward link, $U[\bar{\ell}]E[\ell]^2$.

Using distinct symbols to denote each of these possible combinations of gauge fields which may appear together on a single link enables symmetry transformations to be implemented by simple symbol substitutions which never change the length to the symbol string representing an observable.

For fermions in Hamiltonian theories, distinct symbols are needed for staggered and un-staggered fermion operators, while for Euclidean theories distinct symbols are needed for fermion fields and the Grassmann derivatives of these fields.

To treat theories on lattices of potentially up to four dimensions, an individual symbol (for Hamiltonian theories) must be able to represent any of the above ten different combinations of gauge fields in every possible lattice direction, implying 40 possible values for gauge degrees of freedom, plus four additional possibilities for each fermion flavor, as detailed below. Hence, a single byte is amply sufficient to encode the range of symbols needed for all theories in the intended program scope.

The specific encoding employed, defined in file `Symb.h`, was designed so that simple bitwise operators, rather than look-up tables, suffice to extract the various properties encoded by each symbol value, such as the lattice axis or the above symbol types (for gauge symbols), or the fermion flavor or staggering. Conversion of a symbol into its reflection, charge conjugate, or staggered variant, are also simple bitwise operations. To make the canonicalization table indexing as simple as possible, the smallest symbol values $(0, \dots, 7)$ are used for links. For more detail, see the discussion in section 7.27.

The basic `ymb` type used to encode individual symbols is defined as a type alias for `signed char`, not as a distinct class, even though the latter option might seem natural. Using `ymb` as a mere alias for a `char` allows the symbol string class `SymbStr` to simply inherit from `std::string` and automatically acquire the short string optimizations used in implementations of `std::string`.

For printing and for user input of observables or generators, the following mapping is used to relate printable characters to symbols:

<i>character</i>	<i>associated symbol</i>
<code>x, y, z, w</code>	gauge links in directions 1, 2, 3 or 4, respectively;
<code>X, Y, Z, W</code>	inverse gauge links in directions 1, 2, 3 or 4;
<code>A, B, C, D</code>	electric fields in direction 1, 2, 3 or 4;
<code>f, F, g, G</code>	fermion, conjugate fermion, staggered fermion or Grassmann derivative $\partial/\partial F$ (Hamiltonian or Euclidean), and staggered conjugate fermion or Grassmann derivative $\partial/\partial f$, respectively, all for flavor 1;
<code>h, H, i, I</code>	corresponding flavor 2 fermion fields;
<code>S, s</code>	gauge or fermion entropy, respectively.

So, for example, `xyXY` represents a basic plaquette in the 1-2 plane, `YBy` is a backwards-directed electric field in the 2-direction, `Fxf` is a one-link fermion hopping term in the 1-direction, and `Gf` is a staggered fermion mass term (for Hamiltonian theories).

4.2 Observable storage

C++ standard library strings (`std::string`) are used to store the symbol strings representing observables. More precisely, the class `Obs`, whose instances represent physical observables, inherits from `std::string`. This use of `std::string` enables the `Obs` class to automatically take advantage of the short string optimizations present in implementations of `std::string` (allowing sufficiently short strings to be stored directly in the class object without needing any free store allocation/deallocation operations).

For storing the entire set of generated observables, one needs some form of bidirectional data structure which:

1. Enables highly efficient determination of whether a newly constructed observable is already present in the set;
2. Assigns a unique integer index to each observable in the set;
3. Enables easy access to an observable's symbol string given its index in the set.

The first goal is best accomplished by using a hash table (or `std::unordered_map`), as the typical performance of hash table lookups greatly surpasses that of alternatives like balanced binary trees. To support the other requirements, the observable list data structure (termed an `ObsList`) combines a hash table with a list of pointers to the individual observable objects stored in the hash table. This avoids duplicate storage of the symbol strings of all the stored observables. This approach relies on the fact that pointers to individual entries in an `unordered_map` are not invalidated even when a rehashing takes place.

4.3 Observable canonicalization

Observable canonicalization refers to the transformation of an observable’s symbol string into a unique physically equivalent representation. This may involve application of any lattice symmetry, possibly combined with charge conjugation and, for gauge observables, cyclically rotating the symbol string to a different starting point. With 96 elements of the lattice symmetry group (times C) in three dimensions, or 768 in four dimensions, typical observables will have thousands of physically equivalent representations. It is essential that the transformation of observables into some unique canonical representation be as efficient as possible.

The strategy employed treats loop observables (those only containing gauge links) separately from “special” observables containing electric field insertions and/or fermions. For loop observables, the canonical form is defined as the first representative when all physically equivalent symbol strings for a loop are lexically ordered. But actually constructing and sorting the set of all physically equivalent forms, every time a loop is to be canonicalized, is hugely inefficient. The following procedure is used instead.

A compile-time constant `loopchunksize`, whose value depends on the lattice dimension, serves to define “short” loops as those no longer than `loopchunksize`. Short loops are used directly as keys into a canonicalization cache, which is a hash table mapping observables to the index of their physically equivalent canonicalized form in the master list of canonicalized observables. For loop observables longer than `loopchunksize`, a moving window of `loopchunksize` is walked around the loop, with each such “chunk” converted to an index into a static loop canonicalization table, each of whose entries specifies the subset of symmetry transformations which would transform this chunk into its preferred (lowest lexically ordered) form, and a score, based on that lexical ordering, of the resulting preferred form of the chunk. As the chunk window is walked around the loop, the lowest scoring chunk(s) identify the small subsets of symmetry transformations and starting positions which need to be compared to identify the preferred form for the full loop.

For gauge observables containing electric field insertions, the canonical form is defined as the lowest lexically ordered form among those physically equivalent representations which begin with an electric-field inserted symbol. Since observables will have at most two electric field insertions, this greatly reduces the number of possible starting points which must be compared. To facilitate canonicalization and commutation of gauge observables containing electric field insertions, it is required throughout the program that the symbol string for any gauge observable containing electric field insertions begin with an electric field inserted symbol. Similarly, symbol strings for fermion bilinears always begin with a conjugate fermion symbol and end with a fermion. So “special” observables always begin with a non-link symbol.

The resulting canonicalization procedure for such special observables is similar to that for loops, except that only transformations which lead to acceptable symbol strings starting with an appropriate non-link symbol need be considered. A different compile-time constant `specchunksize` defines the separation between “short” and “long” special observables. The canonicalization cache is used for special observables whose number of (non-fermion) symbols is no more than `specchunksize`. For longer observables,

specchunksize chunks which either begin or end at the endpoints of a fermion bilinear, or on electric field insertions in gauge loops, are converted into indices into a static non-loop canonicalization table, each of whose entries specifies the subset of symmetry transformations which would transform this chunk into its preferred form, and a score, based on lexical ordering, of the resulting preferred form of the chunk. The scores of only two such entries need be compared for fermion bilinears, one for the initial chunk of the symbol string and one for the final chunk (involving symmetries which include charge conjugation). At most four such entries need be compared for gauge observables with two electric field insertions.

The values of **loopchunksize** and **specchunksize** are chosen to balance opposing goals: larger chunks lead to smaller sets of possible symmetries which will need to be considered in fully canonicalizing an observable, but at the cost of requiring larger pre-computed canonicalization tables and a larger cache for “short” observables. If the canonicalization tables are overly large, repeated frequent lookups into those tables will seriously compromise processor cache performance. These considerations lead to the definition of **loopchunksize** as 9, 7 or 6 for lattices of dimension 2, 3 or 3, respectively, which in turn leads to loop canonicalization tables with between 262K and 280K possible entries. Similarly, depending on lattice dimension **specchunksize** is defined to be 4 or 5 in Hamiltonian theories, or 6, 7 or 8 in Euclidean theories (which lack electric-field inserted observables), leading to non-loop canonicalization table sizes ranging from 41K up to 524K for the various theory classes.. In practice, this canonicalization procedure enables the number of symmetry transformations performed per canonicalization to average below one in 2D to three in 4D.

4.4 Commutator evaluation

The basic $U(N)$ Hamiltonian lattice theory commutation relations are:

$$[(E_\ell)_{ij}, (U_{\ell'})_{kl}] = \frac{1}{N} \delta^{\ell\ell'} \delta_{kj} (U_\ell)_{il}, \quad (1a)$$

$$[(E_\ell)_{ij}, (E_{\ell'})_{kl}] = \frac{1}{N} \delta^{\ell\ell'} (\delta_{kj} (E_\ell)_{il} - \delta_{il} (U_\ell)_{kj}), \quad (1b)$$

for the gauge degrees of freedom living on links, together with the anti-commutation relation for fermions residing on sites,

$$\{(\psi_s)_i^\alpha, (\psi_{s'}^\dagger)_j^\beta\} = \delta_{ss'} \delta_{ij} \delta^{\alpha\beta}. \quad (2)$$

Here, ℓ labels links, s labels sites, i, j, k, l are $U(N)$ gauge indices, and α, β denote fermion flavors. The Yang-Mills commutation relations (1) may be realized in a “coordinate representation” for which

$$(E_\ell)_{ij} \equiv \frac{1}{N} (U_\ell)_{ik} \frac{\partial}{\partial (U_\ell)_{jk}}. \quad (3)$$

For Euclidean theories, “electric field insertions” in coherence group generators are defined to mean the first order differential operators (3).

As discussed in Ref. [3], the encoding of observables as strings of symbols, one per lattice link (or site), enables the evaluation of commutators of generators and observables to be converted, modulo a few details, into lexical substitution operations in which, for example, the symbol string representing a coherence group generator with one electric field insertion is sequentially spliced (minus the electric field) into the observable symbol string at a location just in front of a symbol representing a forward-directed link on which the electric field can act (or just after a corresponding backwards-directed link).

The exact lexical procedure is complicated by the choice, discussed above, to use a distinct symbol to represent every possible combination of electric fields and link operators which may reside on a common lattice link. The precise substitution rules may be encoded in a static, dimension-independent splicing table, discussed below in Sec. 7.26. The commutation routines carry out the resulting splicing and dicing operations, keeping track of the associated minus signs, suitably canceling any links and inverse links arising at every joining operation, and yielding a polynomial in observables (with terms collected) produced by each commutator.

As discussed in section 4.5 of Ref. [3], additional terms involving triple products of Wilson loops can arise in the commutator of a double- E inserted loop with a single- E loop, due to the implicit normal ordering of these observables. Similarly, products of Wilson loops can arise in the commutator of fermion bilinears with electric-field inserted bilinears, due to their normal ordering prescriptions. The commutation routines also generate and add these extra normal-ordering induced terms to the resulting observable polynomials. See Sec. 7.5 for further details.

4.5 Observable generation & classification

As discussed in Refs. [2, 3], the most effective observable truncation scheme identified to date is based on a grading of observables defined by their *strong-coupling* order. This is the order in a lattice strong coupling expansion at which an error first appears if the given observable's expectation value is excluded from an otherwise correct strong coupling expansion of the ground state energy (or equilibrium free energy). As explained in section 5.1 of Ref. [3], the strong-coupling order is the sum of an observable's *creation order* plus its *expectation order*. The creation order is the minimal sum of the number of commutations with single-link fermion hopping terms plus twice the number of commutations with single plaquette generators needed to produce the given observable, while the expectation order is the minimal sum of the number of commutations with single-link fermion hopping terms plus twice the number of commutations with single plaquette generators needed to yield the identity operator, or a single-site fermion bilinear, starting from the given observable.⁶

⁶These descriptions are not quite precise, particularly if there are compactified dimensions; see Ref. [3] for details. The counting of plaquette operations with double the weight of fermion hopping operations reflects the structure of the standard Kogut-Susskind Hamiltonian [9]. In $\hat{H}/\lambda$, the gauge kinetic energy is $O(1)$ while the fermion kinetic energy (or hopping term) is $O(1/\lambda)$ and the gauge potential energy (or plaquette term) is $O(1/\lambda^2)$.

Note that the definitions of observable creation, expectation, and strong-coupling orders used in this work are all twice those used in Ref. [3], so as to avoid half-integral creation or expectation orders for

The observable generation procedure uses the approach described in section 5.2 of Ref. [3]. The target strong-coupling order is progressively incremented up to the desired final value. For each strong-coupling order, the possible creation order is progressively incremented from half the strong-coupling order, up to the strong-coupling order minus four, for gauge observables, or minus two, for fermion bilinears. (Four less, for gauge observables, because the expectation order of any multi-plaquette loop is at least four. Two less, for fermion bilinears, because the expectation order of any multi-link fermion bilinears is at least two.) New observables are generated via the commutation of known lower order observables with “primary” coherence group generators. These are generators, such as a plaquette with a single electric field insertion, which cannot be produced via commutation of lower order generators.

Pure gauge Wilson loops (termed **Loop** observables) with specified creation and strong-coupling orders are first generated followed, for Hamiltonian theories, by the generation of loops with one electric field insertion (termed **Eloop**’s), and then loops with two electric field insertions (termed **EEloop**’s). Then during the fermion minimization stage, a similar procedure is used to generate fermion bilinears without any electric field insertions (termed **Fermion** observables) followed, for Hamiltonian theories, by the generation of fermion bilinears with one electric field insertion (termed **Efermions**).

Proper sequencing of these various observable generation passes is needed to enable correct expectation order classification of observables. As discussed below, expectation order classification involves factorizing loops at self-intersections and identifying the sub-loops as previously constructed observables. For this procedure to work, prior to generating observables of any specific creation and strong-coupling order, it is essential that all observables of lower creation order, or lower strong-coupling order, are already known.⁷

As discussed in Appendix C, time reversal symmetry enables one to partition observables into those whose expectation values act as classical coordinates on the $N = \infty$ phase space, and those whose expectation values serve as conjugate momenta and vanish on the time-reversal invariant classical configuration space. Expectation values of Wilson loops, and of loops with two electric field insertions, act as classical coordinates, while expectation values of loops with one electric field insertion serve as conjugate momenta. To generate expressions for the Hamiltonian gradient, curvature, and the geodesic equations used for minimization of the Hamiltonian, there is no need for any single- E loops, all of whose expectation values will vanish, to be present in the master list of canonicalized observables. However, the classification procedure described below for double- E loops relies on the prior generation (and classification and storage) of lower-order single- E loops. Consequently, even though they will not be needed in the minimization, the observable

fermion bilinears.

⁷This is why it is not possible to merge the observable generation and geodesic equation evaluation steps and simply store any observable newly appearing in the evaluation of a given geodesic equation. Moreover, it is not sufficient to merely generate observables with progressively increasing creation orders, irrespective of their strong-coupling orders, as this simplified procedure fails to handle some rare cases where a loop of a given creation order factorizes into a sub-loop of the same creation order but lower strong-coupling order. (As an example, the creation order 14, expectation order 10 loop $\text{xxyXyxYxYXXyxxyYXXyxYXXY}$ requires for its correct classification prior knowledge of the creation order 14, expectation order 6 loop xyxxyYXXyxYXXY .)

generation routines produce, classify and store single- E loops. For any given maximal strong-coupling order, the total number of such single- E loops is only a small fraction of the number of double- E loops, so the cost of retaining these ultimately unnecessary observables is not significant. For fermion bilinears, changing (in a Hamiltonian theory) between the staggered and unstaggered version of the bilinear flips whether its expectation value acts as a classical coordinate or momenta in the large- N dynamics. The observable generation routines only construct and store in the master observable list those fermion bilinears which serve as classical coordinates.

While generating observables with a known creation order is quite straightforward, determining the expectation order of each resulting observable is more difficult. Correct evaluation of the expectation order is needed to determine how an observable's strong-coupling order compares with the specified maximal strong-coupling order for retained observables. A brute force approach, directly applying the above definition, is very inefficient. Several different strategies are used to minimize the computational effort required for expectation order determination. Depending on an observable's type, two different quick approaches are first employed to produce a lower bound on the observable's expectation order. These bounds are sufficient to eliminate from further consideration a large fraction of observables newly produced by the commutators of the observable generation process.

For observables containing electric field insertions, simply removing those electric field insertions can only decrease the expectation order. If the resulting no- E observable, either a Wilson loop or a fermion bilinear, is present in the set of known (and correctly classified) observables, then the expectation order of the no- E observable provides a lower bound on the expectation order of the original observable with electric field insertion(s). Alternatively, if the no- E modification of the observable is not found in the set of already known observables, this indicates that the strong-coupling order of the original observable must be larger than the intended limit.⁸ This determination is performed by the `Obs::noEbound` routine. Of course, this procedure requires that the generation, and classification, of all no- E observables of a given strong-coupling order be performed prior to generating and classifying observables of the same order containing electric field insertions.

A second lower bound on the expectation order is computed for gauge observables (with or without electric field insertions), based on the observation that the expectation order of a single trace observable in an $SU(N)$ gauge theory may be larger than the expectation order of the same observable in a $U(1)$ gauge theory, due to the additional cancellations which may occur on multiply-traversed links in a $U(1)$ theory, but cannot be smaller. So a $U(1)$ theory expectation order determination provides a lower bound on the $SU(N)$ expectation order. Such a $U(1)$ theory determination (above two dimensions) is still very computationally costly. But a lower bound on the $U(1)$ expectation order may be relatively quickly computed by projecting the observable's path through the lattice onto a two-dimensional lattice plane, calculating the net $U(1)$ flux on each plaquette enclosed by this two dimensional projection, summing the absolute values of such flux over each

⁸This inference relies on the fact that the creation order of the no- E modification of an observable will be no larger than the creation order of the observable with electric field insertions.

enclosed plaquette of the projection and then, in three or more dimensions, summing over all different orthogonal lattice planes. This calculation is performed by the `Obs::u1bound` routine.

One or both of these lower-bound determinations are applied to all newly generated observables other than fermion bilinears (without E 's), for which no quick lower bound determination is available. If neither lower bound can rule out the possibility that a newly generated observable lies within the desired truncation set, then a proper determination of its expectation order is needed (or at least a determination that its order is too high for the observable to lie within the truncation set).

For observables without electric field insertions, this is done by finding all self-intersections of the observable, factoring the observable into sub-observables at every such self-intersection, and finding the minimal sum of expectation orders for all such factorizations. This is carried out by the `Obs::factorize` routine.⁹ For no- E observables without any self-intersections, or for which no factorization leads to already known observables, the expectation order must equal the observable's creation order.

For observables with electric field insertions, the only valid procedure known is to actually perform commutations which will reduce the observable to some already known and classified observable. As discussed in Ref. [3], one may argue that a minimal commutation sequence determining the expectation order may begin with commutation by a primary generator (a plaquette, Polyakov loop if there are compactified dimensions, or fermion hopping term) which eliminates an electric field insertion. This procedure is carried out by the `Obs::reduce` routine.¹⁰

Appendix B contains tables listing the resulting number of canonicalized observables of a given type and classification order, on cubic lattices of dimension 2, 3 or 4.¹¹

4.6 Observable approximation

The current code was designed to allow testing the hypothesis that the accuracy of the state representation, based on retaining a large but finite set of observable expectation

⁹For fermion bilinears, it is necessary to consider both factorizations which extract from the self-intersecting bilinear a pure gauge loop, as well as factorizations into products of fermion bilinears with additional fermion/conjugate fermion pairs inserted at the intersection site. This is needed to account for the fact that, depending on the size of the sub-loop contour, the minimal expectation order could arise from either area-law or perimeter-law contributions.

¹⁰One subtlety, not explicitly addressed in Ref. [3], involves the fact that commutation of a double- E loop with a single- E generator leads to both modified double- E observables as well as to products of no- E loops. The `Obs::reduce` routine also examines these loop-splitting no- E terms as part of its expectation order determination.

¹¹For `Loop` observables, the counts shown in Appendix B agree with those shown in tables 1–3 of Ref. [3] for strong-coupling orders below 24 (labeled as order 12 in Ref. [3], but there are discrepancies at order 24, the highest order reported in Ref. [3], with undercounts in the older reference. The reason for this discrepancy with old results is not known. For `Eloop` and `EEloop` observables, there are more extensive discrepancies with the old results. This, it is believed, reflects the subtlety mentioned in the previous footnote: no electric field terms can arise from the extra loop-splitting terms in `[EEloop, Eloop]` commutators but, so far as can be recalled, such terms were not considered in previous observable classification routines.

values, can be improved by approximating expectation values of non-retained observables by products of expectation values of retained observables, instead of simply discarding expectation values of all non-retained observables. The basic idea is to approximate a non-retained observable (whose strong-coupling order exceeds the specified limit) by a product of lower order observables which arise from factorizing the original observable at a self-intersection. This is motivated by the observation that nearly all lattice loops are self-intersecting, and that a factorization-based approximation is consistent when expectation values are well-described by a combination of area-law and perimeter-law terms, which should be the case in both strong and weak coupling regimes.¹²

Given a non-retained observable (generated by the commutation routines and identified as such by the classification routines), the `Obs::approximate` routine attempts to generate a factorization-based approximation. The routine examines all self-intersections of an observable, assigning each possible split a numeric score which is used to select preferred possible factorizations when there are multiple possibilities. The particulars of the intersection scoring are chosen to favor factorizations which split the observable into comparably-sized pieces, as this should maximize the probability that both sub-observables are already known and stored in the master observable list. The scoring also disfavors splits which are immediately next to an electric field insertion, and weakly disfavors splits which arise from multiple traversals of the same link. The scoring algorithm is unquestionably ad-hoc, and is a target for future study and improvement.

4.7 Generator selection & normalization

The coherence group $\mathcal{G}$ is generated by the Lie algebra consisting of all anti-Hermitian `Loop`, `Eloop` and `Fermion` operators. Elements of this coherence group, when acting on a suitable base state $|0\rangle$, generate large N gauge theory coherent states. In a lattice gauge theory, the base state may be taken to be the vacuum state (of the Kogut-Susskind lattice Hamiltonian) in the limit of infinitely strong gauge coupling and large fermion mass.¹³

The coherence group $\mathcal{G}$ has a non-trivial little group $\mathcal{L}$ which leaves this base state invariant. In the gauge sector, the Lie algebra of this little group consists of all `Eloop` generators whose reduction to a `Loop` vanishes. The generator reduction is the result of the “inner commutator” where the electric field insertion in the `Eloop` generator is allowed to act on the links of within the `Eloop` generator itself.¹⁴ In the fermion sector, all even length bilinears are elements of the base state little group. Consequently, representatives of the coset space $\mathcal{H} \equiv \mathcal{G}/\mathcal{L}$ consist of exponentials of anti-Hermitian linear combinations of

¹²This simple factorization-based approximation scheme is certainly not exact, and does not account for the interaction between electric flux sheets — something which is especially significant in two (Euclidean) dimensions where typical self-intersecting loops lead to highly “folded” flux sheets. But the accuracy of the approximation should, it is hoped, improve with increasing lattice dimension.

¹³The state $|0\rangle$ is annihilated by all electric field operators $\{E_\ell\}$, all fermion field $\{\psi_s\}$ on the even sublattice, and all conjugate fermion field $\{\psi_s^\dagger\}$ on the odd sublattice.

¹⁴More formally, if $\text{tr}(E_\ell U(C))$ denotes an operator in which a Wilson loop $U(C)$ has an electric field insertion on some link ℓ , the generator reduction refers to the commutator $[(E_\ell)_{ij}, U(C)_{ji}]$. Examples of gauge generators with vanishing reductions, and hence elements of the little group Lie algebra, are `AyXYx` and `AxyXY - ByXYx`.

Wilson loops, odd length fermion bilinears, plus of so-called “commutator generators”, produced by commuting a `Loop` generator with $\text{tr}(E^2)$.¹⁵

As one moves around in the large- N classical phase space (i.e., moves to a different large- N coherent state) this little group is conjugated. The coset space $\mathcal{H}$ is not itself a group because the little group $\mathcal{L}$ is not a normal subgroup. However, coherence group elements $g \in \mathcal{G}$ may be factorized as $g = h \times l$ with $l \in \mathcal{L}$ and $h \in \mathcal{H}$, and hence only elements of the coset space need be considered when generating coherent states. Consequently, when deforming a coherent state, it should be sufficient to consider only those deformations produced by `Loop` generators, `Eloop` commutator generators. and odd-length fermion bilinears.

As noted initially, truncating the infinite set of coherence algebra generators to a finite computational subset is an unavoidable step in this variational approach. Every generator may be assigned a *creation order* which is, essentially, the minimal sum of the number of commutations with single-link fermion hopping terms plus twice the number of commutations with single plaquette generators needed to produce the given generator. This classification reflects the relative importance of a given generator on the strong coupling dynamics of the theory.

However, it is clearly desirable to examine *ex post facto* assessments of a generator’s importance based on the relative size of its component in the gradient dH (4) or the predicted minimum location δ (5). But no such comparison of relative component magnitudes is meaningful in the absence of some basis-independent normalization of generators. The lack of any natural metric on the coherence Lie algebra makes this problematic.

A partial solution, for Hamiltonian theories, is to regard the base state expectation of the double variation of the gauge kinetic energy, $\langle 0 | [[\text{tr}(E^2), \cdot], \cdot] | 0 \rangle$, as defining a pseudo-metric. This pseudo-metric is non-degenerate within the subspace of generators in the coset space $\mathcal{H}$. But there is a null-space involving coherence group generators of the base state little group $\mathcal{L}$. Such generators will have vanishing norm as defined by the above pseudo-metric.

If all `Eloop` generators retained in the generator truncation have non-vanishing single-loop terms in their reduction and different generators involve geometrically distinct loops, then one may uniquely and physically normalize generators by requiring the strong coupling expectation of their double variation of $\text{tr}(E^2)$ to give unity. Generators involving distinct loops or odd-length fermion bilinears are automatically orthogonal by this measure. Such normalization is performed if the global boolean variable `gennorm` is set to true.

If one chooses to include generators with vanishing (or purely multi-trace) reductions, or even length fermion bilinears, then such generators will have vanishing norm as defined by the above pseudo-metric. Implementing a well-defined normalization for such generators is unavoidably ad-hoc. The current code simply sets to unity the sum of squares of the coefficients of all terms in such generators.

For Euclidean theories, instead of the double variation of $\text{tr}(E^2)$ one may regard the double variation of the gauge field entropy S , evaluated in the strong coupling limit, as defining

¹⁵This form of coset representatives holds at least within the domain of convergence of a Baker-Campbell-Hausdorff expansion of the product of elements of $\mathcal{G}$ and $\mathcal{L}$.

a pseudo-metric for gauge generators (with the same caveats regarding its null space as above). But the fermionic contribution to the entropy cannot serve the same purpose for fermion generators, as only fermion bilinears which begin and end on the same site lead to non-vanishing entropy variations in the strong coupling limit. Instead, in Euclidean theories the double variation of the fermion mass term may be used to define a metric on the space of fermion bilinear generators. This is the procedure used when generator normalization is enabled.

4.8 Polynomial representation

In Euclidean theories, the geodesic equation for any Wilson loop is a linear combination of loops, while the variation of the entropy (needed to minimize the free energy) is a quadratic polynomial in loops. In Hamiltonian theories, terms cubic in Wilson loops arise via normal-ordering in the commutator of a generator with one electric field insertion and a gauge observable with two electric field insertions. Approximating the expectations of observables outside the selected truncation set by a product of expectations of observables in the retained set further can further increase the order of monomials in observables which need to be stored.

As discussed above, every retained observable is assigned a unique index in the master list of canonicalized observables, so an order- k monomial in observables is represented by a floating point coefficient together with a k -tuple of observable indices. Large calculations can involve many millions of polynomial geodesic equations comprising altogether many billions of individual monomial terms. Consequently, storage efficiency is of some concern, needing to be balanced against programming convenience.

Several different representations for storing observable polynomials are used. During commutator evaluation a fixed-size `PolyTerm` class object is used to represent monomials. Each `PolyTerm` contains a `std::array<unsigned int,PSIZ>` data member holding observable list indices plus a floating point coefficient, with `PSIZ` a compile-time constant which is set to 2 for Euclidean theories and 4 for Hamiltonian theories. As the trivial identity loop with a fixed expectation value of 1 is always assigned index 0, a `PolyTerm` instance can represent an observable monomial of any order up to `PSIZ`, while keeping the code needed to manipulate observable polynomials as simple as possible. The multiple monomials comprising a complete observable polynomial are stored either in a simple list (an `ObsPoly` class object) or, to efficiently collect common terms, in a hash table (a `PolyMap` class object).

Because most terms in the geodesic equations are linear monomials, storing a `PSIZ` length index list for every monomial is not the most efficient use of memory. To enable large scale calculations, the `PolyMap` objects generated by the commutation evaluation routines are converted to a packed format (a `Poly`) in which a single header, recording the length of the data structure, is followed by a string of monomial coefficients and indices, with negated indices used to signal non-terminal non-vanishing indices in the set of observable indices for a given monomial, eliminating the need to store redundant vanishing indices. This packed format is efficient for numerical evaluation of the symbolic polynomial representation, as well as for saving to and retrieval from external files.

4.9 Numerics

Programming the numerical part of the coherent state variational algorithm is relatively straightforward compared to the symbolic stages. A list of numerical expectation values of all the retained observables is initialized with known starting values. For the $O(N^2)$ gauge stage of the minimization, this is the infinite coupling limit in which all gauge observables other than the identity loop have vanishing expectation values. For the $O(N)$ fermion minimization stage, this is the large mass limit in which the only non-vanishing fermion bilinears are those in which the fermion and conjugate fermion reside on the same site, with the bilinear expectation value determined by the corresponding Wilson loop expectation.

As explained in appendix C, in Hamiltonian theories all non-vanishing observable expectation values are real. But, as detailed in appendix C.6, in Euclidean formulation theories with fermions, the chosen form of fermion discretization makes translation invariance manifest with a kinetic term of the same form as in Hamiltonian theories, but imparts imaginary expectation values to even-length fermion bilinears. This factor of i is not made explicit anywhere in the code, except in the final (Mathematica format) numerical results files, because such even-length bilinear expectation values are always multiplied by a pure imaginary coefficient. In the list of stored expectation values, and in the polynomials representing the free energy, its gradient or curvature, or the geodesic equations for Euclidean theories, every even-length bilinear means $-i$ times the actual bilinear expectation value, and real arithmetic suffices for evaluating all such polynomials.

After setting (or changing) the finite value of the lattice coupling constant λ or fermion mass m , moving to the resulting non-trivial ground (or equilibrium) state involves iterative Newton minimization. In each such minimization step, one first needs to evaluate the gradient (dH) and curvature (ddH) of the Hamiltonian (or free energy). These are a vector and matrix, respectively, of dimension equal to the number of defined (vacuum representation) generators,¹⁶ with components

$$(dH)_\alpha \equiv [H, e_\alpha], \quad (ddH)_{\alpha\beta} \equiv [[H, e_\alpha], e_\beta], \quad (4)$$

where e_α denotes a vacuum representation generator, and each commutator is a previously evaluated polynomial in the expectation values of observables. The curvature matrix ddH is real but not, in general, precisely symmetric. The antisymmetric part equals a gradient component in the direction of a commutator of generators, $(ddH)_{[\alpha,\beta]} = [H, [e_\alpha, e_\beta]]$. Due to the truncation of generators to a finite set, some antisymmetric portion of the curvature will remain even at the conclusion of the minimization.

After evaluating the gradient and curvature, one finds the predicted location δ of the minimum by solving the linear equation

$$ddH \cdot \delta = -dH. \quad (5)$$

¹⁶More precisely, the number of time-reversal even vacuum representation generators, as these are the operators whose exponentials generate motion within the large- N classical configuration space. Gradient components corresponding to time-reversal odd generators vanish in all time-reversal invariant states, and the curvature matrix is block diagonal with separate time-reversal even and odd blocks.

One then moves to the predicted minimum by integrating the geodesic equations describing the change in each (retained) observable's expectation value. These geodesic equations comprise a large set of coupled, polynomial ordinary differential equations (ODEs) of the form

$$\frac{d\mathcal{O}_i}{ds} = \sum_{\alpha} [\mathcal{O}_i, e_{\alpha}] \delta_{\alpha}, \quad (6)$$

where i runs over all observables in the retained set with $\mathcal{O}_i$ denoting the corresponding observable's expectation value, and each commutator is again a known polynomial in all the expectation values $\{\mathcal{O}_i\}$, which are themselves functions of the affine parameter s which is integrated from 0 to 1. This iterative procedure is repeated until convergence, at which point the excitation spectrum (in Hamiltonian formulations) may be evaluated by solving the small-oscillation generalized eigensystem

$$ddH \cdot \delta = -i\omega L \cdot \delta, \quad (7)$$

where ddH is the Hamiltonian curvature (in any chosen irreducible representation) and L is the corresponding Lagrange bracket matrix, with components

$$L_{\alpha\beta} \equiv [e_{\alpha}, e_{\beta}]. \quad (8)$$

The curvature matrix is block-diagonal with one block, ddH_{++} , involving pairs of time-reversal even generators and one block, ddH_{--} , involving pairs of time-reversal odd generators, while the Lagrange bracket matrix is block off-diagonal (and antisymmetric) with one block, L_{+-} , involving the commutator of time reversal even and time reversal odd generators. The small oscillation eigensystem in the first-order form (7) may be reexpressed as a smaller second-order eigensystem

$$ddH_{++} \cdot \delta_+ = -\omega^2 L_{+-} \cdot (ddH_{--})^{-1} \cdot L_{-+} \cdot \delta_+, \quad (9)$$

which is computationally more efficient.

This summary shows that three types of numerical support routines are needed:

1. An efficient and accurate linear system solver for dense real non-symmetric matrices;
2. An accurate ODE solver for large systems of first order different equations;
3. An accurate generalized eigensystem solver for dense non-symmetric matrices.

The Armadillo C++ library for linear algebra and scientific computing provides high quality and fast routines for the linear algebra tasks 1 and 3.¹⁷ For the ODE integrator, this code employs, by default, an adaptive fourth-order embedded Runge-Kutta algorithm [22] which uses the same set of function evaluations to compute two different approximations to the solution, with a norm of the difference between them used to adjust the integration step size. Various alternative embedded Runge-Kutta schemes, of both lower and higher order, are also available and may be easily selected. For further details and references, see Sec. 7.24.

¹⁷It is also possible to compile the code using the Eigen C++ library for linear algebra. This capability was added for testing purposes, including access to 80-bit arithmetic on Intel processors, but use of the Armadillo library is generally preferred.

4.10 Multi-threading

On multi-core systems it is, of course, desirable to take advantage of the speed-ups which multi-threading can enable. To do so, it must be possible to exploit parallelism without incurring excessive cost from any needed synchronization or locking operations.

The symbolic construction of geodesic equations for all retained observables is a major time-consuming step which can be very easily and effectively parallelized. The loop over all observables is replaced by a parallel `for_each` loop over a much shorter list of buckets of observables, with the evaluation of geodesic equations for each such bucket of observables automatically distributed across multiple threads.¹⁸ The only special considerations needed involve the declaration of various statistics counters as well as a few dynamically-sized buffers used by the commutation and observable assessment routines for construction of symbol strings, temporary sets of observables, etc. With single-threaded execution such buffers could be declared `static`, thereby minimizing the needed free store allocation and deallocation operations. With multi-threaded execution, such buffers must not be declared `static` (since static variables are unique and shared across all threads). Instead, there is a choice between automatic scope, local to a given block, or `thread_local` storage, which persists like static variables but is local to each thread. Explicit testing found that automatic block-scoped declaration of buffers is preferable in the commutation routines, while `thread_local` storage turns out to be preferable for the buffers in the observable analysis routines. The various statistics counters which are updated during polynomial script generation are declared as `alignas(128) std::atomic<ulong>` so as to be thread-safe, with the additional alignment specification serving to prevent cache line contention.

The time needed for construction of the symbolic expressions for the gradient and curvature of the Hamiltonian (or free energy) is generally a small fraction of the time needed for observable generation or construction of the geodesic equations. Consequently, the current code does not bother with parallelization of these steps.

During the numerical minimization stage of a calculation, the evaluation of the derivative vector of all observables (i.e., the numerical evaluation of the polynomial scripts encoding the geodesic equations for each observable) can also easily benefit from parallelization, simply by using another parallel `for_each` loop over buckets of observables to distribute the computation across multiple threads.

Effectively parallelizing the observable generation phase is more challenging. One must prevent race conditions or inconsistencies in access to the growing observable list (or the canonicalization cache). The simplest approach for doing so would be to require acquisition of a `mutex` lock on the observable list every time a new observable is added. Explicit testing found that the cost of such `mutex` locking on every insertion is unacceptably high, yielding performance worse than single threaded execution.

In the second attempt at multi-threaded observable generation, on every pass of the observable generation process the list of possible starting observables was divided into “buckets” of observables, each of which will be assigned to one of the threads spawned

¹⁸Linking with the Intel® Threading Building Blocks template library [19] enables very easy multi-threading via parallel `for_each` loops.

by a parallel `for_each` loop. Each of the observable generation routines called by these parallel `for_each` loops first made a private copy of the current observable list, to which new observables found by the given thread were appended during the evaluation of the commutators involving the assigned bucket of observables. Only following all commutator evaluation for a given bucket of observables was a `mutex` lock acquired to synchronize merging of the augmented local observable list back into the master list. This reduced the cost of thread synchronization to an acceptably small amount and made parallel execution worthwhile. However, when attempting to generate large observable sets, on machines with many cores, replicating the progressively growing observable list (actually a hash table) in every thread can consume problematically large amounts of memory (exceeding available physical memory and essentially stalling forward progress).

To avoid such excessive memory consumption, the next approach used a `thread_local std::unordered_set` to hold observables newly generated by a given thread, with `mutex`-protected merging of these “inboxes” of observables at the end of each bucket-processing observable generation routine, followed by (single threaded) addition to the master observable list of all new observables at the end of each observable generation pass. This approach works, with reasonable memory consumption, but has a downside that the delayed addition of new observables to the master list also delays additions to the canonicalization cache of short observables, leading to much worse canonicalization cache performance during the observable generation phase, with excessively many repeated canonicalizations involving all possible symmetry transformations of short observables.

The current approach remedies this last problem by using single-threaded observable generation, with immediate addition of new observables to the master list as soon as they are identified, as long as the total number of observables is below a threshold (set in `Build.cpp`), while switching to multi-threaded observable generation, as just described, above this threshold. This approach has allowed the largest observable sets listed in appendix B to be successfully generated on desktop computers.¹⁹

Using the Intel® TBB library [19], parallel `for_each` loops will, by default, spawn as many threads as there are available processor cores. This seems to work fine for geodesic script generation and evaluation, but when running on machines with many cores, observable generation for some intermediate size observable sets can experience significant contention between threads due to the synchronization required for inbox merging of different threads. A global variable (`global.maxthread`) may be set to a non-zero value to limit the maximum number of concurrent threads used in any multi-threaded portion of the program, and suitably adjusting the value of this limit, depending on the number of processor cores and maximum memory available, can significantly improve observable generation performance for certain observable generation cases.

¹⁹The generation of the multi-hundred million observable sets shown in appendix B used an Apple Mac Studio with 128Gb of memory. Generation of still larger observable sets, numbering in the billions, is limited both by the needed memory capacity and by the current use of unsigned 32-bit integers to store observable indices. It would, of course, be straightforward to switch to 64-bit integers for such indices. But this would substantially increase the size of polynomial scripts encoding geodesic equations which, as discussed below, can already be problematically large. Hence this step has not been taken (yet).

4.11 Input and output files

The program handles several types of input and output files:

Command input

In addition to reading from the standard input stream `std::cin`, an ordinary text file of pre-prepared commands may be input with the `read` command.

Program output

Program output may be directed to an output file, instead appearing on the standard output stream `std::cout`, by using the `write` command. (One may also redirect output back to `std::cout`.)

Sys info files

All symbolic information, including the selected set of observables and generators, the observable polynomials representing the gradient and curvature of the Hamiltonian (or free energy), and the symbolic geodesic equations may be saved in a *system information* (or `.sys`) file. These binary sus-info files may also be loaded using the `load` command (or by listing on the initial program command line). This allows one to perform the symbolic part of a calculation just once and subsequently reload and use that information multiple times during the numerical part of a calculation. System information files may be saved with the explicit `save sys` command or, if `autosave` is set to `true`, will be written automatically when geodesic equations are generated.

Vev data files

The list of observable expectation values (or *vev's*), together with the values of adjustable coupling constants, may be stored in *vacuum expectation* (or `.vev`) data files. Such files may hold multiple data sets recording observable expectation values for different values of coupling constants. Information stored in such binary vev-data files may also be loaded using the `load` command (or by listing on the program command line). The particular data set loaded may be user specified or, by default, is the last data set in a vev-data file. Vev data files may be saved with the explicit `save vev` command or, if `autosave` is set to `true`, will be written automatically after each successful minimization step.

Results files

When a sequence of minimizations are performed (by the `do coupling-name ...` command), human readable results are automatically written to a result file in a format which may be directly input to *Mathematica*. Such *MMA* (or `.m`) files record the values of the Hamiltonian or free energy, the expectation values of a selected set of observables (by default, those appearing in the Hamiltonian or free energy) and, for Hamiltonian theories, the values of particle masses, or small oscillation frequencies, in all symmetry channels with defined curvatures.

For theories with fermions, separate sys-info, vev-data and MMA results files are generated for the pure gauge parts of a calculation and for the fermion portions of a calculation.

Gauge information is stored in files named to indicate the theory type and the maximum orders of generators and observables, with file extensions indicating the file type. Sys-info and vev-data file names include an additional two (upper case) characters formed from a hash code or check-sum of the entire set of gauge observables; this serves to distinguish which vev-data files are associated with a given sys-info file. If observable approximation is turned on, a further ‘a’ is appended to the file name. As an example, gauge stage output files for 3+1 dimensional Hamiltonian Yang-Mills (or QCD) with order 4 generators and strong-coupling order 20 observables (without observable approximation) could be named `YM3h_4g20QZ.sys`, `YM3h_4g20QZ.vev`, and `YM3h_4g20.m`.

Fermion information is stored in files named to indicate the theory type as well as the generator and observables orders for both gauge and fermion stages, with separate hash codes for the gauge observable set (which will match the associated `YM*` data files) and the fermion observable set. As an example, fermion stage output files for 3+1 dimensional Hamiltonian QCD with order 4 gauge generators and order 20 gauge observables along with order 3 fermion generators and order 10 observables (without observable approximation) could be named `QCD3h_4g20QZ_3f10KW.sys`, `QCD3h_4g20QZ_3f10KW.vev`, and `QCD3h_4g20_3f10.m`.

Sys-info and vev-data files are stored together in theory-specific subdirectories (`YM3h`, `QCD3h`, etc.) of the main save directory (by default, `./save`). Results files are stored in theory-specific subdirectories of the MMA results directory (by default, `./MMA`). These directory specifications may be changed with the `set savedir` and `set MMAdir` commands.

4.12 Disk swapping

As discussed below, with large (multi-million) observable sets and quite a few generators, the size of the polynomial scripts encoding geodesic equations can be hundreds of gigabytes or more, substantially exceeding physical memory size in typical workstations. For such calculations, it may be necessary to swap the geodesic equations for individual buckets of observables to and from disk as needed. With the boolean `global.geoswap` set to true, during the script generation phase each thread handling a particular bucket of observables writes the generated geodesic equations for its bucket of observables to disk upon completion and then frees the associated memory. During the numerical evaluation of the geodesic equations, each thread handling a particular bucket of observables first reads the needed portion of the geodesic equations in from disk, performs the numerical evaluation, and then frees the associated in-memory portion of the geodesic equations. This process is repeated during every derivative evaluation required by the ODE integrator, on every Newton minimization step.

The writing to disk during script generation must use a `mutex` lock to ensure tread-safe operation, but for large calculations this imposes a negligible penalty as the time needed to generate the geodesic equations for a substantial bucket of observables easily exceeds the time required to write the newly generated equations to disk. But when numerically evaluating the geodesic equations, the time needed to evaluate each equation in a given bucket is small compared to the typical time needed to fetch that bucket of equations from disk. Multi-threaded execution is still beneficial, but when running on desktop systems

with a solid-state disk capable of transferring 5–6 Gb/sec, three or four active threads can saturate that bandwidth.

4.13 Resource estimates

The average length of Wilson loops having a strong-coupling order M is very nearly the same as M .²⁰ The number $N(M)$ of physically distinct (canonicalized) loops of strong-coupling order M may be roughly estimated as

$$N(M) \approx (3/2 + d/2)^{M-2} M^{-1} / n_{\text{sym}}(d), \quad (10)$$

where d is the cubic lattice dimension and $n_{\text{sym}}(d) \equiv 2^{d+1} d!$ is the dimension of the lattice symmetry group (lattice point group times charge conjugation). For loops with one or two electric field insertions, the corresponding estimates acquire additional factors of M or $M^2/2$, respectively.

The above expression for $N(M)$ is by no means an exact asymptotic form, but for dimensions 3 and 4 agrees tolerably well (to within $\approx 10\%$) with the explicit counts for larger orders shown in appendix B. In two dimensions, the above expression substantially overestimates the true counts by a factor of 10–15. But the depicted exponential growth in the number of loops with increasing strong-coupling order agrees well with the explicit results in appendix B.²¹

Given that, these days, even low-cost desktop computers have multi-Gb memories, one might naively expect that memory capacity would not be a key limiting factor for calculations involving only, say, a few million observables. However, one must also consider the number of generators in use, the typical number of individual terms in each such generator, and how many terms may be generated in the commutator of an observable with a single generator term. Each generator (beyond a few lowest order ones which are invariant under a subgroup of lattice symmetries) will have at least $n_{\text{sym}}(d)$ terms after projection onto the vacuum symmetry representation. For so-called commutator generators, the total number of terms will be $n_{\text{sym}}(d)$ times the number of links in the underlying loop. When commuting with an observable, each term in an **Eloop** generator can act on every link in the observable which lies in the same direction as the electric field insertion in the generator term. So an **Eloop** generator with $n_{\text{sym}}(d)$ terms acting on a length M observable will typically lead to about $n_{\text{sym}}(d) \times L \times M \times 1/d$ distinct terms in the result of the commutator, where L is the length of the generator in question. So for K generators of average length L , the complete set of polynomials which encode the geodesic equations for all **Loop** observables up through order M can contain upwards of

$$K \times n_{\text{sym}}(d) \times L \times M/d \times N(M) \approx K \times L \times (3/2 + d/2)^{M-2}/d \quad (11)$$

²⁰For example, in two dimensions the average length of order 28 Wilson loops is 29.4. In four dimensions, the average length of order 20 Wilson loops is 20.0.

²¹Given that the average length of order M loops is very close to M , one might think that the number of loops of strong-coupling order M would be close to the number of closed, non-backtracking loops of length M . This is not true, as the number of non-backtracking closed loops grows much faster with length, proportional to $(2d - 1)^M$.

individual monomials, each of which is stored as a `PolyTerm` instance. As indicated above, for Euclidean theories each such `PolyTerm` occupies 16 bytes (an 8-byte coefficient and two 4-byte observable indices), while for Hamiltonian theories each `PolyTerm` requires 24 bytes (for a coefficient and four observable indices).

For a 2D Euclidean theory, the above estimate suggests that a calculation using order 28 `Loop`'s and, say, a dozen generators of average length 6 might require upwards of 12 Tb of storage. The actual size is not nearly so large, because the above approximation for $N(M)$ is a substantial overestimate in two dimensions, and because collecting common terms in the result of commutator evaluation leads to substantially shorter expressions. But the actual size of the polynomial script for the described 2D calculation is still 300 Gb! For a 3D Hamiltonian calculation, [...]

This illustrates why polynomial storage, memory utilization, and possible swapping of information to disk are all critical aspects of program design.

4.14 Data structures

Following standard C++ practice, a variety of custom classes are defined and used to represent observables, operators, symmetries, representations, polynomial terms, and numerous other objects as detailed below in section 7. Liberal use is made of standard C++ container classes, especially `std::vector` and `std::unordered_hash`. Letting the standard container classes take care of all memory allocation and deallocation is highly beneficial.

An initial program version used nested standard containers for the multi-dimensional data structures storing the symbolic polynomial scripts defining the gradient and curvature of the Hamiltonian (or free energy) and the geodesic equations. For example, the curvature was initially defined as a `std::array <std::vector <std::vector <std::vector <ObsPoly>>>>` where the successive indices used to access a particular element select, respectively, the symmetry representation (whose possible range is known at compile-time), a particular term in the Hamiltonian (or free energy), and a pair of defined coherence group generators in the given representation. This works fine for smaller calculations where there is ample physical memory available. However, as detailed above, for larger calculations the storage needed to hold all polynomial script information, particularly for the geodesic equations, can become very large. To permit good virtual memory and processor cache performance, it is important to keep the geodesic equation information for a given bucket of observables grouped together in memory. And for calculations where explicit swapping of such information to and from disk is needed, a serialized representation of data which can be efficiently written to or read in from disk is essential. Therefore, the current code stores the entire set of polynomials defining the geodesic equations for a given bucket of observables as a single vector of packed serialized data in which an eight-byte header separates and identifies each distinct polynomial. Similarly, a single such data record is used for the entire set of polynomials comprising the gradient, and further such data records are used for the curvature and Lagrange bracket in each symmetry representation. These packed representations are efficient to process, enable good processor cache and memory performance, and can be directly written to or read from

disk with no conversion needed. (The various data headers used in these data records are described below in section 7.8 and related functions summarized in sections 7.21 and 7.25.)

5 Programming assumptions & conventions

Basic programming environment assumptions are:

- Both `signed char` and `unsigned char` types are available. Objects in the standard C++ container class `std::string<char>` are used to represent observables.
- `short` integers are 16 bits, `int` integers are 32 bits, and `long` integers are 64 bits.
- `float` floating-point numbers are 32 bits, while `double` floating-point numbers are 64 bits.
- The version of C++ available is at least ISO C++ 2020.
- The Intel[®] Threading Building Blocks template library [19] is available to support multi-threaded execution on multi-core systems.
- The Armadillo C++ library for linear algebra and scientific computing is available.

As discussed below, by setting a preprocessor variable `NUM32` the code may be compiled to use 32 bit integers for observable indices and 32 bit floats for polynomial coefficients. When `NUM32` is not set, the default is to use 64 bit variables for both observable indices and polynomial coefficients. Compiling (and running) in `NUM32` mode nearly halves the size of system information files and memory used for polynomial storage, but in this mode the maximum number of retained observables is limited to $2^{31} - 1$, and 63 is the maximum allowable generator or observable order creation order.

Stylistic conventions employed in the code include:

- Variable names are `lower-case`;
- Class names and source file names are `Capitalized`;
- Preprocessor symbols are `FULLY CAPITALIZED`;
- Brace initialization of variables is generally preferred, except for simple integers or when using parentheses avoids the need for a nuisance cast of integer types;
- Opening and closing braces of code blocks are lined up vertically and indented relative to the preceding statement or expression.²² The few exceptions are generally for short, simple inline functions in header files.

²²This is not the most common C or C++ convention, but it is the best choice for program clarity.

6 Compilation & linking

A single `Makefile` handles compilation and linking for versions of the program targeting any theory class. The choice of theory class may be specified by including an argument of the form `THY=theory` on the `make` command line, where *theory* is one of the available preprocessor shorthand theory names listed in Sec. 7.31. One may also define `THY` via an exported shell environmental variable prior to running `make`. A successful compilation will result in an executable named `gordion-theory`, with `gordion` a symlink pointing to that executable. Alternatively, if `THY` is not specified on the `make` command line (or defined as a shell environmental variable), then the target name of an existing `gordion` symlink is used as the default theory selection. An error message stating the necessity of defining `THY` is generated if `THY` is undefined and no default can be inferred from an existing `gordion` symlink.

It is assumed that the C++ compiler supports a `-M` flag to automatically generate dependency files. The command `make clean` removes all `.o` object files, `.d` dependency files, and the target executable. If the setting of `THY` does not match that of an existing `gordion` symlink, then a `make clean` is automatically performed along with replacement of the `gordion` symlink.

If `DBG` is set (for example, by including `DBG=1` on the command line or via an environmental variable), then a debugging compilation with a `-g` compiler flag is performed. Otherwise, an optimized parallel compilation with `-O3 -DPARALLEL` compiler flags is performed.

If `NUM32=1` is include on the `make` command line, then the preprocessor variable `NUM32` will be set and, as indicated above, 32 bit variables will be used for observable indices and polynomial coefficients. Note that `sys-info` and `vev-data` files produced by `NUM32` compilations are incompatible with those from compilations using the default (64-bit) types for observable expectation values and polynomial coefficients.

The environmental variable `CPATH` may need to be suitably defined so that Armadillo and Threading Building Blocks (TBB) include files are found by the compiler. Likewise, `LIBRARY_PATH` may need to be suitably set so that `-ltbb` and `-larmadillo` linker flags suffice for `ld` to access the Armadillo and TBB object libraries.

7 Source files

The following descriptions of each source file aim to provide a overview of the code at an intermediate level of detail, highlighting the class structure, summarizing the operation of each function, and hopefully making it easier to examine and understand the actual source code.

7.1 `Assess.{h,cpp}`

Header file `Assess.h` includes `Theory.h` and defines:

- `class Site`, holding information relevant to finding and examining self-intersections in observables. Data members are:
 - `ulong point`, recording a site's coordinates relative to the first symbol in the observable;
 - `short pos`, the site's position along the `SymbStr`;
 - `short nEs`, recording the number of preceding electric field insertions in the `SymbStr`;
 - `Symb in` and `Symb out`, recording the `symb`'s which precede and follow this location in the `SymbStr`.

Member functions `operator==` and `operator<` are defining for testing equality or the relative ordering of coordinates with another `Site`.

- `class Intersect`, which holds information relevant to choosing a pair of coinciding `Site`'s at which to factor an observable into sub-observables. Data members are:
 - `float score`, discussed below;
 - `short pos1` and `short pos2`, giving the two positions along the `SymbStr` corresponding to an intersection at which the observable might be factored;
 - `short nEs`, the number of intervening electric field insertions between `pos1` and `pos2`.
 - `static constexpr int maxtry`, which sets the maximum number of factorizations to try when attempting to approximate an observable.

The member function `operator<` is defined for sorting of `Intersect`'s.

File `Assess.cpp` includes `Assess.h`, `Global.h`, `Commute.h`, `Blab.h` and `Gripe.h`, and defines the following functions (all of which are declared in `Obs.h`):

- `ObsList::assess`, whose single argument is an `Obs` generated by the commutation routines. This function contains the logic used to decide whether an observable should be stored in the given `ObsList`, discarded, or approximated, if possible, by a product of observables already stored in the list. The appropriate disposition depends on the list in question, the phase of the program leading to the function call and, for the main list of canonicalized observables, whether the strong-coupling order of the observable (equal to the sum of its creation and expectation orders) falls within the limit set by `global.maxord()`. While the commutation routines easily track the creation order of generated observables, determining the expectation of an observable is non-trivial. When needed, performing this classification is part of the task of this routine. This classification process, described below, can lead to recursive calls to the commutation routines, and hence to `Obs::assess` as well, so coding allowing reentrant calls is essential.

The basic steps are as follows:

- Depending on the setting of the `canonicalize` data member of the `ObsList`, the observable is either fully canonicalized, by calling `Obs::canon`, or, for gauge observables, cyclically rotated to a unique preferred starting location, by calling `Obs::findstart`. (Note that these transformations are done in-place on the observable, which is passed by reference.)
- The hash table of the `ObsList` is used to find the observable in the list, if already present. If so, a `PolyTerm` is returned recording the list index of the observable plus a possible sign from canonicalization.
- If the observable is not found, then if the `ObsList::frozen` function returns true, indicating further additions to the master observable list are disallowed, then the setting of the `ObsList::approx` boolean controls whether the observable is discarded, signaled by the return of a vanishing `PolyTerm`, or if the function `Obs::approximate` is called to attempt construction of a suitable `PolyTerm` approximation.
- When `ObsList::frozen` returns false, if `ObsList::classify` is set then the routine `Obs::classify` is called to determine the expectation order of the observable. (The `ObsList::classify` boolean is only set true for the main `ObsList::obs` list.) If the classification function return value indicates that the observable's strong coupling order equals the target set by `global.maxord()`, then `Obs::retain` is called to retain the observable for addition to the master observable list. Otherwise the observable is discarded.
- If `frozen` and `classify` are both false, then the observable is simply stored in the `ObsList`.

In every case, a `PolyTerm` is returned recording the list index of the observable, if found or stored, plus a possible sign from canonicalization. (For retained observables which are yet to be stored, the index in the returned `PolyTerm` is just an arbitrary placeholder which will not be used.) A vanishing coefficient in the returned `PolyTerm` signals that the observable is to be discarded.

- `Obs::classify`, which attempts to determine the expectation order of an observable. For observables containing electric field insertions, this routine first calls `Obs::noEbound`, and for pure gauge observables `Obs::u1bound` is called. These two routines, as described below, quickly generate lower-bounds on the expectation order of an observable which, very often, are sufficient to determine that the observable's strong-coupling order is larger than the limit set by `global.maxord()`, and hence full classification is not needed. But if the lower bound indicates that the observable might fall within the `maxord` selected set, then either `Obs::reduce`, for observables with electric fields, or `Obs::factorize`, for observables without, are called to do a full classification.
- `Obs::factorize`, which calls `Obs::sitelist` to generate a sorted list of `Site` objects corresponding to every location along the `SymbStr` forming the observable. The `factorize` routine then loops through all instances of coincident `Site`'s. Each such self-intersection defines a possible split of the observable in which a sub-loop

is excised. Both the sub-loop and the remainder of the observable (with sub-loop excised) are looked for in the list of known observables. If both are found, then the sum of their two expectation orders provides a possible expectation order for the initial observable. The observable's `xorder` value is set to the minimum value from all such factorizations, provided this value is not larger than the observable's creation order (if set). For fermions, in addition to factorizations where a gauge sub-loop is extracted from the observable, it is also necessary to test three-way factorizations in which fermion/conjugate-fermion pairs are inserted at both cuts in the observable, transforming it into a product of three fermion bilinears. This enables proper treatment of cases where the correct expectation order of the sub-loop comes not from fully tiling the sub-loop with plaquettes but instead involves fermion hopping contributions. (In other words, cases where perimeter-law effects compete or mix with area-law contributions.)

This routine, as well as the following `approximate` and `noEbound` routines, uses a `SymbStr` buffer in which to construct the terms emerging from a possible factorization or transformation, with each constructed `SymbStr` sequence in the buffer later emplaced in an `Obs`. Because these assessment routines can be called from different threads during the generation of geodesic equations (when multi-threading is enabled) this `SymbStr` buffer, and its analogs in other assessment routines, cannot be declared `static`. The `SymbStr` buffer could be defined as a normal block-scoped variable, with space on the free store (for strings longer than the short string optimization size of `std::string`) dynamically allocated and deallocated by the `std::string` routines on every pass through the block. Alternatively, this buffer could be declared `thread_local`, which should minimize the number of needed free store allocation and deallocation operators, but at the cost of threading specific overhead associated with any thread local variable. Direct comparison (on the multi-core development systems used) found that for these assessment routines `thread_local` storage of the needed `SymbStr` buffers is preferable. Similarly, `thread_local` storage is used for the `vector<Site>` list returned by `Obs::sitelist` (as well as the `vector<Intersect>` used in the following `approximate` routine).

- `Obs::approximate` which, (similar to `factorize`) first calls `Obs::sitelist` to generate a sorted list of `Site` objects identifying every location along the `SymbStr` forming the observable. The routine then loops through all instances of coincident `Site`'s defining a possible split of the observable into sub-observables. Each such possible split is assigned a numeric score which is used to select preferred possible factorizations when there are multiple possibilities. This scoring is defined to prefer more even factorizations for pure gauge observables, increasing the likelihood that both factored sub-observables will already be present in the `ObsList`, while for fermion bilinears the scoring favors splits leading to larger gauge sub-loops with smaller fermion “tails” which, for fermion bilinears shorter than the string-breaking scale, should also increase the likelihood that both sub-observables are already known. Smaller contributions to the score are designed to disfavor splits adjacent to electric field insertions, and at multiply-traversed links.

After sorting the scored list of possible intersections, the most highly scored possible

splits are sequentially tested to see if the two possible sub-observables are already known. To do so, two **Obs** instances are constructed, one representing the loop inside the cuts and the other the remainder of the original observable. If both are already present in the **ObsList**, then a **PolyTerm** with this factorization is returned. If no successful factorization is found after **Intersect::maxtry** attempts, then the process aborts with failure indicated via a zero coefficient of the returned **PolyTerm**.

- **Obs::sitelist**, which is called by both **Obs::factorize** and **Obs::approximate** and prepares a **vector<Site>** containing a sorted list of site coordinates (relative to an origin arbitrarily defined as the beginning of the first **symb** in the **SymbStr**) of every site traversed by the observable, together with the **symb**'s leading into and out of each site, and the number of preceding electric field insertions.
- **Obs::reduce**, which explicitly commutes the observable in question with primary **Op**'s in order to identify which such operation leads to terms already present in the **ObsList** and whose sum of expectation orders plus the order of the generator **Op** is minimal. As discussed in Ref. [3], properly executed, the result defines the expectation order of the initial observable.²³ For **Eloop** observables (i.e., loops containing one electric field insertion), commutation with a single plaquette (or a primary Wilson line) is tested, thereby leading to no- E loops which will already be known (up to the specified **maxord()** limit), so if no term in the commutation result leads to a known loop, the expectation order of the initial **Eloop** must lie beyond the **maxord()** limit. For **Fermion** observables, commutation with single link fermion hopping **Op**'s (those whose staggering is such that they map phase space coordinates to phase space coordinates) are tested with, again, at least one such commutator evaluation necessarily leading to a lower (expectation) order fermion bilinear which will already be known if the original bilinear is destined to be retained. For **Efermion** observables (i.e., fermion bilinears with one electric field insertion), both plaquette (& Wilson line) generators as well as single link fermion hopping generators are tested, while for **EEloop** observables (i.e., loops with two electric field insertions), in addition to commutation with basic plaquettes (& Wilson lines), evaluation of the pure-loop terms which can emerge from commutation with single- E plaquettes (and single- E inserted Wilson lines) is also performed.
- **Obs::u1bound**, which for pure gauge loops, generates a lower bound on the expectation order the loop would have in a $U(1)$ pure gauge theory which, in turn, provides a lower bound on the expectation order in the $U(N)$ theory. The routine first finds a bounding-box for the loop. Then for each $[ij]$ coordinate plane, it “projectively” computes the net $U(1)$ flux which span each stack of $[ij]$ -oriented plaquettes (i.e., those $[ij]$ -oriented plaquettes having the same i - and j -coordinate values, irrespective of their locations in any other coordinate directions). The sum of absolute

²³There is a subtlety involving whether one first collects terms resulting from the commutation which, in translationally invariant states, can lead to cancellations for certain observables such as **[[AXAx]]** or **AXXAx**. Such exact cancellations would not occur in, for example, were the theory to have a varying position-dependent coupling constant. As such variations have no bearing on the structure of the strong-coupling expansion, the **reduce** routine treats the separate terms in these special-case cancellations as establishing a possible expectation order.

values of these fluxes, summed over both all $[ij]$ stacks and then over all $[ij]$ pairs, gives the $U(1)$ lower bound. In two dimensions, this would be the correct $U(1)$ expectation order, but in higher dimensions this is only a lower bound as a true minimal spanning surface may have opposite signs of flux on differing plaquettes within the same coordinate plane stack.

- **Obs::noEbound**, which removes electric field insertions from the observable and then looks to see if the resulting **Loop** or **Fermion** is already known. If so, the expectation order of the no- E observable provides a lower bound on the expectation order of the initial observable. If the no- E observable is not found, this implies that the expectation order of the initial observable is larger than the limit used in generating the current set of no- E observables. (As discussed in Ref. [3], this presumes that observables are generated progressively in order of increasing number of electric field insertions.)

7.2 Blab.{h,cpp}

Header file **Blab.h** includes **Gordion.h** and defines:

- **enum** **BLAB**, whose members name the various major **.ccp** files, and whose integer conversions serve as indices into the static array **Blab::blablevel** whose values control the verbosity of different routines.
- **class** **Blab**, with inline static data members **blablevel** and **blabmap** holding verbosity control information, and static functions **setblab** and **resetblab** for initializing, setting and resetting verbosity levels, respectively.

File **Blab.cpp** includes **Blab.h** and **Gripe.h**, and defines the functions **Blab::setblab** and **Blab::resetblab**.

7.3 Build.{h,cpp}

Header file **Build.h** includes **Gordion.h** defines:

- **namespace** **Build**, which declares the various **mk_*** and **do_*** functions described below involved in the the construction of observables, symbolic expressions for the geodesic equations, and derivatives of the Hamiltonian or free energy.

File **Build.cpp** includes **Build.h**, **Canon.h**, **Commute.h**, **Numerics.h**, **Rep.h**, **Save.h**, **Blab.h** and **Gripe.h**, and defines the following functions:

- **Build::clearobs** which, depending on the stage argument, removes either all observables, or all fermion observables, from both **ObsList::obs** and **Canon.cache**, and calls **Global::clearpolys** to clear the associated script polynomials.

- **Build::mk_obs** which clears any previously build observables for the current minimization stage beyond the specified target strong-coupling order, initializes the master observable list for the current minimization stage if needed and then progressively increments the maximal strong-coupling order of observables up to a specified target, generating on each pass all observables of a given strong-coupling order. Prior to this loop, booleans **ObsList::obs.freeze** and **Canon::cache.freeze** are set to **true** so that additions to the observable list and canonicalization cache may be made, and **ObsList::obs.approx** is set to **false**. Within the loop, when **global.stage == Global::Gauge** the routine **Build::mk_loops** is called, followed (for Hamiltonian theories) by **Build::mk_Eloops** and **Build::mk_EEloops**. Alternatively, when **global.stage == Global::Fermi**, a temporary list of primary conjugate momenta **Op**'s is constructed for later use by the multi-threaded routines **do_Fermion_bckt** and **do_Efermion_bckt**. Then **Build::mk_fermions** is called, followed (for Hamiltonian theories) by **Build::mk_Efermions**. Finally **ObsList::do_fermiinit** is called to generate the fermion initialization map. Upon the completion of the the observable generation loop, if the observable list has lengthened then previously constructed polynomial scripts are cleared, and **global.mk_bcktlist** and **numerics.initialize** are called to update the relevant observable bucket list (used for geodesic script generation), and initialize the observable expectation value list **numerics.vev**. Lastly, the temporary **Obsset**'s **newobs** and **ObsList::inbox** are destroyed, **ObsList::obs.approx** is reset to the value of **global.approx**, and **ObsList::obs.freeze** and **Canon::cache.freeze** are set to **true**.
- **Build::mk_loops**, which loops over target creation orders running from half the specified strong-coupling order limit up to four less than the strong-coupling order limit, then runs a (normally parallelized) **for_each** loop over buckets of observables calling **do_Loop_bckt** to perform the generation of **Loop** observables starting from a given bucket of lower order observables. Following this **for_each** loop, any newly generated observables of the specified order are inserted into the master observable list **ObsList::obs** and any new short observables are added to the canonicalization cache. This routine, along with the other **mk_*** observable generation routines, also updates the value of **nobsG** or **nobsF** in **ObsList::obs** to keep track of the total number of gauge and fermion observables.
- **Build::mk_Eloops**, which builds **Eloop** observables using a similar procedure as **mk_loops**, but looping over target creation orders running from half the specified strong-coupling order limit up to either two less than the strong-coupling order limit if **maxord** is four, or else four less than **maxord**, and calling **do_Eloop_bckt** to perform the observable generation.
- **Build::mk_EEloops**, which builds **EEloop** observables using a similar procedure as **mk_loops**, but looping over target creation orders running from two less than half the specified strong-coupling order limit up to four less than the strong-coupling order limit, and calling **do_EEloop_bckt** to perform the observable generation.
- **Build::mk_fermions**, which builds **Fermion** observables using the analogous procedure as **mk_loops**, but with target creation orders running from half the specified

strong-coupling order limit up to two less than the strong-coupling order limit, and calling `do_Fermion_bckt` to perform the observable generation.

- `Build::mk_Efermions`, which builds `Efermion` observables using a similar procedure as `mk_fermions`, but looping over target creation orders running from half the specified strong-coupling order limit up to one less than the strong-coupling order limit, and calling `do_Efermion_bckt` to perform the observable generation.
- `Build::mk_grad`, which loops over all terms in `global.info().Hterms`, evaluating their gradient with respect to all T -even vacuum representation generators (of either gauge or fermi type, depending on the current minimization stage), by calling `commute_poly` for each term and each such generator. Results are stored in `global.info().gradient`.
- `Build::mk_curv`, which loops over all terms in `global.info().Hterms`, evaluating their second variations with all generators in the specified symmetry representation (of either gauge or fermi type, depending on the current minimization stage), by calling `commute_poly` for each term and one generator, and then calling `commute_poly` with that intermediate result and the second generator. Time reversal symmetry implies that the resulting curvature matrix is block-diagonal; this is taken into account. A temporary `ObsList` is used to store the intermediate results of the first commutation. For the vacuum representation, terms resulting from the first commutation may be canonicalized, but not for non-vacuum representations. The resulting curvature matrix expressions are stored in `global.info().curv[global.repnum]`.
- `Build::mk_lagr`, which evaluates the commutator of pairs of generators in the specified symmetry representation (of either gauge or fermi type, depending on the current minimization stage), by calling `commute_gen`. Time reversal symmetry implies that the resulting Lagrange matrix is block off-diagonal; this is taken into account. Results are stored in `global.info().lagr[global.repnum]`. A warning is given if the list of observables does not contain those which result from the commutator of any two generators, based on a presumption that spectrum results are likely unreliable if observable truncation affects the Lagrange bracket itself. (In such cases, suspending highest order generators in spectrum calculations may be appropriate.) To avoid such truncation in the Lagrange bracket, the observable truncation order should be at least four times the maximal generator creation order.
- `Build::mk_geos`, which evaluates the symbolic geodesic equations for observables of either pure gauge or fermionic type, depending on the minimization stage. To enable efficient multi-threading, the sum over observables is partitioned into multiple buckets, with a `for_each` loop over the list of buckets calling `Build::mk_geo_bkt` for each bucket of observables. As noted above, the setting of the preprocessor symbol `PARALLEL` controls whether a single-threaded or multi-threaded `for_each` loop is used.
- `Build::do_Loop_bckt`, which loops over all observables in a specified bucket, selecting those which are `Loop`'s, and then over all primary `Eloop Op`'s (which are

plaquettes with an electric field insertion, figure eights with an electric field on the crossover link, or related Wilson lines with one electric field insertion). For each such `Op,Obs` pair whose creation orders sum to the target value and whose order classification is such that the commutator may include observables of the desired strong-coupling order, `do_commute` is called to evaluate the commutator. Within `do_commute`, `Obs::assess` is called to handle the classification by expectation order for each observable appearing in the generated terms, and storage in the `thread_local ObsList::inbox` if the resulting strong coupling order equals the target set by `global.maxord()`. After completing all commutations for the designated bucket of observables, a `mutex` lock is acquired, protecting the merging of new additions to `Build::newobs` set of observables, which will be merged into the master `ObsList::obs` at the end of the observable generation pass.

- `Build::do_Eloop_bckt`, which generates new `Eloop` observables starting from a given bucket of initial observables, in a manner analogous to that of `do_Loop_bckt`. New `Eloop`'s, of the specified creation and strong-coupling order, are generated either by commuting an existing `Loop` with a gauge kinetic (`EE`) operator, or by commuting an existing `Eloop` with a primary `Eloop Op`. The same `mutex` protected procedure is used to merge new `Eloop`'s back into `Build::newobs`.
- `Build::do_EEloop_bckt`, which generates new `EEloop` observables starting from a given bucket of initial observables, in a manner analogous to that of `do_Loop_bckt`. New `EEloop`'s, of the specified creation and strong-coupling orders, are generated by commuting an existing `EEloop` with a primary `Eloop Op`. The same `mutex` protected procedure is used to merge new `Eloop`'s back into `Build::newobs`.
- `Build::do_Fermion_bckt`, which generates new `Fermion` observables starting from a given bucket of initial observables, in a manner analogous to that of `do_Loop_bckt`. New `Fermion`'s, of the specified creation and strong-coupling orders, are generated by commuting an existing `Fermion` with either a primary `Eloop Op` or a primary conjugate momentum `Fermion Op` (with, for Hamiltonian theories, implies the correct staggering so that the resulting `Fermion` observables are phase space coordinates, not conjugate momenta). The same `mutex` protected procedure is used to merge new `Eloop`'s back into `Build::newobs`.
- `Build::do_Efermion_bckt`, which generates new `Efermion` observables starting from a given bucket of initial observables. New `Efermion`'s, of the specified creation and strong-coupling orders, are generated by commuting an existing `Fermion` with a gauge kinetic `EE` operator, or by commuting an `Efermion` with a primary `Eloop Op` or primary conjugate momentum `Fermion Op`. The same `mutex` protected procedure is used to merge new `Eloop`'s back into `Build::newobs`.
- `Build::do_geo_bxkt` which loops over each observable in its allocated bucket, and then over all time-reversal even generators in the vacuum representation, calling `commute_poly` to evaluate the corresponding commutators and storing the results in `global.info().geos[bcktnum]`. Following this, `check_xorder` is called to test for inconsistency in the assignment of each observable's expectation order, and the

number of terms of various orders are counted in the newly evaluated geodesic equations and added to the relevant geodesic statistics counters in `global.count()`. Finally, if `global.autosave` is set true, then `Save::write_geo_bckt` is called to write the geodesic equations for the bucket of observables to disk.

- `Build::check_xorder`, which examines each term of a geodesic equation, checking to see if the sum of the expectation orders of the observables in the geodesic equation term, plus the order of the generator producing this term, is ever less than the assigned expectation order of the initial observable. If so, this indicates an inconsistency in the classification of this observable during the observable generation phase, which is reported.
- `Build::do_geostats`, which calls `do_geostat_bckt` in a `for_each` loop to reevaluate the geodesic equation statistics, including the number of polynomial terms and their breakdown by monomial order.
- `Build::do_geostat_bckt`, which scans the symbolic geodesic equations for a specified bucket of observables, adding the appropriate counts to the `ngeos`, `geoterms` and `geotermord` elements of `global.count()`.

The file also defines the static variables:

- `int cord`, used to pass the target creation order to the various `do_*_bckt` observable generation routines;
- `Obsset newobs`, used to hold a growing set of newly generated observables before their insertion into the master observable list at the end of each observable generation pass;
- `nector<Op> primopOps`, used to hold the set of primary conjugate momentum `Op`'s, for use by `do_Fermion_bckt` and `do_Efermion_bckt`;
- `std::mutex obsmutex`, used to synchronize additions to `Obsset newobs` from different threads;
- `constexpr int single_thread=1024`, which sets a threshold for when to switch to multi-threaded generation of further observables.

7.4 Canon.{h,cpp}

Header file `Canon.h` includes `Index.h` and `Obs.h`, and defines:

- `constexpr` values `loopchunksize` and `specchunksize`, defining the chunk sizes used for canonicalizing loop and non-loop observables, respectively.
- `constexpr` functions `looptblsize` and `spectblsize`, computing compile-time dimensions of the canonicalization tables.
- `constexpr` function `ipow`, providing compile-time evaluation of integer powers.

- type alias `SymmSet`, defined as `vector<uint>` and whose instances hold a list of indices of some subset of symmetry transformations contained in `Symm::list`.
- `class Node`, used for entries in the canonicalization tables, with each `Node` instance containing a chunk score and associated symmetry subset index.
- `class CanonCache`, which inherits from `std::hash<string,int>`, contains one data member `bool freeze` signaling when cache additions are allowed (only during single-threaded operation), and defines the member functions:
 - `purge`, which removes cache entries whose observable index is above a specified limit;
 - `load`, which loads suitable short members of an `Obsset` of soon-to-be-stored observables into the cache;
 - `reload`, which loads all suitable short observables from `ObsList::obs` into the cache,

and includes a `friend ostream& operator<<` declaration for printing the cache.

- `class Canon`, which only contains static functions and data members. The static data members are:
 - `inline vector<Node> looptable`, the canonicalization table for pure gauge loops;
 - `inline vector<Node> spectable`, the corresponding table used for other observables;
 - `inline Index<SymmSet> symmset`, an indexed list of possible symmetry subsets;
 - `inline CanonCache cache`, used for observables whose lengths are less than the size of the symbol chunks used for indexing into the main canonicalization tables. This class inherits from `hash<string,int>` and contains one additional boolean `freeze` which signals when additions to the cache are disallowed.
 - `inline struct Stats stats`, holding various canonicalization-related statistics.

Static functions in the class are:

- `speccode`, which computes the `spectable` index corresponding to a special observable chunk;
- `looptblinit` and `spectblinit`, for initializing the canonicalization tables;
- `loopchunk` and `specchunk`, used in those initialization routines for converting a table index back into the corresponding observable chunk.

File `Canon.cpp` includes `Canon.h`, `Symm.h`, `Global.h`, `Gripe.h` and `Blab.h`, and defines the functions:

- **Obs::canon**, which transforms an observable into its canonicalized form (in-place), returning a possible associated sign. This function is a key routine whose efficiency is critical. The general strategy was described above. In a bit more detail:

Loop observables (only containing gauge links with no electric field insertions) that are at most **loopchunksize** in length are converted into lookup keys for **Canon::cache**. If not found (a cache miss), then all possible symmetry transforms, including cyclic rotations, of the loop are generated, and the canonical form based on lexical ordering is selected. If **Canon::cache.freeze** is false, the canonical form is stored in the **ObsList::obs** observable list, and **Canon::cache** entries are created which map all symmetry-equivalent forms of the observable into the correct **ObsList::obs** entry. If need be, prior to storing **Obs::factorize** is called to determine (and set) the expectation order for the short loop. (The canonicalization cache is frozen, along with the main **ObsList::obs** list of observables, after the observable generation stage.) For loop observables longer than **loopchunksize**, a moving “window” of **loopchunksize** is walked around the loop, with the lowest scoring **Canon::looptable** entries for all such chunks providing the small subsets of symmetry transformations needing to be tested to identify the preferred starting position and symmetry transformation for the unique canonicalized form of the full loop. In the special case of a one-dimensional lattice, the loop over all possible starting positions is needless, and is omitted.

“Special” observables (fermion bilinears or gauge loops containing electric field insertions) are treated similarly, except that only transformations which lead to symbol strings starting with an appropriate non-link symbol need be considered. The **Canon::cache** is used for observables of length **specchunksize** or less (with **Obs::factorize** or **Obs::reduce** called, if necessary, to determine the observable’s expectation order prior to storing in the cache), while for longer observables **specchunksize** chunks which either begin or end on the endpoints of a fermion bilinear, or on electric field insertions in gauge loops, are converted into **Canon::spectable** indices, with the lowest scoring entries for these chunks then providing the possible subsets of symmetry transformations and starting positions needing to be tested to identify the preferred canonicalized form. Because (for Hamiltonian theories) observables may contain two electric field insertions, the indexing of a chunk starting with one electric field insertion must allow for the possibility of a second such insertion within the chunk; this makes the indexing of chunks for **Canon::spectable** different and more involved than for **Canon::looptable**, and leads to smaller values for **specchunksize** than **loopchunksize**. In the spacial case of a Hamiltonian theory on a one dimensional lattice, Gauss’ law implies that the position an electric field insertion may be freely moved along a gauge observable. This freedom is used to position both electric field insertions in **EEloop**’s at the beginning of the observable.

- **Canon::looptblinit**, which initializes **Canon::looptable**. For every valid sequence of link symbols (or “chunk”) of length **loopchunksize**, all possible symmetry transforms of the chunk are generated and scored (based on lexical ordering of **symb** strings) with the minimal score and associated symmetry subset (consisting

of all symmetries which transform the initial chunk into the minimal score chunk) stored in the `Node` of `looptable` at the table index generated from the initial chunk.

- `Canon::spectblinit`, which initializes the special (i.e., non-loop) observable canonicalization table `Canon::spectable`. For every valid `specchunksize` sequence of symbols, necessarily starting with an initial special (a `symb` representing either a fermion or a link with one or two electric field insertions), all possible symmetry transforms of the chunk are generated and scored, with the minimal score and associated symmetry subset (consisting of all symmetries which transform the initial chunk into the minimal score chunk) stored in the `Node` of `spectable` at the table index generated from initial chunk.
- `Canon::loopchunk` and `Canon::specchunk`, which are used inside the corresponding initialization functions to determine if a given canonicalization table index value corresponds to a valid symbol string.
- `Canon::speccode`, used inside `Obs::canon` to compute the `spectable` index corresponding to a possible starting chunk.
- `friend ostream& operator<<`, for printing a `CanonCache`.

7.5 Commute.{h,cpp}

Header file `Commute.h` includes `Poly.h` and `Gen.h`, and defines

- `namespace Commute`, which declares the various functions in `Commute.cpp` described next. This namespace also defines the trivial convenience functions:
 - `cordsum`, which sums the creation orders of an `Op` and `Obs`, but only if the `Obs` creation order is set.
 - `commtype`, which returns the `ObsType` resulting from the commutator of an `Op` of any type with an `EEloop` or `Efermion`.

File `Commute.cpp` includes `Commute.h`, `Global.h`, `Splice.h`, `Gripe.h`, `Blab.h`, and contains the functions which evaluate the commutator of generators with observables, or of pairs of generators. These functions (along with those in `Canon.cpp`) form the most performance-critical heart of the program. Functions defined in `Commute.cpp` are:

- `Commute::commute_poly`, which takes as arguments references to a coherence group generator (`Gen`), an observable polynomial in either `ObsPoly` or `PolyMap` form, and an `ObsPoly` for holding the result. The `commute_poly` routine loops over the sequence of monomial terms in the supplied polynomials and calls `commute_term` to do the indicated commutation.
- `Commute::commute_term`, which takes as arguments (references to) a `Gen` and a `PolyTerm`, whose commutator is to be evaluated, the `ObsList` to which the `PolyTerm` refers, and a `PolyMap` to hold the result. The routine loops over the multiple factors

to be acted upon in quadratic or higher order monomial terms. Within these loops, this function directly handles the special case (for Euclidean theories) of variations in entropy. For other cases, this routine then loops over the individual `Op` terms in the generator and calls the further routine `do_commute`. A second sub-function `do_split` is also called to add the terms, potentially cubic in loops, which arise via operator ordering in the commutator of an `Eloop` and an `EEloop`. (See Ref. [3] for an explanation of how such terms arise via normal ordering.)

- `, Commute:do_commute`, which takes as arguments (references to) an `Op`, `Obs`, and `PolyMap` for the result, plus a `PolyTerm` and associated `ObsList` representing further overall factors, not part of the commutation, to be included in the result. Depending on the type of operator and observable, the routine calls the actual evaluation routines `do_commuteA`, `do_commuteB`, `do_commuteC`, `do_commuteD` and `do_commuteE`, described below.
- `Commute::do_commuteA`, which is called when the generator `Op` term is an `Eloop`, and handles terms arising when that electric field operator differentiates a link (possibly containing an electric field insertion) in the observable which the `Op` acts on. Loosely speaking, an `Eloop Op` splices the loop of the `Op` into the observable at all possible locations where the electric field operator can act. The splice table, contained in `Splice.h` and described below, encodes details of this splicing operation. The function `SymbStr::join` is repeatedly called to handle the joining of one symbol string to another, appropriately taking care of canceling adjoining links and inverse links, or the combining of electric field insertions and associated links into the correct “ligatures” defined in the basic `symb` encoding. `ObsList::assess` is called for each resulting observable to determine, whenever possible, the correct expectation order for the observable and to decide (based on the values of `global::maxord` and the the creation and expectation orders of the observable) whether the observable should be stored in the relevant observable list, approximated by products of lower order observables, or neglected.

This routine, as well as the following commutation routines, uses a `SymbStr` buffer in which to construct the various terms emerging from the commutator, with each constructed `SymbStr` sequence in the buffer later emplaced in an `Obs`. Because the commutation routines can be called from different threads during the generation of geodesic equations (when multi-threading is enabled) this `SymbStr` buffer, and its analogs in other commutation routines, cannot be declared `static`. The `SymbStr` buffer may be defined as a normal block-scoped variable, with space on the free store (for strings longer than the short string optimization size of `std:string`) dynamically allocated and deallocated by the `std:string` routines on every pass through the block. Alternatively, this buffer could be declared `thread_local`, which should minimize the number of needed free store allocation and deallocation operators, but at the cost of threading specific overhead associated with thread local variables. In contrast to the situation found for the observable assessment routines, for the commutation routines direct comparison (on the multi-core development systems used) found that `thread_local` storage of the needed `SymbStr` buffers led

to a performance penalty, not a speed-up, compared to the local scope alternative. This, presumably, reflects a higher ratio of buffer-related operations compared to allocation/deallocation overhead in the commutation routines. Hence, the `SymbStr` buffers for these routines are all defined to have purely local scope.

- `Commute::do_commuteB`, which is called when the `Obs` observable is an `Eloop`, and handles terms arising when this electric field differentiates links (possibly with electric field insertions) in the generator `Op`. `ObsList::assess` is again called to determine the disposition of each generated term.
- `Commute::do_commuteC`, which is called when the `Obs` observable is an `EEloop`, and handles terms arising when the initial electric field operator of the `Obs` differentiates links (possibly with an electric field) in the generator `Op`. A complication in this routine is that the various splittings must be carried out in such a manner that the resulting symbol string, containing one or two electric field insertions, either ends up with an electric field inserted link as the initial symbol of the string if the `Op` is an `Eloop`, but the initial symbol remains a fermion when the `Op` is a fermion bilinear.
- `Commute::do_commuteD`, which is called when the `Obs` is an `EEloop` or `Efermion`, and handles terms arising when an electric field insertion not at the beginning of the `Obs` differentiates a link in the generator `Op`.
- `Commute::do_commuteE`, which is called when both the generator `Op` term and the `Obs` observable are both fermion bilinears, and handles terms arising when a fermion in one factor anti-commutes with a conjugate fermion in the other. This routine also handles the generation of the pure loop term which arises, via operator ordering, when both bilinears form closed loops. (Again, see Ref. [3] for details.)
- `Commute::do_split`, which constructs the pure `Loop` terms, potentially up to cubic order, which arise due to normal-ordering in commutators of an `Eloop` with an `EEloop`. (See Ref. [3] for an explanation.) The routine loops over the terms in the generator reduction, calls `do_commuteC` to evaluate the commutator of the `EEloop` with either of the `Loop` factors in the reduction term, and then calls `do_inner` to perform the “inner” commutation of that intermediate result. A temporary `ObsList` is used to hold (and collect terms from) the results of the first commutation step prior to the `do_inner` final reduction. As with the `SymbStr` buffer, a block-scoped temporary `ObsList` proves preferable to `thread_local` storage.
- `Commute::do_inner`, which performs the reduction of an `Eloop` observable to the pure loops, or quadratic polynomial of loops, which result when the electric field acts on links of the observable itself. In other words, if $\text{tr}(E_\ell U(C))$ denotes an operator in which a Wilson loop $U(C)$ has an electric field insertion on some link ℓ , `do_inner` evaluates the commutator $\text{tr}[(E_\ell)_{ij}, U(C)_{ji}]$.
- `Commute::commute_gen`, which evaluates the commutator of two generators by looping over the `Op` terms in each and calling `op_commute` to do the work, and finally calling `Poly::collect` to collect terms in the result.

- `Commute::op_commute`, which evaluates the commutator of two `Op`'s, with code very similar to that in `do_commuteA` and `do_commuteE`.

7.6 Counter.h

Header file `Counter.h` includes `Theory.h` and defines:

- `static constexpr int cachelinelen = 128`, which is used in the following alignment specification. Its value is set equal to the cache line size on Apple silicon (which is twice that of Intel processors);
- `class Counter`, which is defined with `alignas (cachelinelen)` and holds a single data member `std::atomic<ulong> x`. Definitions are included for the functions `operator++`, `operator=`, `operator+=`, a conversion operator to `ulong`, and `friend ostream& operator<<`. These enable a `Counter` to be used much as if it were a simple `ulong`. Instances of `Counter` are used for statistics counters which may be updated by multiple threads.
- `class CountArr<N>`, a templated class which inherits from `array<Counter,N>` and adds member functions:
 - `operator=`, for either assigning a single value to all counters in the array, or copying an equal-sized `ulong` array into the array of counters;
 - `operator+=`, for incrementing all the counters in the array by the values in an equal-sized `ulong` array.
- `struct Counters`, whose instances will hold a variety of statistics `Counter`'s which are incremented by the canonicalization, observable assessment and polynomial script generation routines, and which are reported by the `print stats` and `print geostats` commands. A member function `cleargeostats` is defined to zero the geodesic-related counters.

7.7 Coupling.{h,cpp}

Header file `Coupling.h` includes `Obs.h` and defines:

- `class Coupling`, which inherits from `char8`, holding the name of the adjustable coupling constant, and adds data members:
 - `doub value`, holding the numerical value of the coupling;
 - `int stage`, indicating of the coupling is used (first) in the gauge or fermion minimization stage.

The static data member `vector<Coupling> list` holds the list of all defined adjustable coupling constants, while the static member function `indx` takes a possible coupling name as argument (as either `string` or `char8`) and returns the index

into `Coupling::list` of the specified coupling, if present, or else `-1`. A `friend ostream& operator<<` definition is included for printing the `Coupling` name.

- type alias `Couplings = vector<Coupling>`.
- `struct Factor`, holding an index into `Coupling::list` plus a power of the indicated coupling constant.
- `class Coeff`, inheriting from `vector<Factor>` and holding a list of adjustable coupling and associated powers. One member function, `doub operator()`, returns the numerical value of the product of powers of adjustable couplings.
- `class AdjTerm`, whose instances represent polynomials in observables with a common overall adjustable coefficient. Class data members are:
 - `Coeff coeff`, the adjustable coefficient for the term;
 - `ObsPoly poly`, the non-canonicalized form of the observable polynomial for this term;
 - `ObsPoly cpoly`, the canonicalized form of the observable polynomial.

A `friend ostream& operator<<` declaration is included for printing an `AdjTerm` vector.

File `Coupling.cpp` includes `Global.h` and defines `ostream& operator<<` for `AdjTerm` vectors. If the coupling exponent is non-zero, it outputs the coupling name and associated exponent (if not unity), and then prints the term's observable polynomial (parenthesized).

7.8 Data.h

Header file `Data.h` includes `Gordion.h` and `Gripe.h`, and defines:

- `struct ObsHdr`, with data members `short corder`, `short xorder`, `uchar type`, `uchar pad` and `ushort len`, which will record metadata for an `Obs`, plus one member function `data` is defined which returns a `doub&` reference representing the entire struct.
- `struct OpHdr`, with data members `short order`, `uchar type`, `uchar prim`, `ushort pad` and `ushort len`, which will record metadata for an `Op`, plus the same `data` member function.
- `struct GenRec`, with data members `short rep`, `short order`, `uchar type`, `uchar T_odd` and `ushort len`, which will record metadata for a `Gen`, plus the same `data` member function.
- `struct GradRec`, with data members `ushort gen`, `ushort Hterm`, `ushort pad` and `ushort len`, which will record metadata for a gradient polynomial, plus the same `data` member function.

- `struct CurvRec`, with data members `ushort gen2`, `ushort gen1`, `ushort Hterm` and `ushort len`, which will record metadata for a curvature polynomial, plus the same `data` member function.
- `struct LagrRec`, with data members `ushort gen2`, `ushort gen1`, `ushort pad` and `ushort len`, which will record metadata for a Lagrange bracket polynomial, plus the same `data` member function.
- `struct GeoRec`, with data members `uint indx`, `ushort gen` and `ushort len`, which will record metadata for a geodesic equation polynomial, plus the same `data` member function.
- `struct RecHdr`, with data members `ushort info[3]` and `ushort len`, plus the same `data` member function. Conversion operators to any of the above specialized `*Hdr` types, and constructors which accept any of the above `*Hdr` objects are also defined. A `struct RecHdr` effectively functions as a union of the above specific record header types, but is not defined as such because unions cannot serve as base classes in C++.
- `enum class RecordID`, whose enumerated values, `Null`, `Obs`, `Op`, `Gen`, `Grad`, `Curv`, `Lagr`, `Geos`, `Vev`, `Coup` and `Stat` serve to identify the various specific data record types (with `Null` identifying an uninitialized `RecIndx`);
- `class RecIndx`, with data members:
 - `RecordID id`, giving the `RecIndx` type;
 - `uchar nslice`, recording the number of slices for `PolyCube` data (only used by `Curv` data records);
 - `ushort ncol`, recording the number of columns for multi-dimensional polynomial script data records (used by `Grad`, `Curv`, `Lagr` and `Geos` data records);
 - an unnamed union making `uint nrow` and `uint items` equivalent synonyms, used to record either the total number of items for one-dimensional data records (`Obs`, `Op`, `Gen`, `Vev`, `Coup`) or the number of rows for the multi-dimensional polynomial script data records;
 - `ulong reclen`, recording the total length of the data record in units of `sizeof (RecHdr)` (which is the same as `sizeof (doub)`);
 - `ulong filepos`, holding the file offset of the data record when written to disk.

The static data array `idname[]` maps `RecordID` values to printable names, and one member function `items` returns the total number of items for multi-dimensional data records (equal to `nslice * ncol * nrow`).

- `class RecIndxArr<N>`, a templated class inheriting from `array<RecIndx,N>`. The member function `next` returns a reference to the first uninitialized element of the array;

- `class DataRec`, which inherits from `vector<RecHdr>` and adds the data member `reference_wrapper<RecIndx> indexref` which associates the `DataRec` with a `RecIndx` entry in the `RecIndxArr` `global.sysindex`.

One constructor is defined, taking a `RecordID` as argument. Member functions in class `DataRec` are:

- `entry`, which returns the reference bound to `indexref`;
- `recptr`, which returns the actual data starting address (i.e., `data()`) cast as a `char*` or `const char*` pointer;
- `writerec`, which takes an `fstream` argument, stores the data length and current file position reported by `fstream::tellp` in the `RecIndx` returned by `entry()`, and writes the record data to the supplied file stream;
- `readrec`, which takes an `fstream` argument, resizes the `DataRec` to the size of `entry().reclen`, seeks the input file to the position in `entry().filepos`, and reads the stored data from the stream into resized data vector.
- `clear`, which clears the `vector<RecHdr>`, sets `items` to zero, and sets `nrow` and `nslice` to one.
- `free`, which destroys the `vector<RecHdr>`, forcibly freeing its memory using the C++ swap-trick.

7.9 Gen.{h,cpp}

Header file `Gen.h` includes `Obs.h` and `Op.h`, and defines:

- `class Gen`, which inherits from `OpSum` and encodes a linear combination of coherence group operators defined in the associated `OpList`, which altogether define a symmetry-projected coherence group generator. The `Gen` class adds a number of data members:
 - `short order`, giving the grading order of the generator (which must be identical to the orders of all the individual terms);
 - `OpType type`, giving the type of the generator (which must also be identical to the types of all the individual terms);
 - `bool T_odd`, which indicates whether or not the (anti-Hermitian) generator is time-reversal odd, and hence does not serve as a generator of deformations within the large N classical configuration space;
 - `bool imag`, indicating if the generator (which is necessarily defined to be anti-Hermitian) has an imaginary coefficient.
 - `bool active`, indicating if the generator is active or suspended (default true).
 - `doub coeff`, an overall coefficient (allowing a common factor to be pulled out of the individual terms in the sum);

- **ObsPoly reduction**, which for **Eloop** generators stores the quadratic polynomial in loops which results when the electric field operator in each generator term is interpreted as acting on the generator itself, while for fermionic generators the reduction is the result of anticommuting the fermion operators at either end (if they reside at the same site). These generator reductions are used in evaluating the variation of the entropy (for Euclidean theories), and in the terms involving products of pure loops which can contribute to the commutator of an **Eloop** with an **EEloop** (in Hamiltonian theories).

The **Gen** class has constructors which take a single **Op** plus a **Proj** into which the **Op** is projected, or take an **OpSum** linear combination of **Op**'s plus a projector. Member functions are:

- **collect**, for collecting terms referring to the same **Op** within the **OpSum**;
- **settype**, which sets the type, order and imaginary status of a **Gen** based on information in a single **Op**;
- **normalize**, which normalizes a generator in a particular representation, as described in section 4.7.
- **inner_commute**, which evaluates the “inner commutator” defining the **ObsPoly** reduction of the generator;
- **allzero**, which tests for identical vanishing of the **Gen**;
- **valid**, **is_Loop**, **is_Eloop**, **is_Fermion** and **isgauge**, which are trivial convenience functions.

Static data members are:

- **inline bool autoToddgens**, default **true**, indicating whether **Eloop** generators are to be automatically created from each pure-loop generator, by evaluating the commutator of the loop with the gauge kinetic energy;
- **bool gennorm**, indicating whether generators should be automatically normalized (as described in 4.7),

while static class functions are:

- **project**, which projects an **Op**, or **OpSum** of **Op**'s, onto all defined irreducible representations, storing the resulting symmetry projected generator in the **gen** vector of the **Rep** instance (provided that result is both non-zero and linearly independent from already present generators);
- **isnew**, which tests for linear dependency;
- **addgen**, which symmetry projects an **OpSum** onto all defined representations and adds resulting new independent generators to the set of defined generators;
- **suspend_group**, which suspends all generators of a specified order;
- **activate_group**, which activates all generators of a specified order;

- `suspend_gen`, which suspends an individual generator of the currently active representation;
- `activate_gen`, which activates all, or a specified individual generator, of the currently active representation;
- `normalize`, which normalizes all defined generators in all representations;
- `geninit`, which initializes low-order generators.

A friend `ostream& operator<<` declaration is included for printing `Gen`'s.

File `Gen.cpp` includes header files `Gen.h`, `Global.h`, `Commute.h`, `Rep.h`, `Gripe.h` and `Blab.h`, and defines the above mentioned functions.

The function `Gen::inner_commute` loops over the `Op` terms of the generator and, for `Eloop` generators, calls `do_inner` to evaluate the “inner” commutator, while for `Fermion` generators it tests to see if the `Op`, with fermions removed, forms a closed loop. In both cases, as in `commute_poly`, a `PolyMap` is used to collect common terms.

The initialization function `Gen::geninit` defines one and two plaquette generators when `global.stage == Global::Gauge` plus, for any compactified dimension, straight line Polyakov loops and single plaquette-inserted Polyakov loops. When fermions are present and `global.stage == Global::Fermi`, this function defines single link fermionic generators. These initializations include the definition of all primary generators used for observable generation.

7.10 `Global.{h,cpp}`

Header file `Global.h` includes `Counter.h` `Coupling.h` `Data.h`, `Op.h` and `Version.h`, and defines:

- Preprocessor symbols `FOR_EACH`, `TASK_ARENA` and `FINALIZE`. If the preprocessor symbol `PARALLEL` is defined, then several `tbb` header files are included and `FOR_EACH` is defined as `tbb::parallel_for_each`, the function macro `TASK_ARENA` is defined to initialize a `tbb::task_arena` with a specified thread limit and execute provided code within this arena, and `FINALIZE` is defined to execute `tbb::finalize`. Otherwise, `FOR_EACH` is defined as `std::for_each`, `TASK_ARENA` simply expands to the provided code argument, and `FINALIZE` does nothing. These definitions are used within `Build.cpp` and `Numerics.cpp`.
- `struct StageInfo`, with data members holding minimization stage specific information:
 - `short maxgen`, the maximum order of defined `Gen`'s for the stage;
 - `short maxord`, the maximum strong coupling order of observables for the stage;
 - `uint MMAlimit`, specifying the number of initial observable expectation values, for the stage, to include in minimization numerics output;

- `ObsSet MMAList`, specifying a further set of observables whose expectation values will be included in minimization numerics output;
 - `OpList ops`, holding the list of defined operators for the stage;
 - `array<vector<Gen>,NREP> gens`, holding the list of defined generators, for the stage, in each symmetry representation;
 - `array<ushort,NREP> neven`, giving the number of time-reversal even generators, for the stage, defined in each symmetry representation;
 - `vector<AdjTerm> Hterms`, holding the definition of the Hamiltonian (or free energy) for the given stage, in the form of a sum of `AdjTerm` instances.
 - `vector<uint3> bckt`, holding the list of tuples defining buckets of observables for the stage;
 - `Counters count`, holding the various thread-safe statistics counters defined in `class Counters`;
- `static constexpr int MAXBCKT = 256`, defining the maximum number of buckets of observables. This value is used for compile-time dimensioning of the `geos` polynomial script arrays in `global.stagedata`. (Note that any change to this compile-time constant will invalidate any previously generated system information files.)
 - `struct SerialData`, whose instances will hold minimization stage specific data in a serialized form which may be moved directly to or from disk. Data members are:
 - `DataRec op`, which will temporarily hold a serialized version of the `OpList global.info().ops` during save or load operations;
 - `DataRec obs`, which will temporarily hold a serialized version of (the stage specific portion of) `ObsList::obs` during save or load operations;
 - `DataRec gen`, which during save or load operations will hold a serialized version of a `StageInfo::gens` array of generators;
 - `PolyRec grad`, which holds the polynomial scripts defining the gradient of the Hamiltonian (or free energy) for a given stage, given by commutator of each Hamiltonian term with each vacuum representation generator;
 - `PolyArr<NREP> curv`, which holds the polynomial scripts for the given minimization stage defining the curvature of the Hamiltonian (or free energy), given by the double commutators of each Hamiltonian term with all pairs of generators in each symmetry representation;
 - `PolyArr<NREP> lagr`, which holds the polynomial scripts defining the Lagrange bracket matrix, for each representation, given by commutators of all pairs of generators (of a given gauge or fermi type) in each symmetry representation;
 - `PolyArr<MAXBCKT> geos`, which holds the symbolic forms of the geodesic equations, for a given minimization stage, produced by commutators of each observable with every vacuum representation generator. The resulting polynomial

scripts for each bucket of observables are stored in a separate `PolyRec` data record;

- `DataRec stat`, recording the size and possible on-disk location for the statistics counters in `Coupling::list`.

The first `DataRec` is used to save or restore all defined operators (for a given minimization stage), and likewise the following records for all stored observables, defined generators, the polynomial scripts for gradient, curvature, Lagrange bracket and geodesic equations, observable expectation values, coupling constants, and statistics counters. For the curvature and Lagrange bracket polynomials scripts, a separate data record is used for each symmetry representation, and due to the potential size of geodesic equation polynomial script, separate `DataRec`'s are used for the geodesic equations of individual buckets of observables, as defined by the bucket lists stored in the corresponding `StageInfo` structure.

- type alias `SysIndex = RecIdxArr<2 * (5 + 2 * NREP + MAXBCKT)>`. The number of elements equals the number of `DataRec` or `PolyRec` items in `Global::stagedata`.
- class `Global`, which has a single instance `global` holding much of the core symbolic information needed for applying the coherent state minimization algorithm. Data members are:
 - `enum stage`, defined to have two values: `Gauge = 0` and `Fermi = 1`. This `enum` is effectively a boolean identifying whether the gauge or fermion stage of the variational algorithm is active;
 - `SysIndex sysindex`, which serves as an index to various data records possibly written to disk. Each entry is a `RecIdx` recording the type of data in an associated `DataRec`, the length and associated dimensions of that data, and a sys info file offset for the data record;
 - `SerialData stagedata[2]`, holding stage-specific data which is in a serialized form allowing direct writing to/from disk;
 - `StageInfo stageinfo[2]`, holding stage-specific information which need not be written to disk by a `save` operation, or is not in a serialized form allowing direct writing to disk;
 - `short repnum`, which identifies the currently selected representation for curvature and spectrum calculations (default 0);
 - `short approx`, which if non-zero selects the approximation scheme by which observables whose strong-coupling order exceeds the `global.maxord()` limit should attempt to be approximated via a product of lower order observables (default 0);
 - `uint maxthread` which, if non-zero, sets a limit on the maximum number of threads spawned for parallel execution (default 0). Suitably adjusting this value depending on the number of available processor cores and the maximum memory available can improve performance when generating very large observable sets;

- `bool autosave`, indicating whether a sys info file should be automatically written during geodesic script construction (default `false`);
- `bool geoswap`, specifying whether buckets of geodesic equation polynomials are to be individually swapped to and from disk as needed (default `false`);
- `bool massreinit`, specifying whether Euclidean fermion bilinear expectation values are automatically reinitialized when the fermion mass is changed (default `true`);
- `bool symcurv`, specifying whether to use the symmetrized curvature when computing the small oscillation spectrum (default `true`);
- `bool okfermivev`, signalling whether fermion expectation values have been initialized and are consistent with current gauge vevs;
- `bool oknegeig`, specifying whether the presence of negative curvature eigenvalues (for sectors other than Euclidean fermions) will cause minimization to abort (default `false`);
- `bool vevappend`, specifying if a pre-existing vev data file is opened in append mode (default `false`);
- `bool MMAappend`, specifying if a pre-existing results output file is opened in append mode (default `true`);
- `atombol interrupt`, recording whether an interrupt signal has been received;
- `string MMAdir`, specifying the directory for numerics output data files (default `./MMA/THY` with `THY` the value of the Makefile variable `THY`);
- `string savedir`, specifying the directory sys info and vev data files (default `./save/THY`, with `THY` the value of the Makefile variable `THY`);
- `string sysfile`, specifying the sys info file name (default is auto-constructed by `dfltfilename`);
- `string vevfile`, specifying the vev data file name (default is auto-constructed by `dfltfilename`);
- `string MMAfile`, specifying the results output file name (default is auto-constructed by `dfltfilename`);
- `ofstream sysstream`, holding the output stream for an open sys info file;
- `ofstream vevstream`, holding the output stream for an open vev data file;
- `ofstream MMAstream`, holding the output stream for an open numerics results file;
- `Version version`, holding the running progrma’s version number.

Member functions of class `Global` defined for convenience are:

- `info`, which return a reference to the `stageinfo` structure for either the currently active stage, or a specified stage;

- `data`, which return a reference to the `stagedata` structure for either the currently active stage, or a specified stage;
- `maxgen`, which returns a reference to `info().maxgen`, holding the maximal `Gen` order for the current stage;
- `maxord`, which returns a reference to `info().maxord`, holding the maximal strong-coupling order for retained observables in the current stage;
- `count`, which returns a reference to `info().count`, holding the statistics counters for the current stage;
- `nobs`, which returns the number of retained observables for the current stage;
- `fg`, which returns either an 'f' or 'g' character based on the current minimization stage (used in labeling of generators in printed output).
- `dfltfilename`, which creates default data file names used for numerical output data files or save files, if not overridden by the user. The default file name is constructed from the short-form theory name, the maximum order of generators, the maximum strong-coupling order of the generated observables for the current stage, and the observable approximation setting, followed by the supplied extension (which is `.m` for numerical results files, `.sys` for sys info files, or `.vev` for vev data files).
- `bckt_pos`, which takes an observable number as argument and returns a tuple with the stage, bucket number, and position within the bucket at which the the specified observable is located in the bucket list.
- `mk_bcktlist`, which prepares a `vector<uint3>` bucket list, for a given minimization stage, with each element specifying bucket number, and starting and ending observable index values for the bucket. If the number of observables is less than `1024 * MAXBCKTS` then the number of observables per bucket is set to 1024. Otherwise, the total number of observables are evenly divided among `MAXBCKTS` buckets. (The last bucket need not be full.) This list is used in the parallelizing `for_each` loops in the various `Build::mk_*` routines as well as `Build::do_geostats` and `Numerics::do_dvev`, plus the `Save` routines which read and write geodesic polynomial scripts..
- `clearpolys` which clears all gradient, curvature, Lagrange bracket and geodesic equation polynomials for a specified minimization stage.
- `stageinit`, which sets `global.stage` to the indicated argument.
- `sysfilename`, returning either a user-defined or default sys info file name;
- `vevfilename`, returning either a user-defined or default vev data file name;
- `MMAfilename`, returning either a user-defined or default MMA results file name.

If preprocessor symbol `PARALLEL` is defined, then the class also includes a static data member `tbb::task_scheduler_handle handle`, which is used by the `FINALIZE` macro invoked in `Parse::quit` to wait termination of worker threads upon receipt of an interrupt signal.

- constructor `DataRec::Datarec (RecIndx Id)`, which is defined out-of-line in this header file because the constructor must bind the `indexref` member of any `Datarec` to an entry in `global.sysindex`;
- the templated constructor `PolyArr<N>::PolyArr (RecIndx Id)`, which is defined out-of-line in `Global.h` because the constructor must bind the `indexref` member of each `PolyRec` array element to an entry in `global.sysindex`.

File `Global.cpp` includes `Global.h`, and defines the member functions `dfltfilename`, `bckt_pos`, `mk_bcktlist` and `clearpolys` described above.

7.11 Gordion.{h,cpp}

Header file `Gordion.h` contains `using` declarations for various standard container classes, I/O classes and functions, and basic types used through the code. These are:

- `std::pair`,
- `std::array`,
- `std::string`,
- `std::vector`,
- `std::complex`,
- `std::set`,
- `std::unordered_set`,
- `std::unordered_map`,
- `std::exception`,
- `std::cout`,
- `std::flush`,
- `std::fstream`,
- `std::istream`,
- `std::ostream`,
- `std::ifstream`,
- `std::ofstream`,
- `std::istringstream`,
- `std::ostringstream`,

along with:

- `real` which is set to `float` or `double` if preprocessor symbol `VEV32` is or is not defined, respectively,
- `cmplx = complex<double>`,
- `doub = double`,
- `symb = signed char`,
- `uchar = uint8_t`,
- `ushort = uint16_t`,

- `uint = uint32_t,`
- `ulong = uint64_t,`
- `uint2 = std::pair<uint,uint>,`
- `uint3 = std::array<uint,3>,`
- `char8 = std::array<char,8>,`
- `strview = std::string_view,`
- `hash = std::unordered_map<Key,Value...>.`

The final `hash` alias is a templated definition applicable to any `Key` and `Value` types. A further template definition makes `cast_to` a synonym for `reinterpret_cast`.

This header file also defines:

- `static string program`, which records the invoking program name,
- `static string cmdargs`, a usage message string,
- `static constexpr ulong ipow`, a compile-time integer power function.

Finally, as some older compilation environments lack the `std::format` function (despite having the rest of C++17 and C++20 functionality), this file tests for the definition of `__cpp_lib_format_uchar` and conditionally includes either a `using` definition for `std::format`, or an stand-alone templated replacement for `std::format`.

File `Gordion.cpp` includes `Init.h`, `Parse.h` and `Global.h`, and contains the routines:

- `process_flags`, which handles command line flags. A `-f` flag enables one to override or change an initial start-up file of commands which, by default, is `./gordion`.
- `main`, which calls `process_flags` followed by `initialize` for global variable initialization, `signal` to install `Parse::sig_catch` as a `SIGINT` signal handler, optionally calls `Save::load_save` to load command line arguments naming sys-info or vev-data files, optionally calls `read_input` to read commands from a start-up file, and then calls `read_input` to read from `std::cin`.

This file also defines two static variables:

- `dfltstart`, which records the program start time,
- `infile`, which holds the current input file name.

7.12 Gripe.h

Header file `Gripe.h` defines:

- `class BadInput`, which inherits from `std::runtime_error` and whose instances signal invalid user input. `BadInput` exceptions are caught and reported at the highest level of the command interpreter, aborting further processing of any open command input files;

- `class Abort`, which inherits from `std::runtime_error` and whose instances signal failure of convergence in Newton minimization or ODE integration. `Abort` exceptions are reported, but do not terminate further processing from any open command input files;
- `gripe`, a shorthand inline function which throws a `BadInput` exception with specified error message;
- `abort`, a shorthand inline function which throws an `Abort` exception with specified error message;
- `oops`, a shorthand inline function which throws a `std::logic_error` exception with specified error message. Such exceptions indicate programming errors.

7.13 `Index.{h,cpp}`

Header file `Index.h` includes `Gordion.h` and defines:

- `class Index<T>`, which inherits from `vector<T>` and adds one data member:
 - `hash<string,uint> map`, which serves to map `T` objects, if present in the `vector<T>`, to their list index;

plus member functions:

- `Index<T>::store`, which takes either a `T& item` argument, for types `T` which themselves inherit from `std::string`, or a `string key` plus `T& item` pair, and adds the item, if not already present, to the list and associated (key, list index) pair to the map, returning the list index of the item;

Instances of this template class are used to store indexed lists of `Op`'s, `Symm`'s, `Rep`'s and `SymmSet`'s.

7.14 `Init.{h,cpp}`

Header file `Init.h` merely declares the initialization function `initialize`.

File `Init.cpp` includes the header files `Canon.h`, `Gen.h`, `Numerics.h`, `Rep.h` and `Blab.h`, defines the two global data structures:

- `Global global`,
- `Numerics numerics`,

and brings together the definitions of static data members of certain classes, in particular:

- `ObsList::obs`, the list of canonicalized observables,
- `ObsList::base`, the list of basic (non-canonicalized) observables,

- `ObsList::redu`, the list of observables appearing in generator reductions,
- `Proj::list`, the list of symmetry projectors,
- `Symm::list`, the indexed list of symmetry transformations,
- `Rep::list`, the indexed list of irreducible representation projector rows,
- `Coupling::list`, the list of adjustable couplings,
- `Symm::trans2indx`, the symmetry transformation to symmetry index map.

Finally, this file defines one function `initialize` which calls various class-specific initialization routines, namely:

- `Global::resetlab`, which sets initial verbosity levels;
- `Theory::theoryinit`, which loads the theory name in the default directory names for output files and prints the selected theory name and program version;
- `Symm::symmininit`, which initializes lattice symmetry data;
- `Rep::repinit`, which initializes irreducible representations;
- `Canon::looptblinit`, which initializes the loop canonicalization table;
- `Canon::spectblinit`, which initializes the non-loop canonicalization table;
- `ObsList::base.obsinit`, which initializes the basic observable list;
- `Theory::theorydefn`, which defines the Hamiltonian or free energy of the theory (selected by the value of the Makefile defined variable `THY`);
- `Gen::geninit`, which defines initial sets of generators.

The initial values of `global.stage` and `numerics.rk` are also assigned. The ordering of some of these initializations is critical, which is why the these static initializations are bought together in this one function.

7.15 Linalg.h

Header file `Linalg.h` tests for the preprocessor symbols `ARMA` or `EIGEN` and defines a number of type aliases and inline wrapper functions which allow either the `Armadillo` or `Eigen` linear algebra packages to be used. For the normal case of `Armadillo`, the definitions are:

- using declarations for `arma::abs`, `arma::eig_gen`, `arma::eig_pair`, `arma::pinv`, `arma::rank`, `arma::index_max`, `arma::sort`, `arma::sqrt` and `arma::svd`;
- type alias `Uvec = arma::Col<uword>`, for submatrix view index vectors;
- type alias `Rvec = arma::Col<real>`, for numerical observable expectation values and derivatives of those expectation values;
- type alias `Dvec = arma::Col<doub>`, for gradient and other numerical vectors;
- type alias `Cvec = arma::Col<cmplx>`, for spectrum eigenvalues;

- type alias `Dmtx = arma::Mat<doub>`, for numerical values of the curvature, metric and Lagrange bracket matrices;
- type alias `Cmtx = arma::Mat<cmplx>`, for spectrum eigenvectors;
- `ncol`, returning the number of columns of a matrix;
- `nrow`, returning the number of rows of a vector, matrix;
- `nelem`, returning the total number of elements of a vector or matrix;
- `memptr`, returning a pointer to data of a vector or matrix;
- `set_zero`, setting to zero all elements of a vector or matrix, possibly resized;
- `resize`, resizing a vector;
- `raw_print`, printing a vector or matrix to `cout`;
- `linsolve`, calling `arma::solve` to solve a linear system;
- `infnorm`, calling `arma::norm` to compute the L^∞ norm of a supplied vector;
- `l2norm`, calling `arma::norm` to compute the L^2 norm of a supplied vector;;
- `realpart`, calling `arma::real` to return the real part of a supplied complex vector;
- `imagpart`, calling `arma::imag` to return the imaginary part of a supplied vector;
- `has_nam`, calling `arma::has_nam` on a supplied vector;
- `inv`, returning the inverse of a matrix;
- `transpose`, returning the transpose of a matrix;
- `aliasvec`, returning an `Rvec` bound to pre-allocated memory;
- `sort_index`, returning a `Uvec` holding the indices of sorted elements of a vector.

See the file for the alternative `Eigen` type definitions and additional wrapper functions. The capability to use `Eigen` was added for testing purposes; the `Armadillo` library is generally preferred.

7.16 Numerics.{h,cpp}

Header file `Numerics.h` includes `Global.h`, `Poly.h` and `Ode.h`, and defines:

- `struct Status`, whose instances have data members:
 - `bool warn`, indicating if there are warnings to report;
 - `array<doub,3> negeig`, holding (up to 3) negative curvature eigenvalues;
 - `array<cmplx,3> cmplx eig`, holding (up to 3) complex curvature eigenvalues;
 - `doub maxloopv`, holding the largest `Loop` expectation value;
 - `uint maxloopi`, holding the index of the largest `Loop` expectation value;
- `class Numerics`, which has a single instance `numerics` holding the information needed for numerical minimization. Data members of the class are:
 - `doub H`, holding the expectation value of the Hamiltonian (or free energy);

- `Dvec gradient`, holding the gradient of the Hamiltonian (or free energy);
- `Dmtx curvature`, holding the T -even curvature of the Hamiltonian (or free energy) in the currently selected representation;
- `Dmtx metric`, holding the T -odd curvature of the Hamiltonian (or free energy) in the currently selected representation;
- `Dmtx lagrange`, holding the Lagrange bracket matrix for the current irrep;
- `Dvec delta`, holding the predicted location of the Hamiltonian (or free energy) minimum;
- `Rvec vev`, for numerical expectation values of `ObsList::obs` observables;
- `Rvec dvec`, for derivatives of these expectation values in the direction `delta`;
- `Cvec spectrum`, holding the small oscillation frequencies (from the latest spectrum evaluation);
- `Cmtx modes`, holding the small oscillation eigenvectors (from the latest spectrum evaluation);
- `Uvec Tevens`, holding the indices of active T -even generators in the currently selected representation;
- `Uvec Todds`, holding the indices of active T -odd generators in the currently selected representation;
- `short lastrep`, recording the representation index of the latest spectrum evaluation;
- `Rvec vev_tmp`, a temporary vector used to combine updated gauge or fermion expectation values during geodesic equation integrations;
- `const real* vev_buf`, pointing to an expectation value buffer;
- `real* dvec_buf`, holding a pointer to an expectation value derivative buffer;
- `doub svdcut`, defining the cutoff threshold for singular value omission in the curvature pseudo-inverse;
- `doub dflttol`, defining a default value for the ODE and minimization tolerance;
- `doub mintol`, defining the Newton minimization error tolerance;
- `doub odetol`, defining the Runge-Kutta ODE error tolerance;
- `uint odemax`, defining the maximum number of ODE integration steps;
- `uint minmax`, defining the maximum number of Newton minimization steps;
- `RKdef rk`, holding the selected Runge-Kutta method;
- `struct { uint tries, uint fails } stats`, holding minimization stats.

Member functions of class `Numerics` are:

- `do_minimize`, which runs a Newton minimization loop, calling `do_step` to perform each iteration and terminating when the norm of `numerics.delta`, the predicted displacement from the minimum, falls below `numerics.mintol` (or when the number of minimization steps exceeds `numerics.minmax`). After the minimization, `report_warns` is called to report any warnings.

- `do_step`, which performs one Newton iteration step, first calling `eval_delta` to find the predicted minimum location and then calling `ode_int` to integrate the geodesic equations for all observables along the geodesic to the predicted minimum.
- `do_flow`, which takes as arguments a coupling index, and an initial value, final value, and increment for the specified coupling. The indicated coupling's value is progressively changed from the given initial value to the final value in steps of the specified increment, with `do_minimize` called on each step, followed by `Numerics::save_data` to record the minimization results in the numerics output data file.
- `eval_ham`, which numerically evaluates the Hamiltonian or free energy by evaluating the canonicalized `ObsPoly` defined in each `AdjCoupling` term in `global.info().Hterms`, multiplying by the indicated power of an adjustable coupling constant, and summing the resulting contributions. A boolean argument to the function controls whether the result is printed to `cout`, or merely stored in `numerics.H` for later reporting.
- `eval_geos`, which calls `eval_delta` to predict the minimum location δ then calls `Numerics::do_ddev` to evaluate numerically the symbolic geodesic equation, in the direction δ , for each observable in `ObsList::obs` of the relevant type (gauge or fermi), depending on `global.stage`, with results stored in `numerics.ddev`. Finally, the routine optionally prints a specified subset of observable derivatives.
- `eval_delta`, which calls `eval_grad` and `eval_curv` to evaluate the gradient dH and T -even curvature ddH_{++} of the Hamiltonian (or free energy), and then solves the linear equation

$$ddH_{++} \cdot \delta = -dH, \quad (12)$$

predicting the location δ of the minimum of the Hamiltonian (or free energy). If `numerics.svdcut` is zero then `arma::solve` is used as the equation solver, otherwise `arma::pinv` is called to compute a pseudo-inverse in which contributions from singular values less than `numerics.svdcut` in the singular value decomposition of the curvature matrix are omitted.

- `eval_grad` which, zeros `numerics.grad` and then loops over each `AdjCoupling` term in `global.info().Hterms`, then over every T -even generator in the vacuum representation, evaluating numerically the corresponding `ObsPoly` in the `global.info().grad` entry which corresponds to the given T -even generator. Each such numerically evaluated `ObsPoly` is multiplied by the indicated power of an adjustable coupling constant and added to the corresponding element of `numerics.grad`. A boolean argument to the function controls whether the result is printed to `cout`, or merely stored in `numerics.grad` for later use.
- `eval_curv` which zeros `numerics.curv` and then loops over each `AdjCoupling` term in `global.info().Hterms`, then over pairs of T -even generators in the current representation (and stage), evaluating the associated `ObsPoly` element

of `global.info().curv[global.repnum]`, multiplying by the indicated power of an adjustable coupling constant, and adding the result to the corresponding element of `numerics.curv`. The resulting maxrix is stored in `numerics.curv`. A second integer argument to the routine controls whether nothing, the curvature eigenvalues, or the curvature matrix plus eigenvalues are printed.

- `eval_metr` which zeros `numerics.metr` and then loops over each `AdjCoupling` term in `global.info().Hterms`, then over pairs of T -odd generators in the current representation (and stage), evaluating the associated `ObsPoly` element of `global.info().curv[global.repnum]`, multiplying by the indicated power of an adjustable coupling constant, and adding the result to the corresponding element of `numerics.metr`. A second integer argument to the routine controls whether nothing, the metric matrix eigenvalues, or the metric matrix plus eigenvalues are printed.
- `eval_lagr`, which zeros `numerics.lagrange` and then loops over pairs of T -even/ T -odd generators in the current representation (and stage), evaluating the associated `ObsPoly` element of `global.info().lagr[repnum]` and adding the result to the corresponding element of `numerics.lagrange`. A boolean argument to the function controls whether the result is printed to `cout`, or merely stored in `numerics.lagrange` for use by `eval_spectra`.
- `eval_inuse`, which returns the index list of active generators for a specified representation.
- `eval_spectra`, which calls `eval_curv`, `eval_metr` and `eval_lagr` to evaluate the curvature matrix ddH_{++} , metric matrix ddH_{--} and Lagrange, bracket matrix L_{+-} for the specified representation, and then calls `arma::eig_pair` to solve the generalized eigensystem:

$$ddH_{++} \cdot v = \omega^2 L_{+-} \cdot (ddH_{--})^{-1} \cdot (L_{+-})^T \cdot v. \quad (13)$$

The resulting small oscillation frequencies ω (times $\hbar$) give the excitation energies to translationally invariant excited states, or in other words, the mass spectrum for low-lying particles (either glueballs or mesons, depending on the minimization stage). If `global.symcurv` is true, then the symmetrized curvature matrix is used in the computation (default). Eigenvalues are sorted into ascending order. Overloaded forms allow either a representation name or number as argument. A second boolean argument signals if the resulting eigenvalues are to be printed.

- `initialize`, which initializes the numerics data structures for the indicated minimization stage. For the gauge stage, all elements of `numerics.vcv` are zeroed, (with the first element corresponding to the identity loop then reset to unity). For the fermion stage the last `ObsList::obs.nobsF` elements of `numerics.vcv` are zeroed after which entries in `ObsList::obs.fermiinit` are used to set the numerical value of each indicated closed `Fermion` bilinear equal to the value of the corresponding `Loop` times the uncorrelated fermion condensate ($-1/2$ for Hamiltonian theories, or minus the inverse fermion mass for Euclidean theories).

- `write_data`, which takes the value of a flowing coupling constant as argument and calls `data_write` to write current values of the Hamiltonian or free energy, a subset (defined by `global.info().MMALimit`) of observable expectation values of the relevant type, and, for Hamiltonian theories, the small oscillation spectra for all representations (for which the curvature and Lagrange bracket have been symbolically evaluated). Before writing data, if the output data stream is not already open, the output data file is opened in either truncation or append mode, depending on the value of `dataappend`.
- `status_rpt` which, for the gauge stage first calls `check_loops`, then calls `curv_rpt` to report any problematic curvature eigenvalues, and then reports if any `Loop` expectations exceed (in magnitude) 1.0. If there are eigenvalues more negative than `-numerics.svdcut` (except for the fermion stage of Euclidean theories where extremization, not minimization, is relevant) and `global.oknegeig` is not set, an exception is thrown which terminates the current command.
- `check_loops`, which is called by `status_rpt` (for gauge stage minimization) and stores the value and index of the `Loop` observable with maximal expectation value in `numerics.warnings`, setting `numerics.warnings.warn` to true if the maximum exceeds 1.0.
- `termvalue`, which returns the numerical value of a `PolyTerm` using the list of expectation values stored in the `vev` data member.

Static class members declared in class `Numerics` are:

- `Status status`, holding warning related information.
- `termvalue`, a static overload which takes a `PolyTerm` plus a list of numerical expectation values as arguments, and returns the numerical value of a `PolyTerm` using the provided list. This version of `termvalue` is used by `do_dvev_bkt`.
- `check_curv`, which is called by `eval_delta` and `eval_spectra`, and examines the eigenvalues of the curvature matrix, storing in `numerics.warnings` (for later reporting) those which are the most negative (if any) or having the largest imaginary parts (if any).
- `curv_rpt` which, if `numerics.status.warn` is set, reports negative or complex curvature eigenvalues.
- `built_rep`, which checks if the symbolic forms of the curvature and Lagrange bracket are defined for a given representation.
- `MMAform`, which converts the standard output scientific notation for a `double` to Mathematica's input form by first writing the number to a `stringstream` and then converting an `e` exponent marker, if present, to `^*`. So, for example, `1.23e-04` becomes `1.23^*-04`.
- `do_dvev`, which numerically evaluates the symbolic geodesic equations stored in `global.geodesics`, for all observables of the current stage (i.e., gauge or

fermion), for the geodesic direction defined by `numerics.delta`, using a provided list of observable expectation values. Results are stored in a `Numvec` whose reference is passed as an argument to `eval_dvev`. This function is called by `ode_int` for each sub-step comprising part of a single Runge-Kutta integration step. The full evaluation is broken up into separate buckets of observables, with a parallelizable `for_each` loop calling `do_dvev_bkt` for each bucket.

- `do_dvev_bkt`, which does the actual numerical evaluation of the geodesic equations for a bucket of observables. The list of observable expectation values to be used and the destination for results are passed to `do_dvev_bkt` via the pointers `numerics.vew_buf` and `numerics.dvev_buf`, which are set by `do_dvev`.
- `err_norm`, which evaluates the L_2 norm of a provided ODE error vector. This function is called by `ode_int` for each Runge-Kutta integration step.
- `data_write`, which takes arguments of an output stream, a coupling value v , a variable name and corresponding variable value which (for different overloads) may be either a single `doub`, a `Numvec` real-valued vector, or a `Complexv` complex-valued vector. In each case, a line having the form of

$$name[v] = value;$$

or

$$name[v] = \{value_1, value_2, \dots\};$$

is written to the specified output stream. By design, this output is formatted in a manner which may be directly input to Mathematica.

File `Numerics.cpp` includes `Numerics.h`, `Data.h`, `Gen.h`, `Global.h`, `Rep.h`, `Print.h`, `Save.h`, `Blab.h` and `Gripe.h`, and defines the above functions of class `Numerics`.

7.17 Obs.{h,cpp}

Header file `Obs.h` includes `Symb.h`, `Poly.h` and `Assess.h`, and defines:

- `enum class ObsType`, with members `Loop`, `Eloop`, `EEloop`, `Fermion`, `Efermion` and `Entropy`, for identifying the different types of observables;
- `class Obs`, which inherits from `SymbStr`. Instances of this class store the symbol string and related information representing an observable. Additional data members are:
 - `ObsType type`, giving the observable type;
 - `short corder`, recording the observable's creation order;
 - `short xorder`, recording the observable's expectation order;
 - `short midE`, which records the location of a electric field insertion, if any, in a position other than the start of the `SymbStr`.

Multiple constructors are defined, allowing an `Obs` to be constructed from a printable string (whose characters are converted to their corresponding `ymb` codes, with `ymb` ligatures then appropriately joined), from a `SymbStr` or some subrange of a `SymbStr`, all with creation and expectation orders also specified, or from an `Op`.

Member functions in class `Obs` include:

- `findstart`, which rotates a gauge observable to a unique preferred starting position when full canonicalization is not needed;
- `middleE`, which finds and/or returns the location of a mid-observable electric field insertion;
- `validate`, which examines the data in an `Obs` instance for consistency (used for testing);
- `trans`, which applies (in place) a symmetry transformation to the observable;
- `order`, trivial convenience function returning the strong-coupling order (equal to the sum of creation and expectation orders);
- `known_xord`, trivial convenience function signaling a valid `xorder`;
- `bilinear`, convenience function signaling a `Fermion` or `Efermion` observable;
- `EorEEloop`, convenience function signaling an `Eloop` or `EEloop` observable;
- `midEtype`, convenience function signaling an `EEloop` or `Efermion` which may contain a `midE` insertion;
- `staggered`, convenience function signaling a staggered fermion bilinear (for Hamiltonian theories);
- `is_gauge` and `is_fermi`, convenience functions signaling a pure gauge observable (including gauge entropy) or fermionic observable (including fermion contributions entropy and gauge kinetic energy), respectively;
- `is_Loop`, `is_Eloop`, `is_EEloop`, `is_Fermion`, `is_Efermion` and `is_Entropy`, each signaling the indicated observable type;
- `is_Fermion0`, indicating if a fermion bilinear is odd length;
- `imag`, indicating in Euclidean theories if a fermion bilinear has an imaginary expectation value;
- `has_Es`, signaling an observable which has electric field insertions;
- `is_coord`, signaling observables whose time reversal equals their Hermitian conjugate, and hence whose expectation values serve as coordinates on the classical configuration space;
- `oddlen`, indicating fermion bilinears, possible with an electric field insertion, with an odd number of links.

Further class `Obs` member functions are defined in `Canon.cpp` and `Assess.cpp`:

- `canon`, for canonicalization of the observable;
- `approximate`, which attempts to approximate an observable’s expectation by a product of lower order observables;

- `u1bound`, which constructs a lower bound on expectation order;
- `noEbound`, which constructs a lower bound on expectation order;
- `classify`, which determines an observable's expectation order;
- `Ereduce`, which commutes an observable containing E 's with primary Op 's to determine expectation order;
- `factorize`, which factors an observable not containing E 's at self-intersections to determine expectation order;
- `sitelist`, which constructs a list of coordinates of all sites traversed by an observable.

An inline static boolean `Obs::check` controls whether `validate` is called on every created `Obs`, and the static `constexpr` array `Obs::type_name` stores printable names of the different `ObsType` values.

The static function `Obs::obstype`, called from `Obs::Obs(string&)` constructor, examines a string and determines which type of `Obs`, if any, could have this printable representation;

The static `std::map<string,ObsType> obstypes` serves to map observable types as printable strings into the corresponding `ObsType` enum value. (This is used in `Print::print_obs_select`.)

A friend `ostream& operator<<` declaration is included for printing `Obs`'s.

- `struct Obshash`, which defines member function `operator()(const SymbStr&)` as a hash function for `Obs`'s;
- `struct Obs_eq`, which defines the member function `operator()(const SymbStr&, const SymbStr&)` as an equality testing function for `Obs`'s;
- type alias `Obsset = std::unordered_set<Obs,Obshash,Obs_eq>;`
- type alias `Obsmap = hash<const Obs,uint,Obshash,Obs_eq>;`
- `class ObsList`, which inherits from `vector<const Obs*>`, and whose instances are used to store lists of observables. Additional data members are:
 - `Obsmap map`, a private data member holding a hash table of `Obs`'s in the list.
 - `string name`, giving a name to the observable list;
 - `bool canonicalize`, indicating whether observables are to be canonicalized prior to storing in the `ObsList`;
 - `bool classify`, indicating whether observables are to be classified by expectation order prior to storing in the `ObsList`;
 - `bool approx`, indicating whether classified observables whose strong-coupling order exceeds the limit set by `global.maxord()` should attempt to be approximated via a product of lower order observables;
 - `uint nobsg`, giving the count of contained gauge observables;

- `uint nobsf`, giving the count of contained fermion observables;

Member functions include:

- `operator()`, which takes a `uint` list index as argument and returns a reference to the corresponding `Obs` in the list;
- `find`, which finds an observable with the specified `SymbStr` if present in the list, returning the observable's index or `UINT_MAX` for failure. If the `ObsList` instance is `ObsList::obs` and `ObsList::inbox` is non-empty, then the inbox is also searched with a return of `UINT_MAX-1` signaling the observable is in the inbox, but not yet inserted into the master `ObsList`;
- `purge`, which removes entries from the `ObsList` whose indices are above a specified value;
- `empty`, which removes all entries (except the identity observable) from the list;
- `insert`, which inserts the elements of an `Obsset` into the `ObsList`;
- `frozen`, which tests if the `ObsList` instance is `ObsList::obs` and if so returns the thread-specific value of `ObsList::freeze`;
- `freezeif`, which tests if the `ObsList` instance is `ObsList::obs` and if so sets `ObsList::freeze` to true, returning its previous value;
- `refreezeif`, which tests if the `ObsList` instance is `ObsList::obs` and if so sets `ObsList::freeze` to the supplied value;
- `nobs`, which returns the number of stored observables for the specified minimization stage;
- `neq`, which tests for inequality with a provided `ObsList` reference argument, indicating a need to re-index observables from a different list;
- `do_fermiinit`, which populates `ObsList::fermiinit`. Each element of this vector is a pair of unsigned integers which serve to map the `ObsList::obs` index of a `Fermion` observable having coincident endpoints to the index of the corresponding `Loop` observable.
- `obsinit`, which initializes a list with the basic observables needed for the indicated stage, i.e., those appearing in the Hamiltonian or free energy. For the `Gauge` stage, these are single plaquettes, Polyakov loops (if dimensions are compactified), and squares of electric field operators (for Hamiltonian theories), in all possible lattice orientations, along with entropy (for Euclidean theories). For the `Fermi` stage, these are fermion mass and single link hopping terms, along with the fermion contribution to entropy (for Euclidean theories) or gauge kinetic energy (for Hamiltonian theories);
- `print` for printing to a given output stream either the entire list, or a single indexed observable within the list.

Further member functions taking one or two `Obs` as arguments are:

- **store**, which adds its **Obs** argument in the list, if not already present, and returns the index of the stored **Obs**;
- **catalog**, which first canonicalizes or rotates the supplied **Obs** argument(s) as appropriate, adds the resulting observable(s) to the list (if not already present) and returns a **PolyTerm** which records the list indices of the observable(s). Throws an exception if the list is frozen but the supplied observable(s) are not present in the list;
- **is_known**, which canonicalizes or rotates its r-value argument(s) as appropriate, then tests for the existence of the resulting observable(s) in the list, and returns a **PolyTerm** which records the list indices if present;
- **assess**, which is defined in **Assess.cpp** and encodes the logic determining whether an observable generated by the commutation routines is to be stored, approximated via factorization, or discarded.

class ObsList also defines static data members:

- **ObsList ObsList::obs**, which is the master list of canonicalized observables;
- **ObsList ObsList::base**, which holds non-canonicalized observables appearing in the definition of the Hamiltonian or free energy;
- **ObsList ObsList::redu**, which holds non-canonicalized observables appearing in the “inner commutation” reduction of **Eloop** and **Fermion** generators;
- **inline vector<uint2> ObsList::fermiinit**, containing index pairs which map list indices of those fermion bilinears whose fermion and conjugate fermion fields reside on the same site to the corresponding pure gauge loop (needed for initializing fermion bilinear expectation values).
- **thread_local Obsset inbox**, which serves to hold observables found by the **do_*_bckt** observable generation routines prior to their insertion into the master **ObsList::obs** observable list at the end of the observable generation pass;
- **thread_local bool freeze**, signaling when newly generated observables, in a specific thread, are to be retained (or not),

and the static function:

- **retain**, which adds a newly constructed and classified observable, of the current target order, to a thread’s **inbox**.
- **class ObsStats**, which inherits from **array<vector<vector<uint>>, ...>**, to hold an array of matrices which will collect statistics of observables classified according to their types, creation orders, and expectation orders. The class includes data members **maxc** and **maxx**, for recording the maximum creation and expectation orders, respectively, appearing in the statistics matrices, **avglen**, recording the average length of observables, and **maxloop**, recording the index of the **Loop** with largest expectation value. The inline member function **get**, with two overloads, returns the number of observables with given creation and expectation orders, and optionally

specified type. One constructor is defined, which takes an `ObsList` as argument and steps through the observables accumulating the type-specific statistics.

File `Obs.cpp` includes `Obs.h`, `Global.h`, `Numerics.h`, `Op.h`, `Symm.h`, `Stats.h`, `Gripe.h` and `Blab.h`, and defines above mentioned member functions, other than those indicated as residing in `Canon.cpp` or `Assess.cpp`, along with the `thread_local` variables `ObsList::inbox` and `ObsList::freeze`.

7.18 `Ode.{h,cpp}`

Header file `Ode.h` includes `Gordion.h`, `Linalg.h` and `RK.h`, and defines:

- type alias `OdeRhs = void (*)(doub, const Numvec&, Numvec&)`, for pointers to an ODE derivative function;
- type alias `OdeNorm = doub (*)(const Numvec&, const Numvec&, const Numvec&)`, for pointers to an ODE error norm function;
- class `Ode`, which inherits from class `RKdef` and adds data members:
 - `OdeRhs rhs`, binding a derivative function to the ODE integrator;
 - `OdeNorm norm`, binding a norm function to the integrator;
 - `double tolerance`, giving the specified error tolerance;
 - `double minstepsize`, giving a minimum step size (default 0);
 - `double maxstepsize`, giving a maximum step size;
 - `double initstepsize`, giving an initial step size;
 - `uint steps`, recording the number of steps taken by the `Ode` integrator instance;
 - `uint rejects`, recording the number of rejected steps attempted;
 - `uint maxstep`, giving an maximum number of ODE integration steps;
 - `static struct Stats stats`, holding cumulative ODE integration statistics.
 - `static constexpr int dfltmax = 1500`, the default value for `maxstep`.

The class defines a single member function:

- `integrate`, which takes as arguments initial and final values of the integration variable, and a `Numvec` of initial data, and attempts the specified integration.

File `Ode.cpp` includes `Ode.h` and defines the `Ode::integrate` member function. Static `Numvec` buffers are used for derivative evaluations, trial function values, and the error estimate vector. The adaptive step size scheme is essentially that of Ref. [20], generalized to RK schemes of variable order. Both FSAL (“First Same As Last”) and non-FSAL schemes are supported.

7.19 Op.{h,cpp}

Header file `Op.h` includes `Obs.h` and `Index.h`, and defines:

- type alias `OpTerm = Term<uint>`, used when representing a specific indexed member of `Op::list` times a numeric coefficient.
- class `OpSum`, which inherits from `vector<OpTerm>`, add a single data member, a `reference_wrapper` to the associated `OpList`, and declares the member functions:
 - `oplist`, returning a reference to the associated `OpList`,
 - `collect`, for collecting terms with a common list index;
 - `flipT`, which returns an `OpSum` containing entries in which all fermion `Op`'s have flipped staggering;
 - `loop_dt`, which returns an `OpSum` containing entries in which every `Loop Op` in the calling `OpSum` has been converted into a sum of `Eloop Op`'s each containing one electric field insertion on some link of the loop. Specifically, given the lattice representation of some Wilson loop $U(C)$ around a contour C , `loop_dt` returns the sum of `Op`'s corresponding to the commutator $[\frac{1}{2}\text{tr}(E^2), \text{tr } U(C)]$.

Static versions of `loop_dt` allow one to supply an individual `Op` or `OpTerm`.

An `friend ostream& operator<<` declaration for printing `Op`'s is also included.

- enum class `OpType`, with members `Loop`, `Eloop`, `Fermion` and `Invalid`, for identifying the different types of operators appearing in coherence group generators.
- class `Op`, which inherits from `SymbStr` and whose instances store the form of individual terms appearing in coherence group generators. The class adds data members:
 - `OpType type`, indicating the type of the `Op`,
 - `bool primary`, indicating whether the `Op` is primary,
 - `short order`, giving the grading order of the `Op`.

Member functions are:

- `findstart`, for cyclically rotating `Loop` and `Eloop` instances to their unique preferred starting position;
- `oddlen`, indicating if the `Op` has an odd length;
- `is_Loop`, `is_Eloop` and `is_Fermion`, trivial convenience functions each signaling the indicated operator type;
- `is_Fermion0`, indicating if an `Op` is an odd-length fermion bilinear;
- `staggered`, identifying staggered fermion `Op`'s (for Hamiltonian theories);
- `is_coord`, identifying those `Op`'s whose time reversal equals their Hermitian conjugate, and hence serve as coordinates on the classical configuration space.

- `validate`, a private function called in the constructors for testing purposes;

One conversion operator is defined, `Op(const Obs&)`, for converting an `Obs` (of suitable type) into an `Op`.

The class includes the following static data members and static functions:

- `inline bool check`, controlling whether `validate` is called on every created `Op`;
- `optype`, which is called from the `Op::Op(string&)` constructor and examines a string to determine which type of `Op`, if any, could have this printable representation;

An `friend ostream& operator<<` declaration for printing `Op`'s is also included.

- `class OpList`, inheriting from `Index<Op>` and whose instances hold indexed lists of operators. The class defines member functions:
 - `store`, which stores an `Op`, if new, in the list;
 - `setprimary`, which determines which of the `Op`'s in the list are primary — that is, are not produced from the commutator of any pair of lower order `Op`'s whose grading orders sum to the order of a given `Op`;
 - `purge`, which is called from `Op::setprimary` and `Test::jacobi` and removes all `Op`'s from the `OpList` beyond a specified limit;
 - `clear`, which removes all entries from the `OpList`;
 - `print`, for printing either an individual `Op` or the complete `Op::list`.

File `Op.cpp` includes `Op.h`, `Commute.cpp`, `Global.h` and `Gripe.h`, and defines the above mentioned class functions.

7.20 Parse.{h,cpp}

Header file `Parse.h` includes `Gordion.h` and defines:

- `namespace Parse`, which declares the command line input parsing functions described below. This namespace also defines the inline functions:
 - `eos`, which tests if an `istream` is at the end of the string;
 - `parse_args`, a templated function which extracts any number of arguments, of types inferred from the argument list, from a provided `istream`;
 - `isword`, which compares an input word against a specified command word. To be considered a valid match, the input word need merely match some non-empty initial portion of a command word, thereby allowing commands to be radically abbreviated;
 - `isstar`, which tests if the next word is a single `*`.

File `Parse.cpp` includes headers `Parse.h`, `Build.h`, `Commute.h`, `Numerics.h`, `Print.h`, `Rep.h`, `Save.h`, `Test.h`, `Blab.h` and `Gripe.h`, and defines the functions performing the main input command loop, subsequent parsing of input lines. Specifically:

- `Parse::read_input`, which runs a loop which optionally prompts, reads one line from the current input stream, optionally echos that line to `std::cout`, and then calls `parse` to process the input line. Versions taking both l-value and r-value references to the input `std::istream` are defined.
- `Parse::sig_catch`, which is called upon receipt of a `SIG_INT` interrupt signal. and sets `global.interrupt` to true and, if `Parse::read_input` is awaiting input, directly calls `Parse::quit`.
- `Parse::quit` which, for parallel execution, first calls `tbb::finalize` to wait for all worker threads to complete and then, if `timing` is set to `true` and the elapsed time is greater than 10 seconds, prints the total (wall clock) time elapsed, and finally calls `exit` to terminate the program.
- `Parse::parse_line`, which discards any initial white space or colon (`:`) characters and then breaks a given input line into separate commands separated by semicolons. For each resulting command, `parse_cmd` is called after first discarding any trailing white space. Ignoring initial colons, which are used as a command prompt, allows one to copy an entire prior command line, including the initial prompt, and paste it in to be re-evaluated.
- `Parse::parse_cmd`, which reads an initial word from the provided command, compares it using `isword` to the various command category keywords, and if matched calls one of the following keyword-specific parsing functions. (See appendix A for a listing of the recognized commands.)
- `Parse::parse_read`, which interprets a *read filename* command to process further command input from a specified file.
- `Parse::parse_write`, which interprets a *write filename* command to direct further output to a specified file. If the provided argument is a single `'-'`, then subsequent output is redirected back to `std::cout`.
- `Parse::parse_load`, which interprets a *load [filename]* command to load a binary save file (either a sys info or a vev data file).
- `Parse::parse_save`, which interprets a *save [filename]* command to write either a binary sys info file containing information on all defined observables, operators, and generators, the polynomial scripts defining the gradient, curvature, Lagrange bracket and geodesic equations, or else write or append to a binary vev data file records recording numerical observable expectation values along with adjustable coupling constant values.

- `Parse::parse_set`, which interprets and carries out various *set* commands allowing one to select the active irreducible symmetry representation, select the minimization stage (*gauge* or *fermi*), specify the numerical value of a symbolic coupling constant in the Hamiltonian (or free energy), set the ode and minimization error tolerances, select a Runge-Kutta method, specify the numerical output or save data file names and associated directories, set the limit on observable expectation values to be written to the output data file, set various global booleans including `dots`, `echo`, `timing`, `approx`, `autosave`, `Gen::autoToddgens` `Obs::check`, or set verbosity levels for functions in a particular source file. One place where abbreviated matching is not allowed is when entering the file name (without extension) in a verbosity setting command.
- `Parse::parse_reset`, which interprets and carries out `reset` commands allowing one to return to their initial values the active irreducible symmetry representation, the minimization stage, the max thread limit, all verbosity levels, the ode or minimization error tolerances, the Runge-Kutta method, the output data file, and the numerics data structures (including observable expectation values).
- `Parse::parse_build`, which interprets *build* commands and calls the `mk_stuff` functions in namespace `Build` to construct observables with a specified maximum strong-coupling order, or construct the symbolic expressions for the geodesic equations, gradient and curvature of the Hamiltonian (or free energy), or Lagrange bracket (for Hamiltonian theories).
- `Parse::parse_eval`, which interprets *evaluate* commands and calls the various `eval_stuff` functions in `Numerics.cpp` to numerically evaluate the Hamiltonian or free energy, its gradient or curvature, the Lagrange bracket matrix, the predicted minimum location, or the small oscillation spectrum.
- `Parse::parse_do`, which interprets a *do* command and, depending on the supplied argument(s), calls the function `numerics.do_step` to perform one Newton minimization step, `numerics.do_minimize` to perform iterative Newton minimization, or `numerics.do_flow` to progressively increment a given coupling constant from an initial value to final value in specified steps, carrying out Newton minimization at each step and saving results to the numerics data file.
- `Parse::parse_gen`, which interprets a *generator* command to add a new generator, suspend a previously defined generator or generator group, or activate a suspended generator or generator group.
- `Parse::parse_add`, which interprets an *add generator* command to define additional generators. This deprecated function may be removed at some point.
- `Parse::parse_print`, which interprets a wide variety of *print* commands, calling the various printing functions in `Print.cpp` detailed below.

- `Parse::parse_test`, which interprets a *test* command and, depending on the supplied argument, calls the function `Test::jacobi` to test that results of the commutation routines satisfy the Jacobi identity, or `Test::irreps` to test the orthogonality and completeness of the defined irreducible representation projectors.
- `Parse::parse_call`, which interprets various *call* commands which allow one to enter arguments and directly call various `Symm` and `Obs`-related functions:
 - `canon`, to canonicalize a given observable;
 - `compose`, to compose two symmetry transformations;
 - `transform`, to apply a symmetry transformation to a given observable;
 - `u1bound`, to evaluate the U(1) flux bound on an observable’s expectation order;
 - `noEbound`, to evaluate the electric field removal bound on an observable’s expectation order;
 - `classify`, to classify an observable’s expectation order;
 - `commute`, to evaluate the commutator of an indicated generator and observable;
 - `do_inner`, to evaluate the “inner commutator” of an `Eloop` observable.

This functionality is present for debugging purposes, not normal use.

- `Parse::parse_help`, which prints a command summary in response to either *help* or *?*.

This file also defines and uses the static variables:

- `bool echo`, which controls whether command input lines are echoed to `std::cout`;
- `bool timing`, which controls whether to report the time taken by each command (if greater than 10 milliseconds).
- `bool awaiting`, which lets the interrupt handler know when the program is awaiting input;
- `std::chrono::timepoint starttime`, which records the program start time.
- `ostream myout`, which holds a copy of the initial `std::cout` output buffer;

7.21 Poly.{h,cpp}

Header file `Poly.h` includes `Data.h`, `Term.h` and `Theory.h`, and defines:

- `class Polyindx`, which inherits from `std::array<uint,PSIZ>`. `Polyindx` objects are used to store tuples of indices of observables contained in an `ObsList`. As the first element of every `ObsList` is always the identity, a `Polyindx` can be used to represent linear, quadratic, or higher order monomials in observables, up to the limit set by `PSIZ`. Member functions are:

- `mysort`, which sorts an index array into decreasing order;
- `validate`, which verifies proper index sorting (used for debugging);
- `order`, which counts the number of non-zero indices and returns the resulting order of the monomial term (used for polynomial statistics).

Constructors are defined which allow one to specify 0, 1, or 2 observable indices. With multiple indices, `mysort` is called to place the contained indices in descending order. A friend `ostream& operator<<` for printing `Polyindx` objects is also declared.

- `class PolyTerm`, which inherits from `Term<real,Polyindx>` and represents an observable monomial, with `real` coefficient, of any order up to `PSIZ`. The `PolyTerm` class adds member functions:
 - `operator[]` for subscripting the underlying `Polyindx`;
 - `operator*` for properly combining two `PolyTerm` objects (provided the result is of no higher order than `PSIZ`), or scaling the coefficient of a `PolyTerm` by a specified factor;
 - `reindex` for re-indexing the observables referenced in a `PolyTerm` to a different `ObsList`;
 - `order`, returning the monomial order;
 - `validate`, which verifies correct sorting of the `Polyindx` (used for debugging);
 - `print`, which takes `ostream` and `ObsList` references as arguments, and prints the encoded monomial term to the stream.
- type alias `Polyvec = vector<PolyTerm>`, used to hold lists of `PolyTerm`'s.
- `struct Polyhash`, which declares a hashing function for use in `class PolyMap`.
- `class PolyMap`, which inherits from `hash<Polyindx,doub,Polyhash>` and whose instances encode polynomials up to order `PSIZ` in a hash table so that common terms may be automatically combined. Each `PolyMap` instance contains a reference (actually `reference_wrapper`) to the associated `ObsList` of the indexed observables. The class includes the additional member functions:
 - `add`, for adding a `PolyTerm` to the `PolyMap`, either by creating a new entry in the hash table if the `Polyindx` in the `PolyTerm` has not previously been entered, or by simply changing the `doub` value of an existing map entry with the same `Polyindx` key;
 - `add_gen`, for converting the `OpSum` of a `Gen` to `PolyTerm`'s and adding them to the `PolyMap`;
 - `negate`, which negates the coefficients of all stored elements in the `PolyMap`;
 - `purge`, which removes any map entries with vanishing coefficients;
 - `allzero`, which purges the map and then tests for no remaining entries.

An `friend ostream& operator<<` declaration is included for printing `PolyMap` instances.

- `class ObsPoly`, which inherits from `PolyVec` and whose instances encode polynomials up to order `PSIZ` in observables which are stored in a common `ObsList`. Each `ObsPoly` instance contains a reference to the associated `ObsList`. Member functions in the class are:
 - `push_map`, which adds all terms in a `PolyMap` to an `ObsPoly`;
 - `sort`, which sorts the entries in an `ObsPoly` based on the relative ordering of the index tuples of the individual terms;
 - `negate`, for negating an `ObsPoly`;
 - `allzero`, which tests for a vanishing `ObsPoly`;
 - `obslist`, which returns a reference to the associated `ObsList`.

Constructors allow creating an empty `ObsPoly` given just the `ObsList` to which it is linked, or creating an `ObsPoly` from the index of an already indexed observable in the specified `ObsList`. A `friend ostream& operator<<` declaration is included for printing `ObsPoly` objects.

- `struct Poly`, which inherits from `RecHdr` and serves as the header separating strings of `PolyTerm`'s in packed polynomial script records. The `struct` adds the member functions:
 - `begin()`, which returns a `const PolyTerm*` pointer to the `PolyTerm` which immediately follows the `Poly` in a polynomial script record;
 - `end()`, which returns a `const PolyTerm*` pointing to the end of the set of `PolyTerm`'s, whose number is recorded in the `RecHdr`, which follow the `Poly`.

These definitions enable range-based `for` loops to be used with a `Poly`. A `static constexpr int ptermsize = sizeof(PolyTerm) / sizeof(Poly)` is also defined for convenience. A `friend ostream& operator<<` declaration is included for printing an observable polynomial encoded by the `Poly` header plus following `PolyTerm`'s.

- `class PolyRec`, which inherits from `DataRec` and adds the data member:
 - `mutable vector<ulong> offset`, which is initialized on demand by the polynomial printing function indexing specific polynomials in the `PolyRec` and will record the offsets within the associated `PolyRec` of individual polynomials in the record.

The class also defines the further member functions and nested class:

- `add`, which takes a `RecHdr` and a `PolyMap` as arguments and appends the encoded polynomial, with its associated `Poly` header, to the `PolyRec`;
- `clear`, which calls `DataRec::clear` and also clears `offset`;

- `offloaded`, which signals whether the data record has been offloaded to disk by comparing the `entry().reclen` with the current vector size;
- `operator()`, which takes one, two, or three integer arguments, interpreted as the row, column and slice indices of a rectangular 1, 2 or 3 dimensional block of polynomials (with dimensions specified by the `nrow`, `ncol` and `nslice` members of the `RecIndx` associated with the `DataRec`) and returns a reference to the indexed polynomial. The first use of `operator()` for a given `PolyBlk` triggers a call to `slice_n_dice` to initialize the needed `offset` array.
- `slice_n_dice`, which populates the `offset` array with the offsets (in `RecHdr` units) of each individual `Poly` starting a new polynomial in the `PolyRec`;
- `class PolyIter`, an iterator for polynomials within a `PolyRec`, with the data member and member functions:
 - * `const Poly* ptr`, which will point to a `Poly`;
 - * `operator*`, which returns a reference to the `Poly` to which `ptr` points;
 - * `operator++`, which increments `ptr` by the size of the packed polynomial to which `ptr` points.
 - * `operator!=`, which compares the `ptr` member with another `PolyIter`;
- `begin()`, which returns a `PolyIter` pointing to the beginning of the `PolyRec`;
- `end()`, which returns a `PolyIter` pointing to the end of the `PolyRec`.

The definitions of `PolyIter`, `begin()`, and `end()` enable range-based `for` loops to be used with a `PolyRec`.

- a templated type alias making `PolyArr<N>` equivalent to `array<PolyRec,N>`, with a constructor taking a `RecordID`.

File `Poly.cpp` includes `Poly.h` and `Gen.h`, and defines the above functions including `Polyhash::operator()`, `PolyTerm::operator*`, `PolyMap::add`, the `ObsPoly` constructors and member functions, and the `operator<<` functions for `ObsPoly`, `PolyMap` and `Poly` objects.

7.22 Print.{h,cpp}

Header file `Print.h` includes `Gordion.h` and defines:

- `namespace Print`, which includes the functions:
 - `is_one`, testing if a supplied `doub` equals 1 to within numerical precision;
 - `is_int`, testing if a supplied `doub` equals an integer to within numerical precision;
 - plus all the various other printing functions described next.

File `Print.cpp` includes `Print.h`, `Canon.h`, `Gen.h`, `Numerics.h`, `Rep.h`, `Save.h` and `Blab.h`, and defines a long list of printing functions (all members of `namespace Print`):

- `Print::coeffprt`, for nicely printing coefficients of `Poly`'s or other linear combinations;
- `Print::print_obs`, which prints one or more observables in the current canonicalized observable list `ObsList::obs`. Differing overloads will print one observable (by giving its number in the list or typing its symbol string), a numeric range `[first, last]` of observables, or all observables;
- `Print::print_obs_select`, which prints all observables in `ObsList::obs` having specified creation and expectation orders and optionally a specified observable type;
- `Print::print_base`, which prints the contents of the non-canonicalized basic observable list `ObsList::base`;
- `Print::print_op`, which has several overloads to print all, a range, or one operator from the list of defined operators, `Op::list`;
- `Print::print_primary`, which prints the subset of primary `Op`'s;
- `Print::print_fermininit`, which prints `ObsList::fermininit`, holding the mapping between “closed” fermion bilinears and Wilson loops, used for fermion expectation value initialization;
- `Print::print_gen`, which will print one, or all, defined generators in the currently selected representation and stage, `global.info().gens[rep]`. For each generator, the output shows the current representation, the generator index prefixed with “g” for gauge or “f” for fermion generators and followed by a “*” suffix if the generator is T -odd, and then the actual generator definition.
- `Print::print_rep`, which prints the projection operators for all, or a specified individual irreducible representation row in `Rep::list`. Or, with no argument, reports the names of all defined `Rep`'s and the currently active `Rep`;
- `Print::print_symm`, which will print the explicit symmetry transformations for either all or a particular specified symmetry in `Symm::list`;
- `Print::print_symmsets`, which prints the list of all symmetry subsets used in canonicalization, `Canon::symmset`;
- `Print::print_couplings`, which prints names and values of all adjustable coupling constants stored in `Coupling::list`;
- `Print::print_hamiltonian`, which prints (for Hamiltonian theories) the current active Hamiltonian, namely `global.info().Hterms`;
- `Print::print_freeenergy`, which prints (for Euclidean theories) the current active free energy, namely `global.info().Hterms`;
- `Print::print_grad`, with overloads which will print all, or a selected portion of the symbolic gradient matrix, `global.info().grad`;

- `Print::print_curv`, with overloads which will print all, or a selected portion (specified by either one, two, or three indices), of the symbolic curvature matrix for the currently active representation and stage, `global.data().curv[rep]`.
- `Print::print_lagrange`, with overloads which will print all, or a selected portion (specified by one or two indices), of the Lagrange matrix for the currently active representation and stage, `global.data().lagr[rep]`;
- `Print::print_spectrum`, which prints the small oscillation frequencies stored in `numerics.spectrum` and previously evaluated by an `evaluate spectrum` or `do coupling` command.
- `Print::print_mode`, with overloads which print all, or a specified small oscillation eigenvector stored in `numerics.modes` and previously evaluated by an `evaluate spectrum` or `do coupling` command.
- `Print::print_geodesic`, with overloads which will print all, or a selected portion (specified by one or two indices), of the symbolic geodesic equations stored in `global.data().geos`;
- `Print::print_cache`, which prints all entries in `Canon::cache`;
- `Print::print_sysindex`, which prints a table of `RecIndx`'s in `global.sysindex`;
- `Print::print_vevindex`, which prints a list of coupling constant values and expectation value sets contained in a currently open vev data file;
- `Print::print_theory`, which prints the class of theory for the running compilation, along with lattice size if any dimensions are compactified;
- `Print::print_version`, which prints the version number of the running program.
- `Print::print_rkmethods`, which prints the names of known Runge-Kutta integration methods in `REdef::list`, and reports the currently selected method;
- `Print::print_stats`, which prints a variety of statistics including the number of defined observables, operators, ODE integrations and minimization steps, the number of commutator evaluations, ODE integrations and minimization steps, canonicalizations, and symmetry transformations performed, canonicalization cache performance, etc.;
- `Print::print_state`, which prints various global control variables, including theory class, active representation and minimization stage, maximum observable creation and strong-coupling orders, etc., as well as current verbosity levels.
- `Print::print_blab`, which prints the list of verbosity levels, `Blab::blablevel`;
- `Print::print_bcktlist`, which prints the contents of `global.info().bckt`, the observable bucket list for the current stage;

- `Print::print_obsstats`, which prints a table of counts of defined observables classified according to type, creation order, and expectation order;
- `Print::print_geostats`, which prints the number of non-vanishing geodesic equations, the number of terms in those equations, and the breakdown by monomial order of those terms.

7.23 Rep.{h,cpp}

Header file `Rep.h` includes `Symm.h` and defines:

- `class Proj`, which inherits from `vector<doub>`. Every `Proj` object defines a projection operator for an irreducible symmetry group representation, with the base `vector<double>` providing the list of coefficients with which each symmetry transformation in `Symm::list` is weighted and then summed to form the relevant projector. These projection operators are related to the representation matrices via

$$P_{mn}^{\lambda} = \frac{d_{\lambda}}{|G|} \sum_{g \in G} D^{\lambda}(g)_{mn}^{*} g, \quad (14)$$

where G denotes the symmetry group (with dimension $|G|$), λ labels an irreducible representation (with dimension d_{λ}), and $D^{\lambda}(g) \equiv \|D^{\lambda}(g)_{mn}\|$ defines the associated matrix representation.

The `Proj` class adds data members:

- `string name`, holding the representation name. Conventional names for crystallography point groups are used, augmented by a `p` or `m` denoting a representation even or odd, respectively, under charge conjugation. For multi-dimensional representations two further characters, taken either from the sequence `abcd` or from `xyzw`, are appended to the name to indicate the projector indices;
- `doub denom`, a denominator normalizing the sum of symmetry transforms;

Member function declarations are:

- `operator()` for composing projectors;
- `operator+`, `operator-`, `operator+=` and `operator-=`, used in `Rep::repinit` for constructing representation projectors;
- `allzero` for testing complete vanishing (used for testing orthonormality),
- `C_even`, signaling a charge-conjugation even representation;
- `indices` which, for multi-dimensional irreps, returns the conversion of the two final characters in the projector name into their corresponding numerical index values;
- `rowname` which, for multi-dimensional irreps, returns the representation row name by stripping off the last character from the projector name;

- `rename` which, for multi-dimensional irreps, returns the representation name by stripping off the last two characters from the projector name.

The static data member `vector<Proj> list` holds the list of all defined projectors.

A friend `ostream& operator<<` declaration is included for printing `Proj` objects.

- `class Rep`, which inherits from `vector<Proj>` and holds the projectors which correspond to a given row of an irreducible symmetry representation.

The `Rep` class adds data members:

- `string name`, holding the representation row name. For multi-dimensional irreps this is the same as the names of the projectors held by the `Rep` with their final character stripped off;
- `bool C_even`, which indicates whether the representation is C even or odd.

The static data member `Index<Rep> list` holds the indexed list of all defined projectors. The fully symmetric vacuum representation is always first in the list.

Static functions in class `Rep` are:

- `known`, which maps representation names to the index of the associated `Rep` instance in `Rep::list` (and throws an exception if not found);
- `repinit`, which initializes `Rep::list`.

A friend `ostream& operator<<` declaration is included for printing `Rep` objects.

File `Rep.cpp` includes `Rep.h` and `Gripe.h`, and contains definitions of the above `Proj` and `Rep` member functions, and the `ostream& operator<<` definitions for both `Proj` and `Rep` objects.

Within `Rep::repinit`, the various `constexpr` functions defined in `Theory.h` for distinguishing different lattice types (like `theory.isR2()` and `theory.isR3()`) are used to select pre-programmed definitions of projections onto individual components of irreducible representations of the lattice symmetry point group times charge conjugation for theories living on $\mathbb{R}$, $\mathbb{R}^2$, $\mathbb{R}^3$, or $\mathbb{R}^2 \times S^1$. For $\mathbb{R}^4$, only the charge conjugation even and odd representations which are fully symmetric under all lattice permutations and reflections are currently coded. The various representation labels begin with standard cubic crystallography representation names (those of C_{4v} for theories on $\mathbb{R}^2$, O_h for theories on $\mathbb{R}^3$, and D_{4h} for theories on $\mathbb{R}^2 \times S^1$) followed by a `p` or `m` indicating even or odd behavior, respectively, under charge conjugation followed, for multi-dimensional representations, by two further letters drawn from either `abcd` or `xyzw` indicating the indices of a particular projector within the representation. Thanks to the operator overloads for multiplying and adding representation projectors, the resulting definitions of various representation projections are fairly concise. As an example, `(s("E") - s("C")) * (s("E") + s("Rx")) * (s("E") + s("Ry")) * (s("E") - s("Pxy"))` serves to enter the definition of the two-dimensional `B1m` representation, which is odd under charge conjugation and $x \leftrightarrow y$ permutations and even under $x \leftrightarrow -x$ and $y \leftrightarrow -y$ reflections. (Here, `s` is a pointer to the `Rep::known` function.)

7.24 RK.{h,cpp}

Header file `RK.h` includes `Gordion.h` and defines:

- `class RKdef`, whose instances define an embedded explicit Runge-Kutta integration method.²⁴ Data members of the class are:
 - `const int nstage`, giving the number of stages (# derivative evals per step);
 - `const int order`, giving the order of the higher-order approximation;
 - `const double* c`, pointing to the list of RK nodes;
 - `const double* const *a`, pointing to the matrix of RK coefficients;
 - `const double* b`, pointing to the list of (higher order) RK weights;
 - `const double* bh`, pointing to the list of (lower order) RK weights;
 - `const double* db`, pointing to the list of error vectors weights, equaling $bh - b$;
 - `char* name`, pointing to the method name;
 - `bool fsal`, indicating if the scheme is “First Same as Last (FSAL),” meaning that the last derivative evaluation in a step may be reused as the first derivative evaluation needed in the next step.
 - `static const std::array<RKdef, ...> list`, holding the complete set of defined Runge-Kutta methods.

File `RK.cpp` includes `RK.h` and provides the definitions of particular Runge-Kutta schemes, all of which are coded as `constexpr` definitions. The currently defined RK methods are the following (see the indicated references for details):

- `bs32`: Bogacki/Shampine 3(2) [21];
- `ss32`: Sofroniou/Spaletta 3(2) [22];
- `ss43`: Sofroniou/Spaletta 4(3) [22];
- `dp54`: Dormand/ Prince 5(4) [23];
- `bs54`: Bogacki/Shampine 5(4) [24];
- `jb54`: John Butcher 5(4) [25];
- `jv65`: Jim Verner 6(5) [26].

²⁴“Embedded” RK methods refer to Runge-Kutta schemes in which a common set of derivative evaluations is used to derive two different approximations to the function being integrated, with the difference between the approximations providing an error estimate which may be used for adaptive step size control.

7.25 Save.{h,cpp}

Header file `Save.h` includes `Gordion.h`, `Coupling.h`, `Data.h` and `Version.h`, and defines

- `namespace Save` which declares the various functions described next for writing data to disk or reloading saved data back from disk, and defines:
 - `struct FileHdr` whose instances will constitute a 16-byte header written to the beginning of sys info and vev data files. The `FileHdr` struct contains:
containing:
 - * `char8 name`, for holding a theory name;
 - * `Version version`, for holding the program version;
 - * `uint ncoup`, for holding the number of adjustable coupling constants;
 - * `uint nvev`, for holding the number of observable expectation values,
 - and defines the convenience functions:
 - * `vevsize`, returning `nvev * sizeof (doub)`, the byte-length of each list of expectation values;
 - * `couplesize`, returning `ncoup * sizeof (Coupling)`, the byte-length of the adjustable coupling list;
 - * `cvsetsize`, returning `vevsize() + couplesize()`;
 - * `is_sysfile`, returning `true` if `ncoup` and `nvev` vanish;
 - * `is_vevfile`, returning `true` if `ncoup` or `nvev` are non-vanishing.

The namespace `Save` also defines the static variables:

- `FileHdr filehdr`, holding the file header of an open save file and used by the various `Save` commands;
- `string syspath`, holding the full pathname of an open sys info file;
- `std::mutex savemutex`, used to synchronize writing of a sys info file by multiple threads.

File `Save.cpp` includes the header files `Save.h`, `Canon.h`, `Gen.h`, `Numerics.h`, `Rep.h`, `Blab.h` and `Gripe.h`, and defines:

- `static constexpr int rhdrsiz`, as shorthand for `sizeof (RecHdr)`;
- `static symb symbbuf[rhdrsiz]`, a `RecHdr`-sized `symb` array defined with `alignas (rhdrsiz)` and used by `Save::write_obs` and `Save::write_op` to transfer trailing portions of `Obs` or `Op` symbol strings to a `DataRec` record;
- `Save::save_sys`, which takes as argument a sys info file name, strips off any directory portion (redefining `global.savedir`), attempts to open the file for writing in binary mode and, on success, calls the following `Save::write_*` routines starting with `write_header` and ending with `write_sysindex`. If the theory contains fermions, separate sets of the intervening data records are written for each minimization stage;

- `Save::save_vev`, which takes as argument a vev data file name, strips off any directory portion (redefining `global.savedir`), attempts to open the file for I/O in binary append mode and, on success, calls `read_header` for a pre-existing file to validate the file header, or `write_header` for a newly created vev data file, and then calls `Save::write_coup` and `write_vev` to append a coupling+vev data set to the file.
- `Save::write_header`, which writes a `FileHdr` instance containing `theory.name`, `global.version`, and supplied values of the number of coupling constants and number of vev's to the open sys info or vev data file;
- `Save::write_op`, which steps through the `Op`'s of `global.info(stage).ops`, copying their metadata to a `OpHdr` and then appending that `OpHdr` followed by the `SymbStr` defining the `Op` to `global.data().op`. The current file position and number of stored `Op`'s are recorded in `global.data().op.entry()`, the resulting serialized list of operators is written to the sys info file, and then this serialized data record is cleared (with its memory forcibly released via the C++ swap trick);
- `Save::write_obs`, which loops over all `Obs`'s for the current stage in `ObsList::obs`, copying their metadata to a `ObsHdr` struct and then appending that `ObsHdr` followed by the `SymbStr` defining the `Obs` to `global.data().obs`. After recording the current file position and number of stored observables in `global.data().obs.entry()` and writing the resulting serialized list of observables to the sys info file, this serialized data record is cleared (with its memory forcibly released);
- `Save::write_gen`, which loops over all symmetry representations and then over the `Gens`'s in `global.info().gens[rep]`, copying each `Gen`'s metadata to a `GenHdr` and then appending that `GenHdr` followed by the `OpTerm`'s defining the generator to `global.data().gen`. After recording the current file position and number of stored `Gen`'s in `global.data().gen.entry()` and writing the resulting serialized list of generators to the sys info file, this serialized data record is freed;
- `Save::write_grad`, which writes `global.data().grad` to the sys info file, after first recording the current sys info file position in `global.data().grad.entry()`;
- `Save::write_curv` which, for each symmetry representation, writes the polynomial scripts in `global.data().curv[rep]` to the sys info file (after first recording the current file position in `global.data().curv[rep].entry()`);
- `Save::write_lagr` which, for each symmetry representation, writes the polynomial scripts in `global.data().lagr[rep]`, to the sys info file (after first recording the current file position in `global.data().lagr[rep].entry()`);
- `Save::write_geos` which, for each observable bucket, writes the polynomial scripts in `global.data().geos[bckt]` to the sys info file (after first recording the current file position in `global.data().geos[bckt].entry()`);

- `Save::write_geo_bckt`, which acquires a mutex lock on the sys info file and then writes `global.data().geos[bckt]`, for a specified bucket of observables, to the sys info file, overwriting the initial sys info file index before appending an updated file index and releasing the lock. If `global.geoswap` is `true`, the geodesic bucket data record is then freed;
- `Save::write_stat`, which updates `global.data().stat.entry()`, and then writes the statistics counters in `global.info().count` to the sys info file;
- `Save::rewrite_stat`, which seeks to the file position of the `Stat` record specified in `global.data().stat.entry()`, and then writes to that file location in the sys info file the current values of the statistics counters in `global.info().count`;
- `Save::write_sysindex`, which appends the contents of `global.sysindex` to the sys info file;
- `Save::write_coup`, which appends the contents of `Coupling::list` to the vev data file;
- `Save::write_vevs`, which appends the contents of `numerics.vev` to the vev data file;
- `Save::load_save`, which takes as arguments a save file name and data set index (default `-1`), strips off any directory portion in the supplied file name (redefining `global.savedir`), attempts to open the file for reading in binary mode and on success calls `read_header` to determine if the supplied file is a sys info or vev data file. For sys info files, the following `Save::read_*` routines are called starting with `read_sysindex` and ending with `read_stat`. For vev data files, `load_vev` is called with the supplied data set index passed as argument;
- `Save::load_vev`, which takes a data set index as argument and calls `read_coup` and `read_vev` to load the specified data set from an open vev data file. A set index of `-1` indicates the final data set in the file.
- `Save::read_header`, which reads the `FileHdr` stored at the beginning of sys info and vev data files, and compares the contained program version to `global.version` and the contained theory name to `theory.name`, (raising an exception if either are incompatible with the running program), and for vev data files validating the values of `ncoup` and `nvev` in the file header;
- `Save::read_sysindex`, which reads `sizeof global.sysindex` bytes from the end of a sys info file into `global.sysindex`;
- `Save::read_op`, which reads into `global.data().op` the serialized `Op` data record for the specified stage from the sys info file (after seeking to the file position recorded in the associated `RecIdx` read in by `read_index`), then steps through the data record reconstructing each `Op` and storing it in `global.info(stage).ops`, and lastly frees the data record;

- `Save::read_obs`, which reads into `global.data().obs` the observable data record for the specified stage from the sys info file (after seeking to the file position recorded in the associated `RecIndx` read in by `Save::read_index`), then steps through this record reconstructing each `Obs` and storing it in `ObsList::obs`. The routine also sets `global.info().nobs` and `global.info().maxord` based on the reconstructed observables, calls `global.mk_bcktlist` to (re)initialize the observable bucket list, and frees the data record;
- `Save::read_gen`, which reads into `global.data().gen` the serialized generator data record for the specified stage from the sys info file (after seeking to the file position recorded in the associated `RecIndx` read in by `read_index`), steps through the data record reconstructing each `Gen` and storing it in `global.info().gens`, appropriately updates `global.info().neven` and `global.info().maxgen`, and lastly frees the data record;
- `Save::read_grad`, which reads into `global.data().grad` the polynomial script data record whose file position and length is recorded in the associated `RecIndx` read in by `read_index`;
- `Save::read_curv`, which loops over all representations, seeking to the file position recorded in the associated `RecIndx` read in by `read_index` and reading the specified polynomial script data into `global.data().curv[rep]`;
- `Save::read_lagr`, which loops over all representations, seeking to the file position recorded in the associated `RecIndx` read in by `read_index` and reading the specified polynomial script data into `global.data().lagr[rep]`;
- `Save::read_geos` which, for each bucket of observables, seeks to the file position recorded in the associated `RecIndx` read in by `read_index` and then reads the specified polynomial script data into `global.data().geos[bckt]`;
- `Save::read_geo_bckt`, which seeks to the file position of a particular bucket of geodesics and then reads into the specified `global.data().geos[bckt]` the off-loaded polynomial script data. To enable efficient multi-threading, a `thread_local` `fstream` with a custom stream buffer is employed. Each call to `read_geo_bckt` reads data into a `thread_local` data array which is swapped into the desired `global.data().geos` entry. Calls to `read_geo_bckt` with a negative bucket number are used to signal a reverse swap of the thread-local data array;
- `Save::read_stat`, which seeks to the file position in the associated `RecIndx` read in by `read_index` and then reads the contents of the previously saved `Counters` structure into `global.data().count`;
- `Save::read_coup`, which seeks to the file position of the specified data set in the vev data file (with `-1` indicating the final data set in the file) and then reads the list of `Coupling`'s into either `Coupling::list` or supplied temporary coupling list (with the latter case used by `Print::print_vevindex`);

- `Save::read_vevs`, which seeks to the file position in the of the specified data set in the vev data file (with -1 indicated the final data set in the file) and then reads the specified block of expectation values into `numerics.vev`.

7.26 Splice.h

Header file `Splice.h` defines:

- `struct Splice`, which holds three data members:
 - `signed char coeff`, which is either ± 1 or 0 ;
 - `signed char prefix`, which is either a type-and-reflection `symp` code or -1 ;
 - `signed char suffix`, which is either a type-and-reflection `symp` code or -1 .
- The static `Splice splicetbl[10][10][2]` splicing table used in `Commute.cpp`. The first two indices are the type-and-reflection (`tnR`) codes of two `symp`'s, with the last two-valued index distinguishing two possible resulting terms needed for some combinations.

To understand `splicetbl`, consider a commutator $[\text{tr}(\alpha s_1), \text{tr}(s_2 \beta)]$ where s_1 and s_2 are symbols each representing one of:

- a forward-directed gauge link (shorthand notation `1`),
- an inverse gauge link (`L`),
- an electric field transforming at the start of its link (`E`),
- an electric field parallel transported to the end of its link (`LE1`),
- an electric field followed by its forward link (`E1`),
- an electric field preceded by its inverse link (`LE`),
- a squared electric field transforming at the start of its link (`EE`),
- a squared electric field parallel transported to the end of its link (`LEE1`),
- a squared electric field followed by its forward link (`EE1`),
- a squared electric field preceded by its inverse link (`LEE`),

with α and β denoting the remainder of the `SymbStr`'s for each factor. Assume that both s_1 and s_2 reside on the same lattice link.

The above list shows the ten possible pure gauge symbol types associated with any given lattice link. (Comments using the above shorthands are present in the entries of `Splice.h` defining `splicetbl`.) The result of evaluating the commutator, using the underlying lattice theory commutation relations between electric field operators and gauge links, takes the form:

$$[\text{tr}(\alpha s_1), \text{tr}(s_2 \beta)] = \sum_i \text{sgn}_i \text{tr}(\text{prefix}_i \alpha \text{suffix}_i \beta) + \cdots, \quad (15)$$

where the ellipsis denotes further terms arising from the commutation of other pairs of symbols in the two factors. Here sgn_i is a $\pm$ sign, each prefix_i is either empty,

or some symbol residing on the common link of s_1 and s_2 , each $suffix_i$ is likewise either empty or some symbol on that same link, and the number of terms in the result may be 0, 1, 2 or 3 depending on the types of the two symbols s_1 and s_2 . The **Splice** elements of the **splicetbl** provide these needed signs, prefixes and suffixes. (More precisely, a **Splice** element provides the type-and-reflection portion of the prefix and suffix symbols; the axis direction bits are set by those of s_1 and s_2 .) A zero coefficient indicates that the term is not needed. A prefix of -1 is used to indicate that no prefix is needed, and likewise for a suffix of -1 . The commutation routines perform the above indicated joining of symbol strings using the entries of both **splicetbl**[s_1][s_2] and (with a sign flip) **splicetbl**[s_2][s_1]. The **splicetbl** is constructed so that non-vanishing entries are only present when the first table index does not represent a link or inverse link.

7.27 Symb.{h,cpp}

Header file **Symb.h** includes **Gordion.h** and defines:

- **class SymbStr**, which inherits from **std::string** and whose instances are used to hold **symb** strings. The **SymbStr** class declares the member functions:
 - **join**, which appends a **symb** or **SymbStr** with appropriate canceling of links and inverse links, and combining into appropriate ligatures links and electric field insertions living on the same lattice link;
 - **joinends**, which properly join the ends of a **SymbStr**;
 - **excise**, which removes an interior portion of a **SymbStr**;
 - **isclosed**, which tests if a **SymbStr** forms a closed loop;
 - **print**, which converts a **SymbStr** to a printable string.

A static inline boolean member **dots** controls whether the print form of a **SymbStr** includes periods between symbols. A **friend ostream& operator<<** declaration is included for printing **SymbStr** objects.

- **namespace Symb**, which defines the specific encoding used for symbols. This encoding is designed to allow the same encoding to be used for theories of any dimension (up to 4). The lowest two bits of a **symb** indicate the link axis for a gauge field **symb**, or flavor plus staggering for a fermion **symb**. The third lowest bit signals reflection (for gauge) or conjugation (for fermions). The next three bits encode the type of the **symb**: **link**, **E**, **Elink**, **EE**, **EElink**, **fermion**. The resulting values 0, ..., 39 represent the 40 possible pure gauge symbols in up to four dimensions, while values 40, ..., 47 represent up to two flavors of fermion fields and their conjugates, with or without an included lattice staggering factor. A few further symbol values are used for special purposes, namely to represent either the $O(N^2)$ gauge or $O(N)$ fermion contribution to the entropy in Euclidean theories, or in Hamiltonian theories to distinguish the $O(N)$ fermion contribution to the expectation value of gauge field kinetic energy terms.

The **Symb** namespace also defines various useful constants and flags:

- `symb Rflag (0x04)`, the reflection (or conjugation, for fermions) bit mask;
- `symb addE (0x10)`, the change in `symb` codes for converting a `Link` $\rightarrow$ `Elink`;
- `symb YMend (0x28)`, the number of pure gauge symbols;
- `symb Nsymb (0x30)`, the number of pure gauge plus fermion symbols;
- `symb TandR (0x3c)`, the type and reflection bit mask;
- `symb EntG (0x34)`, the extra symbol for gauge entropy;
- `symb EntF (0x35)`, the extra symbol for fermion entropy;
- `symb Null (0x40)`, the null ligature flag;
- `symb X (-1)`, the invalid ligature or empty prefix/suffix marker;

plus static data members:

- `vector<const string> symbname`, which serves to map symbol values to print forms of each symbol;
- `symb ligtable[10][10]`, which gives the type-and-reflection codes for combining two `symb`'s referring to the same lattice link into the appropriate single `symb`, e.g., for merging an electric field insertion together with its corresponding link. (See the comments in `Symb.h` for details.)

Functions in namespace `Symb` are:

- `to_symb`, which does the inverse mapping from printable character(s) to the corresponding `symb` value or to a `SymbStr` object;
- `in_thy`, which tests whether a given `symb` value is valid for the particular theory under consideration (only used when printing symmetry transformations);
- `ligature`, which uses the static `ligtable` and identifies when two adjacent symbols should be merged into a single symbol based on their types and lattice directions;

plus a slew of additional inline convenience functions:

- `islink`, which tests if a symbol is type `link`;
- `nonlink`, which tests if a symbol is not type `link`;
- `isE`, which tests if a symbol is type `E`;
- `isEE`, which tests if a symbol is type `EE`;
- `isElink`, which tests if a symbol is type `Elink`;
- `isEElink`, which tests if a symbol is type `EElink`;
- `EorElink`, which tests if a symbol is type `E` or `Elink`;
- `EEorEElink`, which tests if a symbol is type `EE` or `EElink`;
- `inclE`, which tests if a symbol is any of the `E`-containing types;
- `isEE_F`, which tests if a symbol is a fermion portion of an `EE`;
- `isentpy`, which tests if a symbol represents entropy;

- `isferm`, which tests if a symbol represents a fermion;
- `isxtra`, which tests if a symbol is one of the extras (not in general observables);
- `nostep`, which tests if a symbol does not represent a step between lattice sites;
- `isrefl`, which extracts the reflection bit of a symbol;
- `isstag`, which extracts the fermion staggering bit;
- `flav`, which extracts the bit representing fermion flavor;
- `axis`, which extracts the two bits defining the lattice axis of a gauge symbol;
- `direction`, which extracts the three bits defining lattice axis and link direction;
- `type`, which returns the type of a `symb`;
- `tnR`, which returns the type plus reflection portion of a `symb`;
- `refl`, which returns the reflection of a `symb`;
- `conj`, which returns the conjugation of a `symb`;
- `stag`, which returns the fermion `symb` with opposite staggering factor;
- `step`, which returns `+1`, `-1`, or `0` depending of whether a `symb` represents a forward step, backward step, or no step.

File `Symb.cpp` includes `Symb.h`, `Theory.h` and `Gripe.h`, and contains the definitions of the `namespace::Symb` member functions `in_thy`, `to_symb`, `ligature`, the class `SymbStr` member functions `join`, `excise`, `isclosed` and `print`, plus `ostream& operator<<` for `SymbStr` objects.

7.28 `Symm.{h,cpp}`

Header file `Symm.h` includes `Op.h` and defines:

- `class SymmTerm`, which inherits from `Term<uint>` and adds the member functions `operator/ (double)` for rescaling the term coefficient, and `operator+` and `operator-` for adding or subtracting `SymmTerm`'s, producing a `SymmSum`. Each `SymmTerm` instance holds the index of a symmetry (in `Symm::list`) plus a numerical coefficient;
- type alias `SymmVec = vector<SymmTerm>`, for sums of `SymmTerm`'s;
- `class SymmSum`, which inherits from `SymmVec` and adds `operator+`, `operator-` and `operator*` member functions for adding, subtracting and multiplying `SymmSum`'s;
- `struct Symmhash`, which declares `operator()` for computing a `Symm` hash function;
- type alias `Symmmap = hash<Symm,int,Symmhash>`, for mapping symmetry transformations to their index in `Symm::list`;
- `class Symm`, which holds information defining a symmetry transformation. Data members are:

- `array<symb,Nsymb> map`, which maps symbol values (representing a link, Elink, Fermion, etc.) to the appropriately transformed symbols;
- `bitset<Nsymb+1> sgn`, which records which symbols need to be accompanied by a sign flip, plus one more bit signaling transformations which include charge conjugation;
- `string name`, which gives a standard name to the symmetry transformation;
- `int indx`, which gives the index of the transformation in the `Symm::list` of all symmetry transformations.

Member function declarations are:

- `operator()`, with two overloads for applying the symmetry transformation to either a supplied `Op` or another `Symm` symmetry transform;
- `operator==`, for comparison with another `Symm`;
- `operator+ & operator-`, for building linear combinations of symmetry transformations;
- `isCodd`, returning a boolean indicating if the transformation includes charge conjugation, which reverses the order of symbols;
- `print`, which produces a printable string showing the `Symm` transformation.

A friend `ostream& operator<<` declaration for `Symm` objects is also included.

Static data members in `class Symm` are:

- `array<symb,Nsymb> idmap`, the identity transformation, used in the default `Symm` constructor;
- `Symmmap trans2indx`, for mapping an explicit symmetry transformation to its corresponding list index;
- `Index<Symm> list`, an indexed list of all symmetries;

while static function declarations are:

- `symminit`, for initialization of `Symm::list` and `Symm::trans2indx`;
- `known`, which returns a `SymmTerm` corresponding to a known symmetry name (or throws an exception for unknown names).

File `Symm.cpp` includes `Symm.h` and contains the definitions of the above `Symm` and `SymmSum` member functions, the `Symmhash::operator()` hashing function, and the `operator<<` for `Symm` objects.

The `Symm::symminit` initialization function loops over whether or not to include charge conjugation, then over all permutations of lattice directions which leave unchanged the overall lattice volume (if compactified), and then over all possible reflections of lattice directions. Inside these loops, a cyclic decomposition of the permutation is performed (to construct a nice naming of symmetries) and the specified `map` and `bitset sgn` members of a `Symm` are initialized, with the result then entered in `Symm::list` and `Symm::trans2indx`.

7.29 Term.h

Header file `Term.h` includes `Print.h` and defines:

- `class Term<T>`, whose instances contain two data members, a `doub coeff` and a type `T item`. Vectors of these templated class objects are used to represent linear combinations of objects of type `T`. (The `Term<T>` class does not simply inherit from a type `T` base class because `T` may be an elementary type, not a class, and hence cannot serve as a base class.) A friend `std::ostream& operator<<` is defined for printing `Term` objects.

Instances of this template class are used to store `Polyindx` monomials in `ObsPoly`'s, `OpTerm` monomials in `OpSum`'s and `Symm` monomials in `SymmSum`'s.

7.30 Test.{h,cpp}

Header file `Test.h` includes `Gordion.h` and defines

- `namespace Test`, containing the functions:
 - `Test::irreps`, which tests if the defined representation projectors satisfy the proper orthonormality relation,

$$P_{mn}^{\lambda} P_{m'n'}^{\mu} = \delta^{\lambda\mu} \delta_{nm'} P_{mn'}^{\lambda}, \quad (16)$$

and the completeness condition,

$$\sum_{\lambda} \sum_{m=1}^{d_{\lambda}} P_{mm}^{\lambda} = e, \quad (17)$$

(where e is the identity group element);

- `Test::jacobi`, which tests the validity of the Jacobi identity. One overload evaluates the identity with a specified observable and all generators in the currently selected representation, that is

$$[gen_1, [gen_2, obs]] + [[gen_2, gen_1], obs] - [gen_2, [gen_1, obs]] \stackrel{?}{=} 0, \quad (18)$$

with the generators and observable specified by their indices within the representation's list of generators and the `ObsList::obs` observable list. A related overload allows one to specify two individual `Op`'s by hand along with an observable's index, while a final overload tests the identity

$$[gen_1, [gen_2, gen_3]] + [gen_2, [gen_3, gen_1]] + [gen_3, [gen_1, gen_2]] \stackrel{?}{=} 0, \quad (19)$$

for all triples of generators for the current minimization stage and symmetry representation. These routines provide consistency checks on the coding of the commutation routines.

File `Test.cpp` includes `Test.h`, `Commute.h`, `Global.h`, `Rep.h` and `Gripe.h`, and contains the definitions of the above functions.

7.31 Theory.{h,cpp}

Header file `Theory.h` includes `Gordion.h` and defines:

- `enum LATTTYPE { Hamilton, Euclid}`, used just in the initialization of the pre-defined class `Theory` instances described below;
- `class Coord`, with data member:
 - `union { short comp[4] ; ulong point ; }`, which is used for encoding the overall lattice dimensions (with 0 indicating an infinite dimension), or storing lattice site coordinates or displacements (needed for finding loop self-intersections). The unnamed union allows accessing either individual coordinate components or the entire set of coordinates as an `unsigned long`, enabling convenient sorting or equality testing of `Coord` instances.

Additional member functions are:

- `modlen`, which for periodic lattice dimensions maps a particular coordinate to lie within the defined period;
- `modulo`, which for periodic lattices, maps all coordinates to lie within the box;
- `isclosed`, which test for a vanishing lattice displacement (after first calling `modulo` for periodic lattices).
- `class Theory`, whose single `constexpr` instance `theory` holds the basic information defining the type of theory. Data members are:
 - `char8 name`, a short theory name which is used as part of the default names for save files and numeric output files;
 - `char dim`, the lattice dimension;
 - `char nf`, the number of fermion flavors;
 - `char nrep`, the sum of dimensions of all irreducible symmetry representations;
 - `bool euclid`, indicating whether theory type is Euclidean (default `false`) or Hamiltonian;
 - `Coord box`, the lattice size.

Various member functions are defined for convenience:

- `unbounded`, signaling an unbounded lattice in all dimensions;
- `oddboxsize`, indicating that one or more lattice dimensions are compactified with an odd length;
- `box_size`, which returns a string with a printable form of the lattice size;
- `parent`, which returns a `char8` containing, for theories with fermions, the name of the corresponding Yang-Mills theory without fermions;
- `isR1`, `isR2`, `isR3`, and `isR4`, indicating that the lattice is unbounded and of the indicated dimension;

- `isS1`, `isR1xS1`, and `isR2xS1`, indicating that the lattice has one compactified dimension and the indicated number of uncompactified dimensions.

Static member functions are `theoryinit`, which validates the theory definition settings, and `theorydefn`, which defines the specific theory of interest.

A series of `#if ... #elif ... #elif ... #else` tests for the preprocessor symbol definitions matching a pre-defined theory name and appropriately defines the `constexpr theory`. Currently, available selections are:

- `YM1h`: for Hamiltonian Yang-Mills on a 1D lattice compactified to a single link;
- `YM2h`: for Hamiltonian Yang-Mills on a 2D infinite lattice,
- `YM3h`: for Hamiltonian Yang-Mills on a 3D infinite lattice,
- `YM1e`: for Euclidean Yang-Mills on a 1D lattice compactified to a single link;
- `YM2e`: for Euclidean Yang-Mills on a 2D infinite lattice,
- `YM3e`: for Euclidean Yang-Mills on a 3D infinite lattice,
- `YM4e`: for Euclidean Yang-Mills on a 4D infinite lattice,
- `QCD1h`: for Hamiltonian single flavor QCD on a 1D infinite lattice,
- `QCD2h`: for Hamiltonian single flavor QCD on a 2D infinite lattice,
- `QCD3h`: for Hamiltonian single flavor QCD on a 3D infinite lattice,
- `QCD1e`: for Euclidean single flavor QCD on a 1D infinite lattice,
- `QCD2e`: for Euclidean single flavor QCD on a 2D infinite lattice,
- `QCD3e`: for Euclidean single flavor QCD on a 3D infinite lattice,
- `YM3ep`: for Euclidean Yang-Mills on S_1 compactified ($\infty \times \infty \times 1$) 3D lattice,
- `YM3hp`: for Euclidean Yang-Mills on S_1 compactified ($\infty \times \infty \times 1$) 3D lattice.

If none of these preprocessor tests succeed then a compilation error message is generated;

- `constexpr int PSIZ = theory.euclid ? 2 : 4`, which sets the size of index tuples used to represent products of indexed observables, with higher order tuples needed for Hamiltonian theories.
- `constexpr int NREP = theory.nrep`, used for dimensioning multiple arrays defined in `Global.h`.

This header files also includes a few static assertions validating the theory definitions. File `Theory.cpp` includes `Theory.h`, `Print.h` and `Obs.h`, and defines:

- Preprocessor symbol `MASSSCALE` as 0, if not already defined. Setting `MASSSCALE` to some non-zero numeric value in the `Makefile` will scale the fermion mass with a specified power of the gauge coupling; see Eq. 22.

- `Theory::theoryinit`, which stores the theory name in the default directories for save files and output data, and prints the selected theory name and program version; and then prints the theory selection plus the program version.
- `Theory::theorydefn`, which makes standard definitions of a Kogut-Susskind lattice Hamiltonian or Wilson Euclidean free-energy. Specifically, these definitions are:

$$H_{\text{gauge}}/N = \lambda \text{tr} E^2 + \lambda^{-1} \text{tr}(2 - U - U^\dagger), \quad (20a)$$

$$F_{\text{gauge}}/N = \lambda^{-1} \text{tr}(2 - U - U^\dagger) - S, \quad (20b)$$

for one-dimensional Yang-Mills, either Hamiltonian or Euclidean, respectively, compactified to a single link. For higher dimensional Yang-Mills, either Hamiltonian or Euclidean, the definitions are:

$$H_{\text{gauge}}/N = \frac{1}{4}\lambda \sum_{\ell} \text{tr}(E[\ell]^2) + \lambda^{-1} \sum_p \text{tr}(2 - U[\partial p] - U[\partial p]^\dagger), \quad (21a)$$

$$F_{\text{gauge}}/N = \lambda^{-1} \sum_p \text{tr}(2 - U[\partial p] - U[\partial p]^\dagger) - S, \quad (21b)$$

where ℓ labels positively oriented lattice links, p denotes positively oriented plaquettes, $E[\ell]$ is the $U(N)$ electric field operator on link ℓ , and $U[\partial p]$ denotes the $U(N)$ gauge connection around plaquette p . For QCD, the additional fermion contribution to the Hamiltonian or free energy is:

$$H_{\text{fermi}} = \frac{1}{2} \sum_{\langle xy \rangle} \psi_x^\dagger \eta_{\langle xy \rangle} U_{\langle xy \rangle} \psi_y + m \lambda^p \sum_x \psi_x^\dagger \sigma_x \psi_x, \quad (22a)$$

$$F_{\text{fermi}} = \frac{1}{2} \sum_{\langle xy \rangle} \psi_x^\dagger \eta_{\langle xy \rangle} U_{\langle xy \rangle} \psi_y + m \lambda^p \sum_x \psi_x^\dagger \psi_x, \quad (22b)$$

where x labels lattice sites, $\langle xy \rangle$ denotes the link connecting nearest-neighbor sites x and y , $U_{\langle xy \rangle}$ is the $U(N)$ gauge connection on link $\langle xy \rangle$, and $\eta_{\langle xy \rangle}$ is a non-dynamical $U(1)$ fermion flavor connection. The fermion mass m may be scaled by a power p of gauge coupling, which may be set via the preprocessor symbol `MASSSCALE` (default 0). The lattice staggering factor σ_x equals $+1$ on the even sublattice and -1 on the odd sublattice. For further details see Ref. [3] and the discussion in appendix C.

Both canonicalized and non-canonicalized forms of the terms in the Hamiltonian (or free-energy) are stored in `global.info().Hterms`, since non-vacuum representation spectrum calculations need the non-canonicalized forms but for minimization and the vacuum representation spectrum the more compact canonicalized forms may be used. Because lattices with compactified dimensions of different sizes are supported, the code in `theorydefn` does not assume that all lattice directions are equivalent. After installing these theory definitions, the routine initializes `global.info().MMAlimit` to equal the number of initial observables of the stage, and clears `global.info().MMAlist`.

7.32 Version.h

Header file `Version.h` includes `Gordion.h` begins with a large comment block summarizing changes made with different versions. It then defines:

- `class Version`, used for program version numbers which are stored in `sys-info` and `vev-data` files. The class defines one non-static data member:
 - `ushort version`, recording a program version number,and several static data members:
 - `constexpr uchar majornum`, the running program's major version number;
 - `constexpr uchar minornum`, the running program's minor version number;
 - `constexpr bool r_is_f`, indicating if `sizeof (real) == sizeof (float)`.
 - `constexpr ushort myversion`, holding the running program's version number which combines `majornum`, `minornum` and `r_is_f`.

Class functions are:

- `ushoft rsize`, which return the size in bytes of a `real`;
- `ushort major`, which returns the major number portion of the class instance;
- `ushort minor`, which returns the minor number portion of the class instance;
- `ushort arith`, which returns least significant bit of the class instance, which indicates if, in the encoded version, `sizeof (real)` equals `sizeof(float)`;
- `newer`, which tests if the given instance version (from a save file) is newer than the running program's version number,
- `incompat`, which tests if the encoded real number size from the given instance is incompatible with the real number size of the running program.

A `friend ostream& operator<<` declaration is included for printing version numbers.

A Command summary

The keyword-driven command line interpreter recognizes the following commands, which may be terminated by either a semicolon (;) or end of line. In the following summary, *italic* indicates user input, brackets ([]) indicate optional arguments, and a vertical bar (|) separates alternative arguments. A plus mark⁺ after the command indicates the command is only applicable in Hamiltonian theories, while a double plus⁺⁺ indicates applicability only for Euclidean theories. Command words as well as subsequent argument words (other than explicit file, representation, or symmetry names) may be highly abbreviated; any initial matching sequence of one or more characters is sufficient to match the first consistent target word. (For example, a single `b` will match “build”, while `p obs *` will print all observables.)

*anything*

A comment, to be ignored.

?

Print a command summary.

add generator [(*order*)] [*coeff*] *op1* [[*coeff*] *op2*] ...

Deprecated form of **generator add** command.

Construct the specified coherence group generator, project it onto all defined irreducible representations, and add the results, when independent and non-vanishing, to the set of defined generators for each representation. One or more **Op**'s (operators in the coherence algebra) may be given, each optionally preceded by a numerical coefficient; this is interpreted as a weighted sum of the indicated **Op**'s. The generator order must either be specified explicitly, as an initial parenthesized integer, or else the first given **Op**, when canonicalized, must be a known observable. If multiple **Op**'s are given and (when canonicalized) are known observables, they must all have a common creation order.

build all *max-order*

Build observables (up to the specified maximum order) as well as the polynomial scripts for geodesics, gradient, curvature and (for Hamiltonian theories) the Lagrange bracket.

build curvature [*repname*]

build curvature all

Build the polynomial scripts used for evaluating the curvature of the Hamiltonian (or free energy) in the current representation. A specified representation becomes the current representation, while the final form builds the curvature scripts for all representations.

build geodesics

Build the polynomial scripts used for evaluating the geodesic equations.

build gradient

Build the polynomial scripts used for evaluating the gradient of the Hamiltonian (or free energy).

build lagrange⁺ [*repname*]

build lagrange⁺ **all**

Build the polynomial scripts used for evaluating the Lagrange bracket (for Hamiltonian theories) in the current representation. A specified representation becomes the current representation, while the final form builds the Lagrange bracket scripts for all representations.

build observables *max-order*

Build observables. All observables for the given minimization stage up to (and including) the specified maximum strong-coupling order are constructed.

call canon *obs*

call classify *obs*
call commute *op obs*
call compose *symm1 symm2*
call do_inner *Eloop-obs*
call noEbound *obs*
call transform *symmetry obs* Call the indicated routines with the given arguments. This functionality is present for debugging purposes.

do coupling-name *initial-value final-value increment*
 Evolve the specified coupling constant from the given initial value to the final value in steps of the given increment, performing Newton minimization at each step and writing results to the output data file.

do minimize
 Perform a sequence of minimization steps, terminating when the L_2 norm of the predicted displacement from the minimum falls below the minimization tolerance (or when the limit on the maximum number of minimization steps is reached).

do step
 Perform one Newton minimization step, evaluating the gradient and curvature, solving for the predicted minimum, and integrating the geodesic equations to move to the predicted minimum.

evaluate curvature [*]
 Numerically evaluate the T -even curvature matrix produced by double generator commutators, using active T -even generators in the currently active irreducible representation. Without an option *, the curvature eigenvalues are printed; with a * the full curvature matrix is first printed as well.

evaluate delta
 Evaluate, and print, the predicted location of the Hamiltonian or free energy minimum, equal to minus the inverse of the (vacuum representation) curvature times the gradient.

evaluate freeenergy⁺⁺
 Numerically evaluate the free energy and print the result.

evaluate geodesics *
evaluate geodesics *print-limit*
 Compute the variation of all observables (of the current minimization stage) in the direction defined by **delta**. Because this output vector can be very long, **evaluate geodesics** takes an argument specifying a limit on the number of observables whose derivatives will be printed, or else a * requesting printing of the entire list.

evaluate geostats
 (Re)evaluates and prints geodesic equation statistics, including the number of polynomial terms and their breakdown by monomial order. (Only relevant for debugging or verifying sys info file integrity.)

evaluate gradient

Numerically evaluate, and print, the gradient of the Hamiltonian (or free energy) produced by all defined vacuum representation generators.

evaluate hamiltonian⁺

Numerically evaluate the Hamiltonian and print the result.

evaluate lagrange⁺

Evaluate, and print, the Lagrange bracket matrix, produced by expectations of generator commutators, for generators in the currently active irreducible representation.

evaluate metric [*]

Numerically evaluate the T -odd curvature matrix produced by double generator commutators, using active T -odd generators in the currently active irreducible representation. Without an option $*$, the curvature eigenvalues are printed; with a $*$ the full curvature matrix is first printed as well.

evaluate spectrum⁺ [repname]**evaluate spectrum⁺ all**

Evaluate, and print, the small oscillation frequency (or mass) spectrum in the current representation derived from the eigenvalues of the generalized eigensystem of the curvature and Lagrange bracket. A specified representation becomes the current representation, while the final form evaluates the spectrum for all symmetry representations.

generator add [(order)] [coeff] op1 [[coeff] op2] ...

Construct the specified coherence group generator, project it onto all defined irreducible representations, and add the results, when independent and non-vanishing, to the set of defined generators for each representation. One or more **Op**'s (operators in the coherence algebra) may be given, each optionally preceded by a numerical coefficient; this is interpreted as a weighted sum of the indicated **Op**'s. The generator order must either be specified explicitly as an initial parenthesized integer, or else the first given **Op**, when canonicalized, must be a known observable. If multiple **Op**'s are given and (when canonicalized) are known observables, they must all have a common creation order.

generator activate (order)**generator activate gen1# [gen2#] ...**

First form activates all generators of the given order, in all representations. Second form activates the numbered generator(s) of the currently active representation.

generator suspend (order)**generator suspend gen1# [gen2#] ...**

First form suspends all generators of the given order, in all representations. Second form suspends the numbered generator(s) of the currently active representation.

help

Print a command summary.

load sys info-file

load [*data-set#*] *vev data-file*

Load information from a previously saved sys info or vev data file. The theory type of the running program must match that of the specified file or, for QCD theories, match the parent Yang-Mills theory. For vev data files, the number of observable expectation values must match either the total number of observables in the defined truncation, or the number of pure gauge observables. The default data set for vev data files is the final one in the file.

load *data-set#*

Load the specified data set from a currently open vev data file.

print baseobs

Print the list of basic observables (those used in the Hamiltonian or free energy).

print blablevels

Print the defined verbosity levels.

print cache

Print the canonicalization cache.

print couplings

Print the defined adjustable coupling constants.

print curvature *

print curvature *term#* [*gen1#* [*gen2#*]]

Print symbolic curvature expressions. The first form prints all curvature components. The second form allows one to print the curvature components of a specified term in the Hamiltonian or free energy (without any overall adjustable coupling constant) and, optionally, specify one or both indices of the generators in the active symmetry representation which act on the indicated term.

print freeenergy⁺⁺

Print the expression defining the free energy.

print fermiinit

Print the mapping from closed fermion bilinears to loops which is used for initialization of fermion bilinear expectation values.

print generator

print generator *

print generator *gen#*

Print generator(s). The first form prints an abbreviated form of all generators in the current representation. The second form prints the full form of all generators in the current representation. The final form prints the full form of a particular generator. For each generator, the output displays the current representation, the generator index prefixed with **g** for gauge or **f** for fermion generators and followed by a ***** suffix if the generator is *T*-odd, and then the actual generator definition.

print geodesics *

print geodesics *obs#* [*gen#*]

Print symbolic geodesic equations. The first form prints all geodesic equations. The second form allows one to specify a particular observable and, optionally, the (T -odd) vacuum representation generator which acts on the indicated observable.

print geostats

Prints geodesic equation statistics, including the number of polynomial terms and their breakdown by monomial order.

print gradient *

print gradient *term#* [*gen#*]

Print symbolic gradient expressions. The first form prints all gradient components. The second form allows one to print the gradient components of a specified term in the Hamiltonian or free energy (without any overall adjustable coupling constant) and, optionally, specify the index of the T -even vacuum representation generator which acts on the indicated term.

print hamiltonian⁺

Print the expression defining the Hamiltonian.

print lagrange *

print lagrange *gen1#* [*gen2#*]

Print symbolic Lagrange bracket expressions. The first form prints all Lagrange bracket components. The second form allows one to specify indices of one or both of the generators, in the active symmetry representation, whose commutator defines a Lagrange bracket component.

print mode⁺ *

print mode⁺ [*mode#*]

Print all, or a specified, small oscillation eigenmode from the most recently evaluated spectrum.

print observable *

print observable *obs1#* [*obs2#*]

print observable *obs*

print observable (*obstype*)

print observable (*corder,xorder*)

print observable (*corder,xorder,obstype*)

Print observables. The first form prints the entire list of constructed observables. The second form allows one to specify a single observable index, or a pair of indices indicating a range of observables. The third form allows one to input the printable form of an observable and determine if the given observable is present in the observable list. Canonicalization is not automatically performed on the input observable. The final forms allow one to print all observables of a specified type, with specified creation and expectation orders or, in the final form, specified orders and type. No spaces may be present within these parenthesized specifications.

print obsstats

Print an observable statistics table showing the break down of the number of constructed observables by type, creation order and expectation order.

print operator *

print operator *op1# [op2#]*

Print operators(s). The first form prints all defined operators (or “Op’s”) whose linear combinations, when symmetry projected, form generators. The second form allows on to specify a single operator index, or a pair of indices to indicate a range of operators.

print primary

Print the list of primary Op’s (those which cannot be produced by commutators of lower order Op’s).

print representation

print representation *

print representation *repname*

Print irreducible representation (irrep) projectors. The first form lists all defined irrep names, while the second form prints the full form of all irreps. The final form allows one to specify a particular representation by name.

print rkmethods

Print the names of available Runge-Kutta ODE integration methods, and report the currently selected method.

print spectrum⁺

Prints the most recently evaluated small oscillation spectrum.

print state

Print information on the theory class, lattice size, and global settings including minimization stage, active representation, ODE and minimization tolerances, ODE integration method, data file name, etc.

print stats

Print various operational statistics including the numbers of observables built, observable classifications, canonicalizations, approximations, symmetry transforms, etc., the number of geodesic equation terms, ODE integration steps, and canonicalization cache performance figures.

print symmetry *

print symmetry *symm-name*

Print symmetry transformations. The first form prints all symmetry transformations, while the second form allows one to specify a particular symmetry by name.

print symmsets

Print the various symmetry subsets used in observable canonicalization.

print sysindex

Print a table showing the contents of `global.sysindex`.

print vevindex

Print a list of data sets in a currently open vev data file.

quit

Exit the program.

read *filename*

Read command input from the specified file.

reset blab**reset maxthread****reset mintol****reset odetol****reset representation****reset rkmethod****reset stage****reset svdcutoff****reset sysfile****reset vevfile****reset MMAfile****reset MMAlist**

Reset the indicated global variables to their default initial values. See the various **set** commands for details.

reset numerics

Re-initialize observable expectation values for the current minimization stage. For gauge minimization, this zeros all pure gauge observable expectation values (except the identity). For fermi minimization, this zeros all fermion bilinear expectation values except those for which the fermion and conjugate fermion reside on the same site. Such bilinears are initialized to minus half the value of the corresponding Wilson loop.

save sys [*filename*]

Write to a binary “sys” file information on all defined operators, observables, generators, and the polynomial scripts defining gradient, curvature, Lagrange bracket and geodesic equations, plus build-related statistics counters. If no filename is specified, then a default file name with extension **.sys** is used which indicates the theory class, the maximum order of defined generators, the maximum strong coupling order of observables, and whether non-retained observables have been approximated. If the supplied filename is a path specification, then the directory portion automatically redefines the directory used for both **sys** and **vev** save files.

save vev [*filename*]

Append to a binary “vev” data file the expectation values of all defined observables and current values of all adjustable coupling constants. Multiple sets of such “vev” data may be present in the same **vev** file. If no filename is specified, then a default file name with extension **.vev** is used which indicates the theory class, the maximum order of defined generators, the maximum strong coupling order of observables used in minimization, and whether non-retained observables have been approximated. If the supplied filename is a path specification, then the directory portion automatically redefines the directory used for both **sys** and **vev** save files.

set *coupling-name coupling-value*
 Sets the value of the named adjustable coupling constant.

set approx true | false
 Specifies whether non-retained observables are to be approximated, when possible, by products of lower order retained observables (default **true**).

set autoToddgens true | false
 Specifies whether T -odd **Eloop** generators are to be automatically created from each pure loop generator by evaluating the commutator of the loop with the gauge kinetic energy, and whether fermion generators of opposite time-reversal symmetry are automatically generated by flipping the staggering of any added fermion generator. (default **true**).

set autosave false | true
 Specifies whether a sys info file should automatically be written during geodesic script construction, and a vev and coupling data set written to a vev data file after every successful minimization (default **true**).

set blab *source-file verbosity-level*
 Sets the verbosity level for particular source files. The specified source file name should omit the extension, is case sensitive, and cannot be further abbreviated. Current settings are reported by **print state**. Non-zero values are generally only relevant when debugging and can produce voluminous output.

set checkobs false | true
 Specifies whether extra validity checking is performed on every constructed observable (default **false**).

set dots false | true
 Specifies whether observables are printed with periods separating the printed forms of individual symbols (default **false**).

set echo false | true
 Specifies whether command input lines are echoed to the output stream (default **false**).

set gennorm false | true
 Specifies whether to normalize generators as described in section 4.7 (default **false**).

set geoswap false | true
 Specifies whether individual buckets of geodesic equation polynomials are to be swapped to and from disk as needed (default **false**). Setting **geoswap** true automatically also sets **autosave** true.

set massreinit true | false
 Specifies whether, in Euclidean theories, fermion bilinear expectation values are automatically reinitialized when the fermion mass is changed.

set minmax *integer*
 Specifies the maximum number of Newton iteration steps to allow during minimization (default 500).

set mintol *value*
 Specifies the minimization convergence criterion (default 10^{-9}).

set oknegeig **false** | **true**
 Specifies whether the presence of negative curvature eigenvalues (for sectors other than Euclidean fermions) will cause minimization to abort (default **false**).

set odetol *value*
 Specifies the ODE error tolerance (default 10^{-9}).

set representation *repname*
 Specifies the active symmetry representation.

set rkmethod *method-name*
 Specifies the Runge-Kutta ODE integration method.

set savedir *directory*
 Specifies the directory in which to write system information and expectation value data files (default `./save/THY`, with *THY* the value of the Makefile variable `THY`).

set stage **gauge** | **fermi**
 Specifies the minimization stage (initially set to **gauge**).

set svdcutoff *value*
 Sets the cutoff below which curvature singular values are truncated (default 10^{-7}).

set symcurv **true** | **false**
 Specifies whether the curvature is symmetrized when evaluating the small oscillation spectrum (default **true**).

set sysfile *filename*
 Specifies the name of the binary “sys” file in which to record information on observables, operators, generators, coupling constants, expectation values, and all polynomial scripts. If not set by the user, a default file name (with extension `.sys`) indicating the theory class, maximum order of generators, maximum strong coupling order of observables, and observable approximation setting is used. If the entered filename is a path specification, then the directory portion also redefines `savedir`.

set timing **true** | **false**
 Specifies whether to report the time taken by each command, when greater than 10 milliseconds (default **true**).

set vevappend **false** | **true**
 Specifies whether the vev data file, if pre-existing, is opened in append mode (default **false**).

set vevfile *filename*

Specifies the name of the binary “vev” file in which to save one or more sets of observable expectation values and adjustable coupling constant values. If not set by the user, a default file name (with extension `.vev`) indicating the theory class, maximum order of generators of generators used in minimization, maximum strong coupling order of observables, and observable approximation setting is used. If the entered filename is a path specification, then the directory portion also redefines `savedir`.

set MMAappend `false` | `true`

Specifies whether the output results file, if pre-existing, is opened in append mode (default `false`).

set MMAdir *directory*

Specifies the directory in which to write output results in Mathematica entry format (default `./MMA/THY`, with `THY` the value of the Makefile variable `THY`).

set MMAfile *filename*

Specifies the name of the output results file to which numerical results are written, in Mathematica entry format, after each minimization step of a `do coupling` command. If not set by the user, a default data file name (with extension `.m`) indicating the theory class, maximum order of generators used in minimization, maximum strong coupling order of observables, and observable approximation setting is used. If the entered filename is a path specification, then the directory portion also redefines `MMAdir`.

set MMAlimit *limit*

Specifies the number of initial observable expectation values, of the current minimization stage, to be included in output written to the numerical results file.

set MMAlist *obs₁* [*obs₂* [...]]

Specifies a further list of observables whose expectation values will be included in output written to the numerical results file.

test irreps

Test the defined irreducible representation projectors for orthogonality, normalization, and completeness. (Used for debugging.)

test jacobi

test jacobi *obs#*

test jacobi *op1 op2 obs#*

Test the validity of the Jacobi identity. The first form tests all triples of defined generators of the currently active representation. The second form tests the identity on the observable with the specified index and all pairs of defined generators (of the currently active representation). The final form uses user-specified coherence algebra `Op`'s and a specified observable. (Used for debugging.)

write *filename*

Redirect command output to the specified file. If the given filename is ‘-’, then subsequent output is returned to the initial output stream.

B Observable counts

Table 1 lists counts of the number of physically distinct (canonicalized) pure gauge observables, specifically Wilson loops with zero, one, or two electric field insertions, with given creation and expectation orders, for cubic lattice of dimension 2, 3 and 4. The following Table 2 lists analogous counts for fermion bilinears with zero or one electric field insertions.

classification order			2D lattice			3D lattice			4D lattice
s.-c.	creat	expct	Loop	Eloop	EEloop	Loop	Eloop	EEloop	Loop
0	0	0	1	0	0	1	0	0	1
4	0	4	0	1	1	0	1	1	0
4	2	2	1	1	0	1	1	0	1
≤ 4			2	2	1	2	2	1	2
8	2	6	0	0	3	0	0	3	0
8	4	4	3	7	27	5	13	45	5
≤ 8			5	9	31	7	15	49	7
12	4	8	0	0	8	0	0	16	0
12	6	6	9	58	338	34	247	1,349	42
12	8	4	8	44	160	29	184	574	36
≤ 12			22	111	537	70	446	1,988	85
16	6	10	0	0	58	0	0	277	0
16	8	8	56	647	5,589	541	7,444	59,283	895
16	10	6	83	1,369	12,298	1,036	16,143	139,347	1720
16	12	4	57	508	1,652	557	5,284	14,852	894
≤ 16			218	2,635	20,134	2,204	29,317	215,747	3,594
20	8	12	0	0	502	0	0	7,017	0
20	10	10	379	7,507	82,911	12,773	261,396	2,671,200	32,573
20	12	8	1,338	30,198	376,347	51,948	1,075,603	12,700,505	130,129
20	14	6	1,698	31,791	408,479	60,284	1,001,731	12,522,855	146,351
20	16	4	574	6,456	19,978	17,088	180,858	493,276	40,687
≤ 20			4,207	78,587	908,351	144,297	2,548,903	28,610,573	353,334
24	10	14	0	0	5,097	0	0	-	0
24	12	12	3,522	91,337	1,223,027	385,887	9,880,228		1,590,641
24	14	10	19,748	571,436	8,668,226	2,483,491	63,836,125		9,904,568
24	16	8	44,423	1,129,554	19,254,313	5,063,909	112,256,717		19,368,065
24	18	6	34,426	663,800	11,269,554	3,426,007	-		12,785,658
24	20	4	7,335	86,532	264,746	633,661	-		2,347,706
≤ 24			113,661	2,621,246	41,593,314	12,137,252	-		46,349,972
28	14	14	35,217			12,740,737			
28	16	12	287,656			116,225,033			
28	18	10	960,294			357,729,596			
28	20	8	1,319,969			424,541,869			
28	22	6	677,029			-			
28	24	4	99,891			-			
≤ 28			3,493,717			-			
32	16	16	377,729						
32	18	14	4,078,227						
32	20	12	18,692,958						
32	22	10	38,937,777						
32	24	8	35,547,316						
32	26	6	12,898,554						
32	28	4	1,426,384						
≤ 32			115,452,662						

Table 1: Counts of canonicalized Loop, Eloop, and EEloop gauge observables with specified creation (“creat”) and expectation (“expct”) orders on cubic lattices of dimension 2, 3 or 4. Interspersed with the rows for specific creation and expectation orders are partial sum rows showing the total number of observables of a given type with strong-coupling (“s.-c.”) orders no greater than the indicated limit. (In the 3D order 24 and 28 results, a ‘-’ indicates counts not currently evaluated.)

classification order			2D lattice		3D lattice		4D lattice
s.-c.	creat	expct	Fermion	Efermion	Fermion	Efermion	Fermion
0	0	0	1	-	1	-	1
2	1	1	1	1			
4	2	2	2	3			
6	3	3	5	14			
6	4	2	1	3			
8	4	4	12	51			
8	5	3	2	17			
8	6	2	1	7			
≤ 8			25	96			
10	5	5	42	252			
10	6	4	16	138			
10	7	3	10	74			
10	8	2	5	31			
≤ 10			98	591			
12	6	6	122	990			
12	7	5	51	636			
12	8	4	61	606			
12	9	3	26	206			
12	10	2	14	94			
≤ 12			372	3,123			
14	7	7	464	4,674			
14	8	6	273	3,745			
14	9	5	397	4,670			
14	10	4	253	2,869			
14	11	3	132	994			
14	12	2	68	424			
≤ 14			1,959	20,499			
16	8	8	1,497	18,218			
16	9	7	993	16,443			
16	10	6	1,823	26,204			
16	11	5	1,160	15,814			
16	12	4	1,076	12,268			
16	13	3	364	2,840			
16	14	2	203	1,337			
≤ 16			9,075	113,623			
18	9	9	5,837	82,315			
18	10	8	4,735	85,257			
18	11	7	10,440	170,305			
18	12	6	8,067	132,268			
18	13	5	8,435	108,808			
18	14	4	5,157	62,247			
18	15	3	1,842	13,936			
18	16	2	969	6,055			
≤ 18			54,557	774,814			

Table 2: Counts of canonicalized **Fermion** and **Efermion** bilinear observables with specified creation (“creat”) and expectation (“expct”) orders, on cubic lattices of dimension 2, 3 or 4. Interspersed with the rows for specific creation and expectation orders are partial sum rows showing the total number of observables of a given type with strong-coupling (“s.-c.”) orders no greater than the indicated limit.

C Symmetry transformations

This section describes, in detail, the action of various symmetries when acting on the fundamental fields and when acting on observables. Theories in Hamiltonian form will be discussed first, followed by Euclidean theories. This material expands on the presentation in Ref. [3] (and corrects a few misprints).

Lattice Hamiltonians under consideration are the sum of gauge and fermion pieces, $H = H_{\text{gauge}} + H_{\text{fermi}}$, with the gauge Hamiltonian having the standard Kogut-Susskind form

$$H_{\text{gauge}}/N = \frac{1}{4}\lambda \sum_{\ell} \text{tr}(E_{\ell}^2) + \lambda^{-1} \sum_p \text{tr}(2 - U[\partial p] - U[\partial p]^{\dagger}), \quad (23)$$

where the sums run over all positively oriented links (ℓ) and plaquettes (p). The fermion Hamiltonian for a single flavor takes the form

$$H_{\text{fermi}} = \frac{1}{2} \sum_{\langle xy \rangle} \psi_x^{\dagger} U_{\langle xy \rangle} \eta_{\langle xy \rangle} \psi_y + m \sum_x \sigma_x \psi_x^{\dagger} \psi_x, \quad (24)$$

with the first sum running over all nearest-neighbor pairs of sites (so that $\langle xy \rangle$ may be either a forward or backward directed link). The “staggering factor” σ_x appearing in the mass term is defined to equal +1 on the even sublattice (i.e., sites displaced from the origin by an even number of unit lattice translations), and -1 on the odd sublattice.²⁵

The “flavor connection” $\eta_{\langle xy \rangle}$ is a non-dynamical U(1) connection whose product around every plaquette equals minus one, $\eta[\partial p] \equiv -1$; this condition ensures recovery of the Dirac equation in the continuum limit. This flavor connection may be chosen to be pure imaginary, and this is assumed in the following.²⁶ Henceforth, the gauge connection and the flavor connection will always appear together in any fermion bilinear, so it is convenient to define the product

$$\hat{U}[\Gamma] \equiv U[\Gamma] \eta[\Gamma] \quad (25)$$

as the fermion connection for any lattice path Γ .

C.1 Cubic lattice symmetries

Let Θ denote a cubic lattice symmetry (some combination of discrete translations, rotations and/or reflections) which maps sites to sites, $x \rightarrow \theta x$ and links to links, $\alpha \rightarrow \theta \alpha$. The gauge fields (in a Hamiltonian formulation) transform as:

$$(U_{\alpha})_{ij} \xrightarrow{\Theta} (U_{\theta \alpha})_{ij}, \quad (E_{\alpha})_{ij} \xrightarrow{\Theta} (E_{\theta \alpha})_{ij}. \quad (26)$$

If a link α (or its transform) is backwards directed, so that $\alpha = \bar{\ell}$ for some forward directed link ℓ , then

$$(U_{\bar{\ell}})_{ij} \equiv (U_{\ell}^{\dagger})_{ij} = (U_{\ell})_{ji}^*, \quad (E_{\bar{\ell}})_{ij} \equiv -(U_{\ell} E_{\ell} U_{\ell})_{ij}. \quad (27)$$

²⁵The length of any compactified dimension must be even for the distinction between sublattices to be meaningful, and hence for the mass term to be well-defined.

²⁶A standard form for the flavor connection is $\eta_{\langle xy \rangle} = i \prod_{\nu < \mu} (-1)^{x_{\nu}}$ for a link $\langle xy \rangle$ running in the $+\mu$ direction starting at site x with coordinates $\{x_{\mu}\}$. But no such specific choice is ever needed.

Due to the $U(1)$ fermion flavor connection appearing in the Hamiltonian (or action) the transformation of fermion operators must include a flavor connection running from the initial site to the transformed site,

$$\psi_x \xrightarrow{\Theta} \eta[\Gamma_{x,\theta x}] \psi_{\theta x}. \quad (28)$$

The particular path $\Gamma_{x,\theta x}$ associated with a lattice symmetry Θ is constructed by designating (arbitrarily) some site o as the origin, choosing some path $\Gamma_{x,o}$ connecting site x to the origin, and choosing a path $\Gamma_{o,\theta o}$ connecting the origin to its transform under the lattice symmetry in question. Then the complete flavor connection path is given by

$$\Gamma_{x,\theta x} \equiv \Gamma_{x,o} \Gamma_{o,\theta o} (\overline{\theta\Gamma})_{\theta o,\theta x}, \quad (29)$$

where $(\overline{\theta\Gamma})_{\theta o,\theta x}$ denotes the transformed, reversed path which runs from θo to θx . This definition leaves the fermion hopping term invariant:

$$\begin{aligned} \frac{1}{2} \sum_{\langle xy \rangle} \psi_x^\dagger \eta_{\langle xy \rangle} U_{\langle xy \rangle} \psi_y &\rightarrow \frac{1}{2} \sum_{\langle \theta x \theta y \rangle} \psi_{\theta x}^\dagger \eta[\Gamma_{x,\theta x}]^\dagger \eta_{\langle xy \rangle} \eta[\Gamma_{y,\theta y}] U_{\langle \theta x \theta y \rangle} \psi_{\theta y} \\ &= \frac{1}{2} \sum_{\langle \theta x \theta y \rangle} \psi_{\theta x}^\dagger \eta_{\langle \theta x \theta y \rangle} U_{\langle \theta x \theta y \rangle} \psi_{\theta y} = \frac{1}{2} \sum_{\langle xy \rangle} \psi_x^\dagger \eta_{\langle xy \rangle} U_{\langle xy \rangle} \psi_y. \end{aligned} \quad (30)$$

To see this, note that

$$\begin{aligned} &\eta_{\langle \theta x \theta y \rangle}^\dagger \eta[\Gamma_{x,\theta x}]^\dagger \eta_{\langle xy \rangle} \eta[\Gamma_{y,\theta y}] \\ &= \eta_{\langle \theta x \theta y \rangle}^\dagger \eta[(\overline{\theta\Gamma})_{\theta o,\theta x}]^\dagger \eta[\Gamma_{o,\theta o}]^\dagger \eta[\Gamma_{x,o}]^\dagger \eta_{\langle xy \rangle} \eta[\Gamma_{y,o}] \eta[\Gamma_{o,\theta o}] \eta[(\overline{\theta\Gamma})_{\theta o,\theta y}] \\ &= (\eta[(\overline{\theta\Gamma})_{\theta o,\theta x}] \eta_{\langle \theta x \theta y \rangle} \eta[(\overline{\theta\Gamma})_{\theta o,\theta y}]^\dagger)^\dagger (\eta[\Gamma_{x,o}]^\dagger \eta_{\langle xy \rangle} \eta[\Gamma_{y,o}]) \\ &= 1. \end{aligned} \quad (31)$$

In the penultimate form, the second factor $(\eta[\Gamma_{x,o}]^\dagger \eta_{\langle xy \rangle} \eta[\Gamma_{y,o}])$ is the flavor connection around a closed path running from the origin o to x , then to y , and back to o , while the first factor $(\eta[(\overline{\theta\Gamma})_{\theta o,\theta x}] \eta_{\langle \theta x \theta y \rangle} \eta[(\overline{\theta\Gamma})_{\theta o,\theta y}]^\dagger)^\dagger$ is the inverse flavor connection of the symmetry transformed path running from θo to θx , then to θy , and back to θo . These two factors cancel because the two closed paths are geometrically similar; the flavor connection around every plaquette gives an identical factor of -1 , and any surface spanning the first loop will necessarily contain the same number of plaquettes as the symmetry transform of that surface spanning the second loop.

Physical observables transform in the natural manner:

$$\text{tr}(U[\Gamma]) \xrightarrow{\Theta} \text{tr}(U[\theta\Gamma]), \quad (32a)$$

$$\text{tr}(:E_\alpha U[\Gamma]:) \xrightarrow{\Theta} \text{tr}(:E_{\theta\alpha} U[\theta\Gamma]:), \quad (32b)$$

$$\text{tr}(:E_\alpha U[\Gamma] E_\beta U[\Gamma']:) \xrightarrow{\Theta} \text{tr}(:E_{\theta\alpha} U[\theta\Gamma] E_{\theta\beta} U[\theta\Gamma']:), \quad (32c)$$

$$:\psi_x^\dagger \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{\Theta} :\psi_{\theta x}^\dagger \hat{U}[(\theta\Gamma)_{\theta x,\theta y}] \psi_{\theta y}:, \quad (32d)$$

$$:\psi_x^\dagger \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{\Theta} :\psi_{\theta x}^\dagger \hat{U}[\theta\Gamma] E_{\theta\alpha} \hat{U}[\theta\Gamma'] \psi_{\theta y}:. \quad (32e)$$

Here, $:\mathcal{O}:$ denotes the normal ordered form of an observable containing electric field insertions and/or fermions. This ordering is defined to symmetrize the placement of electric field insertions at the beginning and end of the observable while also antisymmetrizing the ordering of fermions [3]. Specifically:

$$: U[\Gamma]_{ij}(E_\alpha)_{jk}U[\Gamma']_{kl} : \equiv \frac{1}{2}((E_\alpha)_{jk}U[\Gamma]_{ij}U[\Gamma']_{kl} + U[\Gamma]_{ij}U[\Gamma']_{kl}(E_\alpha)_{jk}), \quad (33a)$$

$$: U[\Gamma]_{ij}(E_\alpha)_{jk}U[\Gamma']_{kl}(E_\beta)_{lm}U[\Gamma'']_{mn} : \equiv \frac{1}{2}((E_\alpha)_{jk}U[\Gamma]_{ij}U[\Gamma']_{kl}U[\Gamma'']_{mn}(E_\beta)_{lm} + (E_\beta)_{lm}U[\Gamma]_{ij}U[\Gamma']_{kl}U[\Gamma'']_{mn}(E_\alpha)_{jk}), \quad (33b)$$

$$\begin{aligned} : (\psi_x^\dagger)_i G[\Gamma]_{ij}(\psi_y)_j : &\equiv \frac{1}{2}((\psi_x^\dagger)_i(\psi_y)_j - (\psi_y)_j(\psi_x^\dagger)_i) : G[\Gamma]_{ij} : \\ &= (\psi_x^\dagger)_i : G[\Gamma]_{ij} : (\psi_y)_j - \frac{1}{2}\delta_{xy} \text{tr}(: G[\Gamma] :), \end{aligned} \quad (33c)$$

where $G[\Gamma]$ denotes any string operator, possibly including electric field insertions.

The simple form (32) for the transformation for fermion bilinears is only possible because the connection $\hat{U}$ appearing in all fermionic operators is always understood to be the product (25) of the $U(N)$ gauge connection and the $U(1)$ flavor connection. Verifying the transformation (32d) involves the same argument as above, with the flavor connection around the closed path $(\Gamma_{x,o})^\dagger \Gamma_{xy} \Gamma_{y,o}$ canceling that around the transformed path $(\bar{\theta}\Gamma)_{\theta o, \theta x} (\theta\Gamma)_{\theta x, \theta y} (\bar{\theta}\Gamma)_{\theta o, \theta y}^\dagger$, and likewise for (32e).

A staggered sum over all lattice sites of a fermion bilinear transforms as

$$\sum_x : \psi_x^\dagger \sigma_x \hat{U}[\Gamma_{xy}] \psi_y : \xrightarrow{\Theta} \sigma_\theta \sum_{\theta x} : \psi_{\theta x}^\dagger \sigma_{\theta x} \hat{U}[\theta\Gamma_{\theta x, \theta y}] \psi_{\theta y} : , \quad (34)$$

with the overall factor of $\sigma_\theta \equiv \sigma_x \sigma_{\theta x}$ equaling -1 for lattice symmetries which interchange even and odd sublattices, and $+1$ otherwise. The fermion mass term is such a staggered sum, and hence only translations by even numbers of lattice spacings are symmetries of the theory when massive fermions are present.

As discussed in section 4.1, observables are represented as symbol strings, with distinct symbols representing each possible combination of gauge connections and electric field insertions residing on a single lattice link. This enables every lattice symmetry transformation to be implemented as simple substitution on symbols,

$$\mathbf{s}_1 \mathbf{s}_2 \cdots \mathbf{s}_n \xrightarrow{\Theta} \theta(\mathbf{s}_1) \theta(\mathbf{s}_2) \cdots \theta(\mathbf{s}_n), \quad (35)$$

combined with a possible minus sign associated with each symbol substitution to track the sign flips of reflected electric fields.

C.2 Charge conjugation

The action of charge conjugation on the gauge fields is given by

$$(U_\alpha)_{ij} \xrightarrow{C} (U_\alpha)_{ij}^* = (U_{\bar{\alpha}})_{ji}, \quad (E_\alpha)_{ij} \xrightarrow{C} (E_\alpha)_{ij}^* = -(E_\alpha)_{ji}. \quad (36)$$

The naive definition of the action of charge conjugation on fermions is

$$\psi_x \xrightarrow{C} \psi_x^\dagger c_x^*, \quad (37)$$

with c_x a charge conjugation phase which satisfies the condition

$$c_x = -\eta_{\langle xy \rangle}^* c_y \eta_{\langle yx \rangle}. \quad (38)$$

With the assumed pure imaginary form for the flavor connection, this charge conjugation phase may be set to unity.

The transformations (36) and (37) leave all terms in the Hamiltonian invariant, except the fermion mass term which changes sign. With multiple fermion flavors and a mass matrix which is unitarily equivalent to its negative, one may define charge conjugation to include a $\mathbb{Z}_2$ flavor exchange transformation and thereby produce a symmetry of the massive theory. But with a single staggered lattice fermion flavor, the only way to compensate the change in sign of the fermion mass is to combine the naive charge conjugation transformation with a lattice transformation which interchanges even and odd sublattices, such as a translation S by a single lattice spacing (in some direction),

$$C \equiv C_{\text{naive}} S. \quad (39)$$

This is the definition used.

When acting on physical observables, charge conjugation reverses the orientation of the gauge connection, as well as producing a sign change for each electric field insertion:

$$\text{tr}(U[\Gamma]) \xrightarrow{C} \text{tr}(U[\Gamma])^* = \text{tr}(U[\bar{\Gamma}]), \quad (40a)$$

$$\text{tr}(:E_\alpha U[\Gamma]:) \xrightarrow{C} \text{tr}(:E_\alpha U[\Gamma]:)^* = -\text{tr}(:E_\alpha U[\bar{\Gamma}]:), \quad (40b)$$

$$\text{tr}(:E_\alpha U[\Gamma] E_\beta U[\Gamma']:) \xrightarrow{C} \text{tr}(:E_\alpha U[\Gamma] E_\beta U[\Gamma']:)^* = \text{tr}(:E_\alpha U[\bar{\Gamma}'] E_\beta U[\bar{\Gamma}]:), \quad (40c)$$

where $\bar{\Gamma}$ denotes the reverse of a path Γ . Fermion bilinears summed over all lattice sites (i.e., momentum zero) transform as:

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{C} -(-1)^{|\Gamma|} \sum_y :\psi_y^\dagger \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \quad (41a)$$

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{C} (-1)^{|\Gamma|} \sum_y :\psi_y^\dagger \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:. \quad (41b)$$

Note that charge conjugation preserves the normal ordered forms of observables. The factor of $-(-1)^{|\Gamma|}$ in (41a) arises from the reversal of the implicit flavor connection which (with a pure imaginary flavor connection) gives a -1 for each link in the path Γ , plus an additional minus sign from the anticommutation of the fermion fields, while (41b) has one further minus sign from the electric field transformation. For staggered (i.e., momentum π) fermion bilinears, the relations analogous to (41a) and (41b) read:

$$\begin{aligned} \sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma_{xy}] \psi_y: &\xrightarrow{C} (-1)^{|\Gamma|} \sum_x :\psi_y^\dagger \hat{U}[\bar{\Gamma}_{yx}] \sigma_x \psi_x: \\ &= \sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \end{aligned} \quad (42a)$$

$$\sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{C} - \sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:, \quad (42b)$$

where the change by a factor of $-(-1)^{|\Gamma|}$ in these expressions relative to (41) comes from rewriting $\sigma_x = (-1)^{|\Gamma|} \sigma_y$, together with the additional minus sign from the unit shift in the definition of C .

With the symbol encoding described in Sec. 4.1, in which $\mathbf{f}$ and $\mathbf{g}$ represent unstaggered and staggered fermion fields, respectively, a charge conjugation transformation on observables is implemented by the symbol substitutions:

$$\mathbf{x} \leftrightarrow \mathbf{X}, \quad \mathbf{A} \leftrightarrow -\mathbf{A}, \quad \mathbf{f} \leftrightarrow \mathbf{G}, \quad \mathbf{F} \leftrightarrow -\mathbf{g}, \quad (43)$$

etc., combined with reversal of the order of symbols,

$$\mathbf{s}_1 \mathbf{s}_2 \cdots \mathbf{s}_n \xrightarrow{C} C(\mathbf{s}_n) \cdots C(\mathbf{s}_2) C(\mathbf{s}_1). \quad (44)$$

The interchange of unstaggered and staggered fermions in the symbol substitutions (43) serves to reproduce the factors of $(-1)^{|\Gamma|}$ in (41), together with their absence in (42).

C.3 Time reversal

Time reversal (in the Hamiltonian formulation) is an anti-unitary transformation which acts on the gauge fields as:

$$(U_\alpha)_{ij} \xrightarrow{T} (U_{\bar{\alpha}})_{ji}, \quad (E_\alpha)_{ij} \xrightarrow{T} -(E_\alpha)_{ji}. \quad (45a)$$

while the naive definition of time reversal on fermions is

$$(\psi_x)_i \xrightarrow{T} (\psi_x^\dagger)_i \sigma_x. \quad (45b)$$

This transformation leaves the fermion kinetic term invariant,

$$\sum_{\langle xy \rangle} \psi_x^\dagger \eta_{\langle xy \rangle} U_{\langle xy \rangle} \psi_y \xrightarrow{T} \sum_{\langle xy \rangle} \psi_x \sigma_x \eta_{\langle xy \rangle}^* U_{\langle xy \rangle}^* \sigma_y \psi_y^\dagger = \sum_{\langle yx \rangle} \psi_y^\dagger \eta_{\langle yx \rangle} U_{\langle yx \rangle} \psi_x. \quad (46)$$

The product of staggering factors cancels the minus sign arising from anticommuting the fermions, while the anti-unitary nature of the transformation causes the non-dynamical flavor connection in this (and any other fermion bilinear) to be complex-conjugated, which is equivalent to reversing its direction. The pure gauge terms in the Hamiltonian are likewise invariant.

The transformation (45b) acts on the fermion mass term as

$$m \sum_x \psi_x^\dagger \sigma_x \psi_x \xrightarrow{T} -m^* \sum_x \psi_x^\dagger \sigma_x \psi_x = -m \sum_x \psi_x^\dagger \sigma_x \psi_x. \quad (47)$$

Since the fermion mass must be real (for a Hermitian Hamiltonian), the sign of the mass term flips. Therefore, just as for complex conjugation, in the presence of fermions a time reversal transformation will be combined with a single lattice spacing translation,

$$T \equiv T_{\text{naive}} S, \quad (48)$$

to produce a valid symmetry.

When acting on our set of physical observables, the action of time reversal is identical to the effect of charge conjugation:

$$\text{tr}(U[\Gamma]) \xrightarrow{T} \text{tr}(U[\bar{\Gamma}]), \quad (49a)$$

$$\text{tr}(:E_\alpha U[\Gamma]:) \xrightarrow{T} -\text{tr}(:E_\alpha U[\bar{\Gamma}]:), \quad (49b)$$

$$\text{tr}(:E_\alpha U[\Gamma] E_\beta U[\Gamma']:) \xrightarrow{T} \text{tr}(:E_\alpha U[\bar{\Gamma}'] E_\beta U[\bar{\Gamma}]:). \quad (49c)$$

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{T} -(-1)^{|\Gamma|} \sum_x :\psi_y^\dagger \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \quad (49d)$$

$$\sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{T} \sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \quad (49e)$$

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{T} (-1)^{|\Gamma|} \sum_y :\psi_y^\dagger \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:, \quad (49f)$$

$$\sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{T} -\sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:. \quad (49g)$$

C.4 Hermitian conjugation

While not a symmetry per se, it is useful to compare the above transformations with the action of Hermitian conjugation on observables:

$$\text{tr}(U[\Gamma])^\dagger \xrightarrow{\dagger} \text{tr}(U[\bar{\Gamma}]), \quad (50a)$$

$$\text{tr}(:E_\alpha U[\Gamma]:) \xrightarrow{\dagger} \text{tr}(:E_\alpha U[\bar{\Gamma}]:), \quad (50b)$$

$$\text{tr}(:E_\alpha U[\Gamma] E_\beta U[\Gamma']:) \xrightarrow{\dagger} \text{tr}(:E_\alpha U[\bar{\Gamma}'] E_\beta U[\bar{\Gamma}]:). \quad (50c)$$

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{\dagger} \sum_y :\psi_y^\dagger \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \quad (50d)$$

$$\sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma_{xy}] \psi_y: \xrightarrow{\dagger} (-1)^{|\Gamma|} \sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x:, \quad (50e)$$

$$\sum_x :\psi_x^\dagger \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{\dagger} \sum_y :\psi_y^\dagger \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:, \quad (50f)$$

$$\sum_x :\psi_x^\dagger \sigma_x \hat{U}[\Gamma] E_\alpha \hat{U}[\Gamma'] \psi_y: \xrightarrow{\dagger} (-1)^{|\Gamma|} \sum_y :\psi_y^\dagger \sigma_y \hat{U}[\bar{\Gamma}'] E_\alpha \hat{U}[\bar{\Gamma}] \psi_x:. \quad (50g)$$

The factors of $(-1)^{|\Gamma|}$ in (50e) and (50g) result from moving the staggering factor to the other end of the bilinear.

These Hermitian conjugation transformations differ from the corresponding actions of time reversal, or charge conjugation, by at most a minus sign. Consequently, the expectation value of any observable $\mathcal{O}$ for which charge conjugation equals either plus or minus Hermitian conjugation, $C^\dagger \mathcal{O} C = \pm \mathcal{O}^\dagger$, in any state $|u\rangle$ which is charge conjugation

invariant must be either purely real or purely imaginary, since

$$\langle u|\mathcal{O}|u\rangle^* = \langle u|\mathcal{O}^\dagger|u\rangle = \pm\langle u|C^\dagger\mathcal{O}C|u\rangle = \pm\langle u|\mathcal{O}|u\rangle. \quad (51)$$

Comparing (49) to (50) one sees that observables with real expectation values (in any C -invariant state) are:

- Wilson loops;
- Wilson loops with two electric field insertions;
- Odd-length, unstaggered fermion bilinears;
- Even-length, staggered fermion bilinears;
- Even-length, unstaggered fermion bilinears with one electric field insertion;²⁷
- Odd-length, staggered fermion bilinears with one electric field insertion.

while observables with imaginary expectation values are:

- Wilson loops with one electric field insertion;
- Even-length, unstaggered fermion bilinears;
- Odd-length, staggered fermion bilinears;
- Odd-length, unstaggered fermion bilinears with one electric field insertion;
- Even-length, staggered fermion bilinears with one electric field insertion.

In Hamiltonian theories, the relation between time reversal and Hermitian conjugation implies that in any time-reversal invariant state the expectation values of observables $\mathcal{O}$ for which $T\mathcal{O}T^{-1} = -\mathcal{O}^\dagger$ (which are the same as observables for which $C^\dagger\mathcal{O}C = -\mathcal{O}^\dagger$) actually vanishes in any T -invariant state. This follows from combining this relation with the defining relation for anti-unitary transformations, $\langle T\Psi|T\chi\rangle = \langle\Psi|\chi\rangle^*$, and the invariance of the state $|u\rangle = T|u\rangle$, to obtain:

$$\langle u|\mathcal{O}|u\rangle = \langle Tu|\mathcal{O}|Tu\rangle = \langle Tu|\mathcal{O}^\dagger|Tu\rangle^* = -\langle Tu|T\mathcal{O}T^{-1}|Tu\rangle^* = -\langle u|\mathcal{O}|u\rangle. \quad (52)$$

The first equality is T -invariance of $|u\rangle$, the second is the definition of Hermitian conjugation, the third is the assumed property of $\mathcal{O}$, and the last is the defining relation of time-reversal with $\langle\Psi| = \langle u|$ and $|\chi\rangle = \mathcal{O}T^{-1}|Tu\rangle = \mathcal{O}|u\rangle$. Consequently, in any C - and T -invariant state, observables for which charge conjugation would imply an imaginary expectation value actually have vanishing expectation values.

²⁷“Length” means the number of links in the bilinear; the electric field insertion does not count.

C.5 Implications

The above symmetry relations have numerous implications regarding the structure of the large- N phase space and associated dynamics. These include:

- Time reversal invariance implies that the $N = \infty$ dynamics has a natural configuration space corresponding to the subspace of time-reversal invariant points within the $N = \infty$ phase space, with these points arising from the large- N limit of (equivalence classes of) T -invariant coherent states.
- Any function on phase space may be decomposed into components which are either even or odd under time reversal. In particular, phase space coordinates may be partitioned into those which are even under time reversal, i.e., configuration space coordinates, or those which are odd, i.e., classical momenta.
- Classical coordinates correspond to expectation values of the operators with real expectation values (in C -invariant states), while classical momenta correspond to expectation values of the operators for which charge conjugation implies imaginary expectation values, but whose expectation values actually vanish on the classical configuration space due to time reversal invariance.
- Elements of the coherence algebra, which generate deformations in phase space, may be written as a linear combination of basis generators, $\mathbf{g} = \sum_a c^a \hat{e}_a$, with the coefficients $\{c^a\}$ serving as Riemann normal coordinates in the neighborhood of any point in phase space. The generators $\hat{e}_a$ may be partitioned into those which are time-reversal even, and those which are time-reversal odd. Time-reversal even generators are anti-Hermitian linear combinations of loops with a single electric field insertion, even-length unstaggered fermion bilinears, and odd-length staggered fermion bilinears; the coefficients $\{c^a\}$ of these generators serve as configuration space coordinates. Correspondingly, time-reversal odd generators are anti-Hermitian linear combinations of Wilson loops, odd-length unstaggered fermion bilinears, and even-length staggered fermion bilinears; the coefficients $\{c^a\}$ of these generators serve as classical momenta.
- In order to be properly anti-Hermitian, coherence group generators composed of linear combinations of operators for which $C^\dagger \mathcal{O} C = \mathcal{O}^\dagger$, namely Wilson loops, odd-length unstaggered bilinears, and even-length staggered bilinears, must have imaginary coefficients when projected onto C -even symmetry representations, and real coefficient when projected onto C -odd representations. Because time reversal is anti-unitary, both types of such generators are time-reversal odd.

The opposite situation applies to generators composed of linear combinations of operators for which $C^\dagger \mathcal{O} C = -\mathcal{O}^\dagger$, namely Wilson loops with an electric field insertion, even-length unstaggered bilinears, and odd-length staggered bilinears. Such generators must have real coefficients when projected onto C -even symmetry representations, and imaginary coefficients when projected onto C -odd representations. Both types of such generators are time-reversal even.

T	C	gauge generators	fermion generators
+	+	$\text{tr}(EU[\Gamma] - EU[\bar{\Gamma}])$	$(\psi^\dagger \sigma \hat{U}[\Gamma_o] \psi + \psi^\dagger \sigma \hat{U}[\bar{\Gamma}_o] \psi) \quad (\psi^\dagger \hat{U}[\Gamma_e] \psi - \psi^\dagger \hat{U}[\bar{\Gamma}_e] \psi)$
-	+	$i \text{tr}(U[\Gamma] + U[\bar{\Gamma}])$	$i(\psi^\dagger \hat{U}[\Gamma_o] \psi + \psi^\dagger \hat{U}[\bar{\Gamma}_o] \psi) \quad i(\psi^\dagger \sigma \hat{U}[\Gamma_e] \psi + \psi^\dagger \sigma \hat{U}[\bar{\Gamma}_e] \psi)$
+	-	$i \text{tr}(EU[\Gamma] + EU[\bar{\Gamma}])$	$i(\psi^\dagger \sigma \hat{U}[\Gamma_o] \psi - \psi^\dagger \sigma \hat{U}[\bar{\Gamma}_o] \psi) \quad i(\psi^\dagger \hat{U}[\Gamma_e] \psi + \psi^\dagger \hat{U}[\bar{\Gamma}_e] \psi)$
-	-	$\text{tr}(U[\Gamma] - U[\bar{\Gamma}])$	$(\psi^\dagger \hat{U}[\Gamma_o] \psi - \psi^\dagger \hat{U}[\bar{\Gamma}_o] \psi) \quad (\psi^\dagger \sigma \hat{U}[\Gamma_e] \psi - \psi^\dagger \sigma \hat{U}[\bar{\Gamma}_e] \psi)$

Table 3: Schematic forms of anti-Hermitian coherence group generators, classified according to their behavior under charge conjugation and time reversal symmetries. Γ_e denotes an even-length lattice path and Γ_o an odd-length path, overbars indicate path reversal, and σ is the lattice staggering factor at the location of the preceding conjugate fermion field. Coefficients of the listed time-reversal even generators serve as Riemann normal coordinates within the classical configuration space of the fermion large- N phase space, while coefficients of time-reversal odd generators serve as conjugate momenta. Only the first category of fully symmetric generators need be used for minimization.

These various categories of coherence group generators are shown schematically in Table 3, classified according to their behavior under C and T transformations. Of course, each such category can be further projected onto particular irreducible representations of the lattice cubic symmetry point group.

C.6 Euclidean theory symmetries

Lattice actions for Euclidean gauge theories under consideration are the sum of gauge and fermion contributions, $A = A_{\text{gauge}} + A_{\text{fermi}}$, with the gauge action having a standard Wilson form,

$$A_{\text{gauge}}/N = \lambda^{-1} \sum_p \text{tr}(2 - U[\partial p] - U[\partial p]^\dagger). \quad (53)$$

Obviously, this gauge action is real provided the gauge coupling λ is real, which is assumed. The most economical discretization for a single (massive) fermion flavor has just N pairs (for $U(N)$ fundamentals) of independent Grassmann generators, $\{\chi_x^i, \bar{\chi}_x^i\}$, residing on each lattice site. The fermion lattice action may be defined in a manner completely analogous to the Hamiltonian form (24),

$$A_{\text{fermi}} = \frac{1}{2} \sum_{\langle xy \rangle} \bar{\chi}_x U_{\langle xy \rangle} \eta_{\langle xy \rangle} \chi_y + m \sum_x \sigma_x \bar{\chi}_x \chi_x, \quad (54)$$

where, once again, the $U(1)$ flavor connection $\eta_{\langle xy \rangle}$ is defined to be pure imaginary and to have a product of minus one around every plaquette, $\eta[\partial p] = -1$. More abstractly, this fermion action is a quadratic form, $A_{\text{fermi}} = (\bar{\chi}, D \chi)$, where D is a discretized Dirac operator with non-vanishing matrix elements only for coincident or nearest-neighbor sites:

$$(D)_{xy}^{ij} \equiv \frac{1}{2} U_{\langle xy \rangle}^{ij} \eta_{\langle xy \rangle} + m \sigma_x \delta_{xy} \delta^{ij}. \quad (55)$$

Because both the $U(N)$ gauge connection and the flavor connection are unitary connections (i.e., $U_{\langle xy \rangle}^\dagger = U_{\langle yx \rangle}$ and $\eta_{\langle xy \rangle}^* = \eta_{\langle yx \rangle}$) this Dirac operator is Hermitian, provided the fermion mass m is real, which is assumed. Hermiticity of D ensures reality of the fermionic partition function,

$$Z_{\text{fermi}} \equiv \int d\bar{\chi} d\chi e^{-A_{\text{fermi}}} = \det(D). \quad (56)$$

This may be equivalently expressed as invariance of the action A_{fermi} under a complex conjugation transformation (an involutive automorphism) which performs ordinary complex conjugation on all commuting numbers while simultaneously interchanging Grassmann variables,

$$\chi_x \xrightarrow{*} \bar{\chi}_x, \quad \bar{\chi}_x \xrightarrow{*} \chi_x, \quad (57)$$

and flipping the order of all products of Grassmann variables. Any statistical ensemble, such as the equilibrium ensemble $\rho_{\text{eq}} \equiv Z^{-1} e^{-A}$ or those generated by the Euclidean large- N coherence group acting on an initial uncorrelated ensemble, must satisfy this same reality condition,

$$\rho(\{U_\ell\}, \{\bar{\chi}_x\}, \{\chi_x\})^* = \rho(\{U_\ell\}, \{\bar{\chi}_x\}, \{\chi_x\}). \quad (58)$$

The relevant physical observables in Euclidean theories are just Wilson loops and fermion bilinears. The reality condition (58) implies that expectation values of these observables satisfy the basic Hermiticity relations:

$$\langle \text{tr}(U[\Gamma]) \rangle^* = \langle \text{tr}(U[\bar{\Gamma}]) \rangle, \quad \langle \bar{\chi}_x \hat{U}[\Gamma_{xy}] \chi_y \rangle^* = \langle \bar{\chi}_y \hat{U}[\bar{\Gamma}_{yx}] \chi_x \rangle, \quad (59)$$

which are identical in form to the corresponding relations in Hamiltonian theories.

Ref. [3] chose to discuss Euclidean theories with a fermion action of the form (54), as this choice makes the action of symmetry transformations as similar as possible between Hamiltonian and Euclidean formulations and also makes the reality condition (57) and its consequences (59) as simple as possible. However, the presence of a lattice staggering factor in the mass term is an inconvenience which, in particular, obscures the fact that the Euclidean theory (54) actually possess full lattice translation invariance, including translations by an odd number of lattice spacings (unlike the Hamiltonian theory).

The independence of the Grassmann generators $\bar{\chi}$ and χ in the Euclidean formulation allows one to make a change of variables involving independent phase rotations of these variables, and this enables one to make a field redefinition which will simplify the form of lattice symmetry transformations. Let

$$\epsilon_x \equiv \begin{cases} 1, & x \text{ even;} \\ i, & x \text{ odd,} \end{cases} \quad (60)$$

denote a square root of the staggering factor σ_x , and define

$$\bar{\chi}_x \equiv \bar{\psi}_x \epsilon_x^*, \quad \chi_x \equiv i \sigma_x \epsilon_x \psi_x. \quad (61)$$

This field redefinition converts the lattice staggering factor in the fermion mass term into a factor of i , while leaving the hopping term unchanged, so the fermion action becomes

$$\tilde{A}_{\text{fermi}} = \frac{1}{2} \sum_{\langle xy \rangle} \bar{\psi}_x U_{\langle xy \rangle} \eta_{\langle xy \rangle} \psi_y + i m \sum_x \bar{\psi}_x \psi_x. \quad (62)$$

The key benefit of this form for the fermion action is that it obviates the need for separate representations of staggered and unstaggered versions of fermion bilinears. The fermion action (62) agrees with the form used in most other literature on Euclidean staggered lattice fermions (see, e.g., Ref. [27]), except for an overall factor of i in the action (62) (hidden in the imaginary $\eta_{\langle xy \rangle}$ in the hopping term and explicit in the mass term). The presence, or absence, of this overall factor of i is physically irrelevant, but its inclusion makes $\eta_{\langle xy \rangle}$ a conventional unitary Abelian connection which is convenient for the following symmetry analysis.

The fermion action (62) remains a quadratic form, $\tilde{A}_{\text{fermi}} = (\bar{\psi}, \tilde{D} \psi)$, but with a transformed Dirac operator $\tilde{D} \equiv -i\epsilon^* D \epsilon$ now obeying a Hermiticity condition of the form $\tilde{D}^\dagger = -\sigma \tilde{D} \sigma$. This Hermiticity condition may also be expressed as invariance of the action (62) under a complex conjugation transformation in which the redefined Grassmann fields transform as

$$\psi_x \xrightarrow{*} i \bar{\psi}_x \sigma_x, \quad \bar{\psi}_x \xrightarrow{*} i \sigma_x \psi_x, \quad (63)$$

(while also inverting the order of Grassmann products). In terms of the redefined Grassmann fields, the reality condition (59) on fermion bilinear expectation values becomes

$$\langle \bar{\psi}_x \hat{U}[\Gamma_{xy}] \psi_y \rangle^* = -(-1)^{|\Gamma|} \langle \bar{\psi}_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x \rangle. \quad (64)$$

Before proceeding to an explicit discussion of the implications of different symmetries, recall that coherence group generators for Euclidean theories contain first order derivatives acting on the fundamental fields. These are defined as

$$E_{\ell,ij} \equiv \frac{1}{N} U_{\ell,ik} \frac{\partial}{\partial U_{\ell,jk}}, \quad \bar{v}_{x,i} \equiv \frac{\partial}{\partial \psi_{x,i}}, \quad v_{x,i} \equiv \frac{\partial}{\partial \bar{\psi}_{x,i}}, \quad (65)$$

with the Grassmann derivatives $v_{x,i}$ and $\bar{v}_{x,i}$ satisfying the basic anticommutation relations $\{\bar{v}_{x,i}, \psi_{y,j}\} = \delta_{xy} \delta_{ij}$ and $\{v_{x,i}, \bar{\psi}_{y,j}\} = \delta_{xy} \delta_{ij}$, with other anticommutators vanishing. Invariance of these relations under a complex conjugation transformation (63) implies that the Grassmann derivatives behave as

$$v_x \xrightarrow{*} -i \bar{v}_x \sigma_x, \quad \bar{v}_x \xrightarrow{*} -i \sigma_x v_x \quad (66)$$

under complex conjugation.

Cubic lattice symmetry transformations act identically to the Hamiltonian formulation transformations (26) and (28), with the understanding that link derivatives $\{E_\ell\}$ transform in the same manner as electric field operators of the Hamiltonian theory (and hence are denoted identically), while the conjugate fermion $\bar{\psi}_x$ transforms as does the Hermitian conjugate $\psi_x^\dagger$ of the Hamiltonian theory.²⁸ The Grassmann derivatives $\bar{v}_x$ and v_x transform in the same manner as do $\bar{\psi}_x$ and ψ_x , respectively.

²⁸Time-reversal, if one so designates some lattice direction, is just part of the lattice symmetry group.

Since the fermion mass term no longer contains a lattice staggering factor, both gauge (53) and fermion (62) actions in the Euclidean formulation are invariant under odd lattice translations as well as even.²⁹

The action of charge conjugation on gauge fields is identical to the Hamiltonian form (36), while the action on fermion fields and their derivatives may be defined as

$$\psi_x \xrightarrow{C} \bar{\psi}_x \sigma_x, \quad \bar{\psi}_x \xrightarrow{C} -\sigma_x \psi_x, \quad v_x \xrightarrow{C} -\bar{v}_x \sigma_x, \quad \bar{v}_x \xrightarrow{C} \sigma_x v_x. \quad (67)$$

In the mass term, the explicit minus sign from the transformation cancels the minus sign from anticommuting the fermion fields back to their original order. In the hopping term, the factor of $-\sigma_x \sigma_y$ equals one since x and y are nearest-neighbor sites, while the minus sign from anticommuting the fermion fields cancels the minus sign from reversing the flavor connection. The transformations for the Grassmann derivatives v_x and $\bar{v}_x$ follow from their definition as derivatives.

Acting on Wilson loops or fermion bilinears, charge conjugation merely reverses the paths:

$$\text{tr}(U[\Gamma]) \xrightarrow{C} \text{tr}(U[\bar{\Gamma}]), \quad \bar{\psi}_x \hat{U}[\Gamma_{xy}] \psi_y \xrightarrow{C} \bar{\psi}_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x. \quad (68)$$

Just as for the hopping term of the action, the product of lattice staggering factors from the transformation (67) cancels the minus signs from reversal of the flavor connection. Charge conjugation acts on fermion bilinear coherence group generators as

$$\bar{v}_x \hat{U}[\Gamma_{xy}] \psi_y \xrightarrow{C} -\bar{\psi}_y \hat{U}[\bar{\Gamma}_{yx}] v_x, \quad \bar{\psi}_x \hat{U}[\Gamma_{xy}] v_y \xrightarrow{C} -\bar{v}_y \hat{U}[\bar{\Gamma}_{yx}] \psi_x. \quad (69)$$

With the symbols $\mathbf{g}$ and $\mathbf{G}$ now representing the Grassmann derivatives v and $\bar{v}$, respectively, a charge conjugation transformation acting on Euclidean theory observables or generators is implemented by the symbol substitutions:

$$\mathbf{x} \leftrightarrow \mathbf{X}, \quad \mathbf{A} \leftrightarrow -\mathbf{A}, \quad \mathbf{f} \leftrightarrow \mathbf{F}, \quad \mathbf{F} \leftrightarrow \mathbf{f}, \quad \mathbf{g} \leftrightarrow -\mathbf{G}, \quad \mathbf{G} \leftrightarrow -\mathbf{g}, \quad (70)$$

etc., along with reversal of the order of symbols.

Combining the charge conjugation relations (68) with the complex conjugation relations (59) and (64) shows that in any C -invariant state:

- $\langle \text{tr}(U[\Gamma]) \rangle$ is real;
- $\langle \bar{\psi} \hat{U}[\Gamma_e] \psi \rangle$ is imaginary;
- $\langle \bar{\psi} \hat{U}[\Gamma_o] \psi \rangle$ is real;

with Γ_e and Γ_o denoting arbitrary paths of even or odd length, respectively. So the “price” for making the field redefinition (61) is that all even length bilinears now have pure imaginary expectation values, while odd length bilinears have real expectations. The compensating benefit of using the form (62) for the fermion action is that physical observables now have very simple behavior under all lattice symmetry transformations as well as charge conjugation.

²⁹Note, however, that to define a flavor connection satisfying $\eta[\partial p] = -1$ on every plaquette, it is necessary that the length of any compactified lattice direction be even.

C	gauge generators	fermion generators
+	$\text{tr}(EU[\Gamma] - EU[\bar{\Gamma}])$	$(\bar{v} \hat{U}[\Gamma_e] \psi - \bar{\psi} \hat{U}[\bar{\Gamma}_e] v) \quad i(\bar{v} \hat{U}[\Gamma_o] \psi - \bar{\psi} \hat{U}[\bar{\Gamma}_o] v)$
-	$i \text{tr}(EU[\Gamma] + EU[\bar{\Gamma}])$	$i(\bar{v} \hat{U}[\Gamma_e] \psi + \bar{\psi} \hat{U}[\bar{\Gamma}_e] v) \quad (\bar{v} \hat{U}[\Gamma_o] \psi + \bar{\psi} \hat{U}[\bar{\Gamma}_o] v)$

Table 4: Schematic forms of coherence group generators for Euclidean gauge theories, classified according to their behavior under charge conjugation. Γ_e denotes an even-length lattice path and Γ_o an odd-length path, while overbars indicate path reversal.

The fact that even length bilinears have imaginary expectation values when using the form (62) of the Euclidean fermion action is a relatively minor nuisance. There is no need for employing complex arithmetic in any of the steps of the variational algorithm, as every even length fermion bilinear appearing in the free energy, its gradient or curvature, or the geodesic equations will be accompanied by a pure imaginary coefficient.

Table 4 shows the schematic forms of coherence group generators for Euclidean gauge theories when projected onto charge conjugation eigenstates. The overall factors of i in odd-length C -even and even-length C -odd generators are dictated by the requirement that deformations of the statistical ensemble generated by these generators, when multiplied by real coefficients, preserve the reality condition for the statistical measure.

D Prior work errata

D.1 CSVA II [3]

- Eq. (4.3) has a misprint in the $:E^\alpha V^\Gamma$: transformation; the electric field insertion in the result should be E^α , not $E^{\bar{\alpha}}$. The same misprint is present in the subsequent $:E^\alpha V^\Gamma$: transformations shown in Eqs. (4.4) and (4.5).
- Eq. (4.10e) should have an anticommutator, not commutator, in the intermediate expression.
- In Eq. (10.4), the power of λ should be -2 , not -1 , in both the step function and within the square root.
- Except for Loop observables below strong-coupling order 24, the various counts of observables listed in tables 1–3 in section 5.4 do not precisely agree with the new results in appendix B. See the discussion in footnote 11.

D.2 CSVA I [2]

- In Eq. (7.1), the expression for the one-plaquette Hamiltonian (in a form consistent with subsequent results) should have $\text{tr}(2 - \hat{x} - \hat{x}^\dagger)$ instead of $\text{tr}(1 - \hat{x} - \hat{x}^\dagger)$.
- Following Eq. (7.5), in the unnumbered equation for the one-plaquette ground state energy the integral within braces should have a coefficient of $-\frac{1}{6}$, not $+\frac{1}{6}$.

- In the unnumbered expressions for the curvature of the Hamiltonian toward the end of Sec. 7.1, the upper limit on the final summation over q in the last term of the result for $\langle [[E^2, \phi_m], \phi_n] \rangle$ should be n , not p , followed by $(2 - \delta_{qn})$, instead of $(2 - \delta_{qp})$, so the complete expression reads:

$$\begin{aligned}
\langle [[E^2, \phi_m], \phi_n] \rangle &= 2m \{ (m + 2n) \langle E x^{m-n} E \rangle + (-m + 2n) \langle E x^{m+n} E \rangle \} \\
&\quad - 2m \sum_{p=1}^n (2 - \delta_{pn}) p \langle x^{n-p} \rangle (\langle x^{m+p} E \rangle + \langle x^{-m+p} E \rangle) \\
&\quad - \sum_{p=1}^m (2 - \delta_{pm}) p \langle x^{m-p} \rangle \{ (n - p) \langle x^{n+p} E \rangle + (n + p) \langle x^{-n+p} E \rangle \} \\
&\quad + \sum_{p=1}^m (2 - \delta_{pm}) p (m - p) (\langle x^{m+n-p} \rangle - \langle x^{m-n-p} \rangle) \langle x^p E \rangle \\
&\quad - \frac{1}{2} \sum_{p=1}^m (2 - \delta_{pm}) p \sum_{q=1}^n (2 - \delta_{qn}) q \langle x^{m-p} \rangle \langle x^{n-q} \rangle (\langle x^{p+q} \rangle - \langle x^{p-q} \rangle).
\end{aligned}$$

- In the unnumbered equation for the Lagrange bracket at the end of Sec. 7.1, an overall factor of m is missing in the final expression.

References

- [1] L. G. Yaffe, “Large N limits as classical mechanics,” *Rev. Mod. Phys.* **54**, 407 (1982).
- [2] F. R. Brown and L. G. Yaffe, “The coherent state variational algorithm: A numerical method for solving large- N gauge theories,” *Nucl. Phys. B* **271**, 267-332 (1986).
- [3] T. A. Dickens, U. J. Lindqwister, W. R. Somsy and L. G. Yaffe, “The coherent state variational algorithm. 2. Implementation and testing,” *Nucl. Phys. B* **309**, 1 (1988).
- [4] L. G. Yaffe, “Resurrecting the coherent state variational algorithm for large N gauge theories,” *arXiv:2601.20150 [hep-lat]*.
- [5] F. R. Brown, “Coherent state variational methods for large N gauge theories: numerical calculations and strong coupling expansions,” Princeton Univ. Ph.D. theses, 1987, UMI-87-24799.
- [6] U. J. Lindqwister, “Numerical studies of large N lattice gauge theories,” Princeton Univ. Ph.D. thesis, 1988, UMI-88-09316.
- [7] T. A. Dickens, “Numerical studies of fermionic field theories at large N ,” Princeton Univ. Ph.D. thesis, 1987 UMI-87-16885.
- [8] W. R. Somsy, “The coherent state variational algorithm and the QCD deconfinement phase transition,” Princeton Univ. Ph.D. thesis, 1989.
- [9] J. B. Kogut and L. Susskind, “Hamiltonian Formulation of Wilson’s Lattice Gauge Theories,” *Phys. Rev. D* **11**, 395-408 (1975)
- [10] K. G. Wilson, “Confinement of quarks,” *Phys. Rev. D* **10**, 2445-2459 (1974).
- [11] D. J. Gross and E. Witten, “Possible third order phase transition in the large N lattice gauge theory,” *Phys. Rev. D* **21**, 446-453 (1980).
- [12] S. R. Wadia, “ $N = \infty$ phase transition in a class of exactly soluble model lattice gauge theories,” *Phys. Lett. B* **93**, 403-410 (1980).
- [13] A. Jevicki and B. Sakita, “Loop space representation and the large N behavior of the one plaquette Kogut-Susskind Hamiltonian,” *Phys. Rev. D* **22**, 467 (1980).
- [14] H. Neuberger, “Nonperturbative contributions in models with a nonanalytic behavior at infinite N ,” *Nucl. Phys. B* **179**, 253-282 (1981).
- [15] F. Berruto, E. Coletti, G. Grignani and P. Sodano, “Strongly coupled ’t Hooft model on the lattice,” *Nucl. Phys. B Proc. Suppl.* **83**, 685-687 (2000) [*arXiv:hep-lat/9909027*].
- [16] F. Berruto, L. Giusti, C. Hoelbling and C. Rebbi, “A study of the ’t Hooft model with the overlap Dirac operator,” *Phys. Rev. D* **65**, 094516 (2002) [*arXiv:hep-lat/0201010*].

- [17] J. Kiskis, R. Narayanan and H. Neuberger, “Proposal for the numerical solution of planar QCD,” *Phys. Rev. D* **66**, 025019 (2002) [arXiv:hep-lat/0203005].
- [18] M. García Pérez, A. González-Arroyo, L. Keegan and M. Okawa, “t Hooft model on the Lattice,” *PoS LATTICE2016*, 337 (2016) [arXiv:1612.07380 [hep-lat]].
- [19] Intel[®] oneAPI Threading Building Blocks Developer Guide and API Reference.
- [20] W.H. Press, S.A. Teukolsky, W.T. Vetterling, and B.P. Flannery, “Numerical Recipes in C: The Art of Scientific Computing,” 3rd ed., Cambridge University Press (2002), pgs 714–718.
- [21] P. Bogacki, L.F. Shampine, “A 3(2) pair of Runge-Kutta formulas,” *Applied Mathematics Letters*, Volume 2 (1989), pgs 321–325.
- [22] M. Sofroniou, G. Spaletta, “Construction of explicit Runge-Kutta pairs with stiffness detection,” *Mathematical and Computer Modelling*, Volume 40 (2004), pgs 1157–1169.
- [23] J.R. Dormand, P.J. Prince, “A family of embedded Runge-Kutta formulae,” *Journal of Computational and Applied Mathematics*, Volume 6 (1980), pgs 19–26.
- [24] P. Bogacki, L.F. Shampine, “An efficient Runge-Kutta (4,5) pair,” *Computers & Mathematics with Applications*, Volume 32 (1996), pgs 15–28. (See also this document.)
- [25] J.C. Butcher, “Numerical Methods for Ordinary Differential Equations,” 3rd ed., Wiley (2016), pg 225.
- [26] J.H. Verner, “Numerically optimal Runge–Kutta pairs with interpolants,” *Numerical Algorithms* 53, (2010), pgs 383–396.
- [27] A. S. Kronfeld, “Lattice gauge theory with staggered fermions: how, where, and why (not),” *PoS LATTICE2007*, 016 (2007) [arXiv:0711.0699 [hep-lat]].